学校代码：10284

分 类 号：TP316.4

密　　级：公开

U　D　C：004.75

学　　号：522022330004



南京大学

硕 士 学 位 论 文

论 文 题 目　云原生容器环境下的 AI 模型推理缓存池优化研究

作 者 姓 名　陈启明

专业学位类别（领域）　工程硕士（计算机技术）

研 究 方 向　大数据与云计算

指 导 教 师　黄宜华 教授、顾荣 副教授

2025 年 5 月 30 日

答辩委员会主席 李 武 军 教 授

评 阅 人 李 武 军 教 授

朱 琨 教 授

论 文 答 辩 日 期 2025 年 5 月 19 日

研究生签名：

导 师 签 名：



南京大学申请硕士学位论文

云原生容器环境下的 AI 模型推理缓存池优化研究

作者：　陈启明

专业：　计算机技术

研究方向：　大数据与云计算

指导教师：　黄宜华 教授、顾荣 副教授

南京大学计算机科学与技术系

2025 年 5 月



Model Cache Pool Optimization for AI Inference in Cloud-Native Container Envirnment

Presented By

Qiming Chen

Supervised by

Professor Yihua Huang, Associate Professor Rong Gu

A DISSERTATION
FOR THE APPLICATION OF MASTER DEGREE
SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE
AND TECHONOLOGY OF NANJING UNIVERSITY

May 2025

南京大学学位论文原创性声明

本人郑重声明，所提交的学位论文是本人在导师指导下独立进行科学研究工作所取得的成果。除本论文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写过的研究成果，也不包含为获得南京大学或其他教育机构的学位证书而使用过的材料。对本文的研究做出重要贡献的个人和集体，均已在论文的致谢部分明确标明。

本人郑重声明愿承担本声明的法律责任。

签名：

日期：

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目：云原生容器环境下的 AI 模型推理缓存池优化研究

计算机技术 专业 2022 级硕士生姓名： 陈启明

指导教师（姓名、职称）： 黄宜华 教授、顾荣 副教授

近年来，以深度学习和大模型为代表的新一代人工智能技术飞速发展。人工智能模型（AI 模型）被广泛应用在数据分析、推荐系统、图像识别、内容生成、聊天机器人等领域。模型推理是释放人工智能技术在行业应用价值的重要途径，因此当前越来越多行业在加速部署构建 AI 模型推理服务应用。然而，AI 模型推理负载的实时性要求高且依赖强大的计算能力，因此在资源开销和推理性能方面具有低成本、高吞吐、低延迟方面的要求。

由于云原生应用开发范式具有运维便捷、资源弹性、按需付费等优点，为了满足 AI 模型推理负载在资源开销和性能上的要求，在云平台上部署容器化的 AI 推理服务正在成为一种趋势。弹性容器服务是一种新颖的云原生应用部署方式，用户无需管理底层服务器，只需要提供打包好的容器镜像，自定义容器规格，即可运行容器，并按照运行时长与容器规格进行付费。然而，云原生容器环境下通常采用存储-计算分离的架构，这导致了以下两个问题：首先，云上异构存储系统使得 AI 模型数据被存储在不同数据源上，导致推理引擎访问数据的方式复杂，模型存储难以统一管理；其次，存算分离的架构会导致跨网络数据传输，AI 模型加载的高延迟会加剧推理服务冷启动问题，从而影响端到端的推理时延。

现有关于云原生 AI 推理的相关工作主要关注推理服务执行阶段时延、吞吐量以及金钱成本优化。然而，这些工作缺少对 AI 推理服务启动阶段性能与成本的联合优化。为了解决云原生容器环境下 AI 模型推理服务启动慢的问题，本文基于分布式缓存池架构，对模型加载过程的性能与成本进行联合优化。具体地，通过研究 AI 模型推理缓存池优化方法，对模型缓存预热阶段缓存数据的统一访问、模型加载阶段缓存池资源配置成本和服务质量的平衡、以及模型服务运行阶段的模型缓存弹性伸缩三个方面进行了优化，设计并实现了云原生容器环境下 AI 模型推理缓存池编排系统。本文的主要研究工作和贡献点包括：

（1）针对云原生容器环境下模型缓存池统一管理困难的问题，本文首先提出了模型缓存池的统一管理方法，以标准、统一、透明的方式管理 AI 模型缓存。其次，针对 AI 模型推理服务启动过程的预热时间占比大的问题，设计了一种近

似数据结构进行推理请求访问频率的轻量实时精准统计，并提出了推理请求动态感知的缓存预热策略，有效降低模型预热时间。

（2）针对云原生容器环境下模型缓存池资源配置成本和服务质量难平衡的问题，本文提出了面向最低成本的模型缓存池资源配置算法与短路读优化策略。首先对模型加载时间进行建模，提出了缓存池资源感知的模型加载时间预估方法。其次，对模型缓存池资源配置问题进行形式化定义，将其描述为约束优化问题，并提出了高效的分段序列二次规划算法对该优化问题进行求解。最后，从云厂商的角度，进一步通过容器的亲和性调度实现分布式缓存的短路读优化策略，进一步降低模型加载时间。

（3）针对云原生容器环境下静态模型缓存难以适应动态变化的推理负载的问题，本文提出了面向多任务动态场景的模型缓存池弹性伸缩机制。首先将推理任务与缓存资源的匹配转化为二分图最大权匹配问题进行求解，实现了缓存资源的复用。进一步地，提出了模型加载时间感知的模型缓存弹性扩缩容算法，通过动态调整容器实例数量，在满足服务质量的前提下实现资源的高效利用。

（4）基于上述研究，设计并实现了云原生容器环境下的 AI 模型推理缓存池编排系统。该系统构建了统一的模型缓存池，可根据 SLO 约束推荐合适的缓存池配置，并根据负载的动态变化进行资源扩缩容，实现了低时延、低成本的模型服务启动过程。与前沿方法相比，首先，本文提出的缓存预热方法将 AI 模型缓存预热总时间平均降低 25.98%；其次，本文提出的缓存池资源配置算法将模型加载成本最多降低 23.62%，将参数搜索效率提升 25 倍，短路读优化策略将模型加载时间进一步降低 38.52%；最后，本文提出的缓存弹性扩伸缩机制能有效适应推理负载的动态变化。

关键词：云原生；容器；AI 推理；模型加载；缓存系统

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Model Cache Pool Optimization for AI Inference in Cloud-Native Container Envirnment

SPECIALIZATION: Computer Technology

POSTGRADUATE: Qiming Chen

MENTOR: Professor Yihua Huang, Associate Professor Rong Gu

Recent advancements in artificial intelligence (AI), particularly deep learning and large models, have led to widespread use across data analysis, recommendation systems, image recognition, content generation, and chatbots. Efficient model inference is critical to unlocking the value of AI in industrial applications, with many industries accelerating the deployment of AI infercne services. However, these services face stringent real-time demands and high computational requirements, necessitating low-cost, high-throughput, and low-latency solutions.

Cloud-native application development, with its ease of maintenance, scalability, and pay-as-you-go model, has emerged as a popular solution for AI inference. Elastic container service, in particular, allows users to deploy AI models without managing underlying infrastructure, offering flexible resource allocation and cost efficiency. However, the cloud-native environment usually adopts a storage-computing separation architecture, which leads to the following two problems: First, the heterogeneous storage system on the cloud makes AI model data to be stored on different data sources, which makes it difficult for the inference engine to accesse the data in a unified way. Second, the storage-computing separation architecture leads to cross-network data transmission, and the high latency of AI model loading leads to the cold start issue.

Existing related work on cloud-native AI infernce focuses on optimizing the inference latency, throughput, and monetary cost of reasoning. However, little attention has been given to jointly improving the startup performance and cost efficiency. To address the problem of slow startup of AI model inference service in cloud-native container environments, this thesis proposes an efficient model cache pool optimizing method for AI inference in cloud-native container envirnment. The contributions can be summarized as follows:

(1) Aiming at the problem of the difficulty of unified management of model cache

pools under cloud-native container environment, this thesis designs a standardized model cache management framework and a lightweight data structure for real-time request frequency tracking, enabling a dynamic cache warm-up strategy that reduces the warm-up delays.

(2) Aiming at the problem of difficulty to balance the cost and quality of service of model cache pool resource configuration in cloud-native container environment, this thesis proposes a model cache pool resource configuration algorithm and short-circuit read optimization strategy to minimize model loading cost. First, this thesis models the loading time based on cache pool resource. Second, the thesis proposes an optimization problem and the segmented sequence quadratic programming algorithm to solve it. Finally, the thesis proposes a short-circuit read optimization strategy to further reduce the model loading time from the perspective of cloud vendors.

(3) Aiming at the problem that static model cache in cloud-native container environment is difficult to adapt to dynamically changing inference load, this thesis proposes a model cache pool elastic scaling mechanism. First, the thesis proposes a bipartite graph-based approach to match the inference tasks with cache resources for efficient reuse. Second, a loading-time-aware cache elastic scaling algorithm is proposed to balance service quality and resource efficiency.

(4) Based on the above research, this thesis implements an efficient model cache pool orchestration system for AI inference in cloud-native container envirnment. The system constructs a unified model cache pool, optimizes configurations based on SLO constraints, and dynamically scales the cache resources. Experimental results show that the approach reduces AI model cache warm-up time by 25.98%, cuts model loading costs by up to 23.62%, improves parameter search efficiency by 25×, further lowers loading time by 38.52% through short-circuit read optimization. and finally scales the cache resourcethe yo adapt to the dynamic change of inference load.

Keywords: Cloud-Native; Container; AI Inference; Model Loading; Cache System

目　录

第一章　绪论.....1
1.1 研究背景.....1
1.2 相关工作与不足.....2
1.2.1 云原生 AI 推理服务快速就绪.....2
1.2.2 云原生 AI 推理服务资源分配.....5
1.2.3 云原生 AI 推理服务弹性伸缩.....6
1.3 研究难点问题.....7
1.4 本文主要研究工作.....9
1.5 本文的组织结构.....10
第二章　背景知识.....11
2.1 云原生与弹性容器服务.....11
2.2 云原生 AI 推理服务.....12
2.3 分布式缓存.....13
第三章　云原生推理平台模型缓存池统一管理方法.....15
3.1 问题背景与分析.....15
3.2 云原生环境的模型缓存抽象与管理.....17
3.2.1 AI 模型缓存抽象.....17
3.2.2 AI 模型缓存池缓存状态管理.....18
3.3 推理请求动态感知的模型缓存池预热策略.....19
3.3.1 轻量级访问频率实时统计方法.....20
3.3.2 面向缓存收益最大化的模型缓存预热方法.....22
3.4 实验评估与分析.....23
3.4.1 实验环境与配置.....23
3.4.2 模型缓存池有效性评估.....26
3.4.3 模型缓存池开销评估.....27
3.4.4 模型缓存池预热策略的有效性评估.....28
3.5 本章小结.....29
第四章　面向最低成本的模型缓存池资源配置与短路读优化.....31
4.1 问题背景与分析.....31

4.2 缓存池资源感知的模型加载时延预估方法.....34
4.2.1 模型加载时间建模.....34
4.2.2 模型加载时间回归分析.....35
4.3 模型缓存池资源配置算法与短路读优化策略.....37
4.3.1 问题定义与形式化.....37
4.3.2 基于分段序列二次规划的模型缓存池资源配置算法.....39
4.3.3 短路读优化策略.....44
4.4 实验评估与分析.....45
4.4.1 实验环境与配置.....45
4.4.2 模型加载时延预估方法准确性评估.....47
4.4.3 模型缓存池资源配置有效性评估.....47
4.4.4 模型缓存池资源配置算法搜索效率对比.....50
4.4.5 短路读优化有效性评估.....51
4.5 本章小结.....52
第五章 面向多任务动态场景的模型缓存池弹性伸缩机制.....53
5.1 问题背景与分析.....53
5.2 基于二分图最大权匹配的模型缓存资源分配策略.....55
5.2.1 问题定义与形式化.....55
5.2.2 分配策略算法描述.....56
5.3 模型加载时间感知的缓存弹性扩缩容算法.....57
5.3.1 RPS 感知的推理服务容器水平伸缩方法 58
5.3.2 模型缓存弹性扩缩容算法.....59
5.4 实验评估与分析.....60
5.4.1 实验环境与配置.....60
5.4.2 模型缓存分配策略有效性评估.....61
5.4.3 模型缓存池弹性伸缩机制有效性评估.....62
5.5 本章小结.....64
第六章 系统设计与实现.....65
6.1 系统架构.....65
6.2 组件实现与运行流程.....66
6.2.1 分布式缓存池.....66

6.2.2 缓存池配置优化器.....68
6.2.3 模型缓存扩缩容组件.....69
6.3 系统运行示例.....69
6.4 本章小结.....74
第七章 总结与展望.....75
7.1 本文工作总结.....75
7.2 下一步工作.....76
参考文献.....79
致 谢.....85
附 录.....87

插图清单

图 2.1 KServe 控制平面架构[32].....13
图 2.2 Alluxio 分布式缓存的架构[37].....14
图 3.1 AI 推理服务的不同数据源及其协议.....15
图 3.2 AI 模型推理服务各阶段占端到端时延的比例.....16
图 3.3 推理应用通过模型缓存池的 DataModel 读取不同数据源中模型.....18
图 3.4 AI 模型缓存池缓存状态管理流程.....19
图 3.5 Count-Min Sketch 近似数据结构示意图.....20
图 3.6 带有时间窗口的 Count-Min Sketch 结构.....21
图 3.7 推理请求动态感知的模型缓存池预热策略整体流程.....23
图 3.8 使用不同缓存介质的 DataModel 加载模型和对象存储加载模型的加载时间对比.....26
图 3.9 本地直接挂载与 DataModel 挂载方式的模型加载时间对比.....27
图 3.10 缓存预热策略的缓存命中率与模型预热时间评估.....28
图 4.1 云原生容器环境下 AI 推理服务从模型缓存池中读取模型的过程.....32
图 4.2 容器数据访问带宽以及云盘类型对模型加载时间的影响.....32
图 4.3 单个云盘缓存量和短路读因素对模型加载时间的影响.....33
图 4.4 stable-diffusion 模型的加载时间与数据访问吞吐量关系的散点图.....36
图 4.5 取对数后模型加载时间与数据访问吞吐量之间的关系.....36
图 4.6 短路读优化策略.....44
图 4.7 不同数据访问吞吐量下 stable-diffusion-v2-1 模型加载时间的预测值与真实值.....47
图 4.8 不同数据集下不同资源配置搜索算法对应的金钱成本.....48
图 4.9 不同数据集下不同资源配置搜索算法对应的模型加载时间违反率.....49
图 4.10 不同资源配置搜索算法下的平均参数搜索时间.....50
图 4.11 不同模型下短路读策略对模型加载时间的影响.....51
图 5.1 NYS Thrway 数据集负载变化.....54
图 5.2 多任务动态场景下的容器扩缩容.....54
图 5.3 HPA 自动扩缩容机制.....58
图 5.4 使用/不使用缓存分配策略下的推理服务启动时间对比.....61
图 5.5 推理服务容器数与 RPS 随时间的变化.....62

图 5.6 缓存容器数随推理服务容器数的变化.....63
图 6.1 云原生容器环境下 AI 模型推理缓存池编排系统架构图 65
图 6.2 分布式缓存池控制器运行流程.....67
图 6.3 缓存池配置优化器运行流程.....68
图 6.4 模型缓存扩缩容组件运行流程.....69
图 6.5 opt-2.7b 加载时间回归模型参数 70
图 6.6 缓存池配置优化器返回最优容器与云盘配置.....70
图 6.7 描述 DataModel 的 YAML 文件 71
图 6.8 描述 Cache Runtime 的 YAML 文件 72
图 6.9 模型预热成功日志.....72
图 6.10 模型推理结果.....73
图 6.11 模型推理容器内日志.....73

表格清单

表 4-1 优化问题中各符号的含义 38
表 4-2 阿里云 ESSD 云盘参数 45
表 4-3 模型加载时延预估方法在不同模型上的拟合效果 47

第一章　绪论

1.1 研究背景

如今，以深度学习（Deep Learning，DL）和大模型为代表的人工智能技术正在飞速发展，人工智能模型（AI 模型）在众多领域得到了广泛的应用，例如数据分析[1]、推荐系统[2]、图像识别[3]、内容生成[4]、聊天机器人[5]等领域。深度学习应用包含模型训练和模型推理两个阶段，其中模型推理阶段可以给各行各业产生直接价值。近年来大语言模型（LLM，Large Language Model）展现出的强大推理能力，促使越来越多的行业加速部署构建 AI 模型推理服务。然而，AI 模型推理负载的运行依赖于强大的计算能力，在资源开销和推理性能方面具有低成本、高吞吐、低延迟方面的要求。

近年来，得益于云原生（Cloud Native）应用开发范式运维便捷、资源弹性、按需付费等优点，为了满足 AI 模型推理负载在资源开销和性能上的要求，在云平台上部署 AI 推理服务正在成为一种趋势。在云原生环境中，应用负载运行所依赖的所有文件被打包到容器镜像中，应用负载则以容器的形式运行在相对隔离的执行环境中。容器编排框架（如 Kubernetes[10]）负责将应用负载调度到某个特定节点上运行，在负载波动的情况下，容器编排框架通过对应用负载的弹性伸缩实现自动化运维。

然而，传统的云原生开发范式仍然要求开发者维护基础设施以及 Kubernetes 集群。为了帮助开发者更加关注于业务开发而免于基础设施运维，各大云厂商推出了弹性容器服务产品，例如 AWS Fargate[7]、Google Cloud Run[8]、阿里云 ECI[9]等产品。在这类容器服务中，用户无需维护服务器节点，仅提供容器镜像即可完成容器服务的部署，且完全兼容云原生中的 Kuberenetes 生态，具有无需运维、秒级伸缩、按需付费等特点。因此，在云平台下部署 AI 推理服务正在成为一种趋势[11]，为低成本、高性能的 AI 推理服务带来了新的机遇。

为了实现服务的灵活部署，云原生容器环境采用了存储-计算分离的架构。然而这种架构会带来以下两个问题：首先，云上异构存储系统使得 AI 模型数据被存储在不同的数据源上，导致推理引擎访问数据的方式复杂，模型存储统一管理难；其次，模型加载的时间往往是推理执行时间的数倍，模型加载过程通常会成为模型推理过程中的瓶颈[18]，而存算分离的架构会导致跨网络的数据传输，

AI 模型加载的高延迟加剧了推理服务冷启动问题，从而影响端到端的推理时延。

为了加速推理服务的启动，一种常见的方式是使用分布式缓存对分布在不同存储数据源 AI 模型进行预取，构建具有统一访问接口的模型缓存池，提供数据读取加速能力。使用分布式缓存加速 AI 模型加载有以下两个优势：其一，可以通过充分利用集群内部的带宽实现数据读取的加速；其二，在高并发推理场景下，分布式缓存的多副本机制，可以有效缓解大量容器实例同时加载模型时产生的 I/O 性能瓶颈。

在云原生容器环境下，分布式缓存的各个组件均以容器化的方式部署，为了进一步加速模型加载，通常可使用云平台提供的按量付费的高性能云盘作为缓存存储介质。然而，云平台提供的弹性容器与云盘计费粒度精细且配置组合种类多，推理服务的性能与资源配置相关，开发者难以选择恰当的配置实现金钱成本与服务质量之间的权衡。

在上述背景下，本文对模型缓存预热阶段缓存数据的统一访问、模型加载阶段缓存池资源配置成本和服务质量的平衡、以及模型服务运行阶段模型缓存弹性伸缩三个方面进行了优化，提出了云原生容器环境下的 AI 模型推理缓存池优化方法。首先，针对 AI 模型缓存数据源多样导致的模型数据统一管理困难的问题，本文提出了模型缓存池统一管理的方法。其次，针对模型缓存资源配置成本和 AI 推理任务的服务质量难以权衡的问题，提出了面向最低成本的模型缓存池资源配置算法和短路读策略。最后，由于多任务动态场景下静态缓存资源难以适应动态负载变化，本文提出了模型缓存资源分配的策略以及模型缓存池弹性伸缩机制。结合以上内容，本文设计并实现了云原生容器环境的 AI 模型推理缓存池编排系统，兼具低延迟与低成本的优点。

1.2 相关工作与不足

为了实现低延时、低成本、高吞吐云原生 AI 推理服务，研究者围绕多个方向开展了大量云原生 AI 推理相关的研究工作，包括 AI 推理服务快速就绪[12]-[19]、资源分配[20]- [24]、弹性伸缩[25]-[29]等方面。本节对相关工作进行梳理，总结其核心思想、关键创新与不足。

1.2.1 云原生 AI 推理服务快速就绪

服务器无感知计算[6]（Serverless Computing）是云原生下的一种典型开发范

式。该类工作主要解决 Serverless AI 推理服务的冷启动问题，其实现路径主要包括资源分配优化和模型加载优化。其中，资源分配优化侧重于从云函数和容器资源合理分配的角度加速推理服务的启动；模型加载优化侧重于优化模型的加载和传输机制。下面从资源分配优化与模型加载优化两个方面介绍相关工作。

（1）资源分配优化

资源分配优化指通过对问题建模与理论化求解，通过为云函数或容器合理分配资源的方式，实现冷启动的优化。

J San Juan[12]等人主要关注云原生 Serverless GPU 推理服务中的冷启动问题。该方法利用系统中未充分利用的内存和网络资源，并采用分层存储的方式存储模型以减少模型下载时间，并且通过亲和性感知的调度方法方法平衡资源利用率与冷启动时延。然而该方法未考虑云平台计费模型，缺少对金钱成本的优化。

Kongyangge Zhao[13]等人提出了边缘计算场景下 AI 推理的冷启动解决方案。该方法通过动态调整云函数的配置和推理质量，实现推理延迟与推理质量（精度）的平衡。该方法使用正则化技术，通过平滑地调整云函数的数量以减少云函数冷启动的次数，然而该方法建立在边缘计算场景的背景上，即边缘设备部署轻量级模型，云端部署高精度模型，该场景在云原生容器环境下并不成立。

PLAYS[14]是一种基于优先级的层级调度算法，其主要思想是考虑 DNN 层之间的依赖关系、容器启动时间与资源需求，优先调度资源需求较大的层，最小化容器启动时间。PLAYS 将问题建模成了非线性整数规划问题进行求解，并通过对 PLAYS 算法的理论分析，推导了推理时间的理论上界。然而使用非线性整数规划求解问题的复杂度较高，在动态负载场景下的计算开销较大。

Wang[15]设计了一种 Serverless 环境下 AI 推理数据编排系统，该系统使用分布式缓存对模型加载过程进行加速，并使用序列二次规划算法推荐分布式缓存资源配置，在满足 SLO 的前提下实现最小金钱成本。该系统构建了模型加载时间与数据访问带宽之间的数量关系，并对模型加载阶段引入的分布式缓存金钱成本与模型加载时间进行了联合优化，然而该方法未解决云原生环境下模型数据源异构导致的数据访问方式复杂的问题，缺少对模型数据的统一管理。

（2）模型加载优化

该部分介绍通过模型加载优化的方式缓解 Serverless AI 推理服务冷启动问题的相关研究工作。

ServerlessLLM[16]是一个针对大型语言模型（LLM，Large Language Model）

的 Serverless 推理系统。为了解决 LLM checkpoint 数据量大导致的模型加载耗时的问题，该系统首先设计了一种新的 checkpoint 格式以支持高效内存寻址，接着基于 GPU 服务器的多层级存储，充分利用各个存储介质（如 GPU 显存、DRAM、SSD）的带宽以加速 checkpoint 的加载过程，最后利用本地存储的 checkpoint 实现推理任务的平滑迁移。该方法从系统层面上优化了 LLM 的加载过程，但是缺乏云原生容器环境下推理性能与金钱成本的联合优化，且没有解决云原生环境下存算分离架构导致的模型统一管理难，访问方式复杂的问题。

Medusa[17]通过状态物化（State Materialization）的方式缓解 Serverless 环境下 LLM 推理的冷启动问题。LLM 推理在模型加载时会引入 CUDA 图构建和 KV Cache 初始化的阶段，该方法在离线阶段物化 CUDA 图和 KV Cache 初始化所需的信息，并且在在线推理阶段恢复这些状态，从而避免了在线动态分析和构建的开销，从而降低了模型加载时间。然而离线物化存储会占用大量额外的存储空间，且需要在离线阶段为每一种 LLM 模型构建物化 CUDA 图，难以满足云上推理的实时性要求。

Optimus[18]通过云函数之间的模型转换加速 Serverless AI 推理的预热，从而避免从头开始加载模型，以加速模型加载的过程。该系统通过利用空闲容器中已有的相似结构的模型来加速新模型的加载，并提出了五种模型转化的元操作符实现模型间的转换。最后，提出了一个高效的模型转换方法，以获得最优的模型转换策略，缓解模型加载导致的冷启动问题，降低端到端推理时延。然而 Optimus 主要关注基于 CNN（卷积神经网络）的模型转换，难以推广到基于 Transformer 架构的大模型上，具有较大的局限性。

Adiren Gallego[19]等人提出了一种将机器学习模型切分为多个较小子模型的方法，以加速模型的加载过程。通过将模型分解为多个切片，可以在云平台上以云函数的形式运行推理任务，降低内存的使用并缓解模型加载导致的冷启动问题。然而该方法提出的切片机制会引入额外的通信开销，从而影响推理时延。

综上所述，以上相关工作从资源分配或模型加载机制优化的角度，有效缓解了云原生环境下 AI 推理的冷启动问题，降低了端到端推理时延。然而，这些工作没有解决云原生环境下存算分离架构导致的模型统一管理难、数据访问方式复杂的问题，且缺少对推理服务启动阶段金钱成本与模型加载性能的联合优化。

1.2.2 云原生 AI 推理服务资源分配

该类工作关注云原生 AI 推理服务资源分配相关的问题，通过对容器实例资源分配以及合理调度，构建低成本、低延时的云原生推理服务。

SMIless[20]针对动态场景下基于 DAG（有向无环图）的推理服务进行资源分配的优化，通过自适应的预热窗口以及异构函数实例的 CPU 与 GPU 资源分配的协同优化，使用在满足 SLA（服务级别协议）要求的同时，显著降低了成本。其具体实现方式是考虑不同云函数之间的级联效应，使用 TopK 路径搜索方法，搜索到满足 SLA 要求且成本最小的预热窗口与函数实例配置。SMIless 方法使用 LSTM 网络预测推理请求的到达模式，根据推理请求间隔决定预热窗口的大小，在突发推理负载下难以准确决定正确的预热窗口。

INFless[21]是一个用于机器学习推理的 Serverless 平台，它提供了 CPU 各 GPU 异构资源抽象方法，并使用启发式调度方法，为每个函数实例分配合适的容器实例数量、batchsize 以及 CPU/GPU 配置，以达到最小的资源占用量。然而启发式的调度方法缺乏理论上的最优保证，且未考虑云平台计费模型，缺少对金钱成本的优化。

RIBBON[22]通过使用一组异构实例，在满足推理服务质量要求的前提下最大限度节约成本。其核心思想构是构建一个异构实例池，以更好地应对不同模型、服务质量目标和 batchsize 的变化。它采用了贝叶斯优化策略，快速寻找到最优的实例配置，最小化金钱成本。然而贝叶斯优化策略引入了较大的计算开销，参数搜索效率较低。

PESEI[23]提出了一种推理负载感知的主动弹性资源分配算法，优化推理服务的服务质量和成本。其核心思想是使用 LSTM 网络预测未来请求，然后根据预测请求率的均值和方差动态计算每个推理服务分配的内存和 CPU 资源，提前进行资源分配，降低的端到端推理时延。该方法使用 LSTM 方法预测推理请求的到达模式，在突发推理负载下难以合理分配内存与 CPU 资源。

HarmonyBatch[24]是一个成本高效的资源分配框架，通过异构 Serverless 函数实现多 SLO 的 DNN 推理。其具体思想是基于 GPU 时间分片和请求到达率分布构建 CPU 和 GPU 函数上的 DNN 推理性能模型和成本模型，并设计了两阶段合并策略，将多 SLO 的 DNN 推理请求合并处理，在保证 SLO 要求的前提下最小化函数供应成本。然而该方法提出的 GPU 时间分片方法会引入上下文切换开销，且缺乏对 Serverless 函数启动阶段模型加载过程的优化。

上述方法主要关注云原生推理服务容器或 Serverless 函数的资源分配与调度问题，对推理时延、金钱成本、资源利用率从等因素进行了联合优化，达到资源利用率最大化的目的。然而，这些工作缺少对云原生推理服务启动阶段，尤其是模型加载阶段的优化，忽略了大模型加载带来的时间开销。另外，这些工作主要集中于对云原生推理服务计算资源的优化，然而在大模型推理场景中通常利用缓存加速模型加载过程，这些工作并未考虑对存储或缓存资源的优化。

1.2.3 云原生 AI 推理服务弹性伸缩

在云环境下，推理实例为了适应负载变化通常会根据一定的策略进行适应性弹性扩缩容，该类工作关注云原生 AI 推理服务的弹性伸缩机制。

ElasticRec[25]是为推荐系统设计的微服务模型服务架构，基于 embedding table 访问热度不同的事实，它根据每个分片的部署代价，使用动态规划算法将推荐模型的 embedding table 分解为多个细粒度的冷/热分片，每个分片作为一个单独的微服务进行部署，这些分片可以根据对资源的需求独立地扩缩容，实现了资源的弹性伸缩，同时大幅度节省了内存资源。然而，该方法引入了模型分片之间的通信与协调的开销，且分片难以根据冷热变化自动化调整分片策略。

AsyFunc[26]是一种高性能资源高效的云原生 Serverless 推理系统，能有效应对突发推理负载。它根据 AI 模型各层对资源敏感度的不同，将推理函数分成 Body Function 和 Shadow Function 两种函数，其中 Body Function 加载完整模型，Shadow Function 加载模型中部分的资源敏感层。该方法提出了模型级的扩缩容算法，通过伸缩 Body Function 实例以面对稳定负载；提出了层级扩缩容算法，通过为 Body Function 匹配合适的 Shadow Function 实例，仅加载资源敏感层即可应对突发负载，满足推理任务的实时性能要求。然而，AsyFunc 系统 Body Function 与 Shadow Function 的划分原则是基于 AI 模型各层对资源敏感度差异，这种差异性仅仅体现在 CNN 模型上，难以适用于基于 Transformer 架构的大模型，故层级扩缩容算法难以在大模型上适用。

FaPES[27]旨在为云原生 Serverless 机器学习平台提供高效的弹性扩展能力。它将推理任务分配到专用的推理虚拟集群中，避免与训练任务造成资源竞争。FaPES 通过调整 GPU 资源实现扩缩容，当推理任务的需求增加时，系统可以从训练集群中动态借贷 GPU 资源，反之亦然。这种机制实现了推理任务与训练任务的协同优化，确保推理任务能够满足 SLO 要求，然而推理任务与训练任务的

资源竞争仍可能导致训练任务的 SLO 违反。

FaST-GShare[28]旨在为云原生 Serverless 推理实现高效的 GPU 共享机制。FaST-GShare 首先根据离线的性能建模为每个函数实例分配合适的 GPU 资源，接着基于函数的处理能力和预测的负载，利用启发式自动扩缩容算法动态调整函数实例数量，在保证 SLO 的同时提高 GPU 的利用率和吞吐量。该方法需要预先对不同资源分配下的函数性能进行测试，生成性能曲线，如果函数行为或输入数据分布变化较大（如 batchsize、模型结构变化），可能需要重新分析。

FLASH[29]是一个为云原生 Serverless 边缘推理设计的推理系统，它根据流量负载动态调整推理实例的 CPU 核数，请求负载增加时，系统会增加 CPU 核数以加速推理；当请求负载减少时，系统会减少 CPU 核数以释放资源。然而该方法依赖于 CPU 并行计算，但并非所有模型都能高效利用多核并行计算，例如串行依赖较强的 RNN（循环神经网络）模型。此外，虽然 FLASH 通过多核并行加载模型参数减少了冷启动时间，但模型加载仍受限于磁盘与内存带宽，在大模型推理场景下，冷启动可能会导致违反 SLO。

上述方法主要通过启发式地调整推理服务实例数量和计算资源（例如 CPU、GPU 资源）以实现弹性伸缩，以适应动态变化的推理负载。然而，在推理服务容器的首次启动时会经历模型加载的过程，弹性扩缩容通常会导致推理服务容器的变化，从而影响模型加载的并发量，进而影响模型加载时间。上述工作均建立在模型存储资源不变的假设上，虽然考虑了推理服务容器与计算资源的弹性扩缩容，但并未考虑相应模型存储资源的弹性伸缩以匹配计算资源的变化。

1.3 研究难点问题

针对现有现状与不足，本文提出了云原生容器环境下的 AI 模型推理缓存池优化方法，旨在优化 AI 模型推理启动阶段的性能与成本。本文面临以下云原生容器环境下模型缓存池统一管理困难、模型缓存池资源配置成本和服务质量难平衡、静态模型缓存难以适应动态变化的推理负载三个方面的挑战：

（1）云原生容器环境下模型缓存池统一管理困难。

云原生容器环境下，AI 推理任务提供服务的必要条件是从模型存储数据源加载 AI 模型。然而在云原生容器环境中存储与计算分离，模型推理服务往往需要通过网络从异构的多个模型存储源中加载 AI 模型，这会导致较高的推理服务启动时延。

为了加速模型加载的过程，一种常见的方式是使用分布式缓存对分布在不同存储数据源 AI 模型进行预取，以提供模型加载的加速能力。然而，模型存储的数据源种类多样，且各自采用不同的协议。由于不同数据源所提供的 API 不尽相同，开发者在编写程序时往往需要根据不同的数据存储源编写不同的代码，导致推理引擎访问数据的方式复杂，难以以统一的方式管理 AI 模型缓存。

（2）云原生容器环境下模型缓存池资源配置成本和服务质量难平衡。

本文采用分布式缓存实现 AI 模型缓存池，在云原生容器环境下，分布式缓存的各组件均以容器方式部署，而云平台提供的按需付费高性能云盘可作为缓存存储介质。然而，云环境下缓存池配置种类繁多，包括缓存容器数量、缓存容器带宽、缓存容器 vCPU 数量、缓存容器内存等配置、云盘类型、云盘容量、云盘吞吐量等配置。以阿里云平台为例[40][41]，容器 vCPU 取值为 1~64，容器内存取值为 2~512GB，云盘容量取值为 1~65536GB，云盘吞吐量取值最大为 4000MB/s，缓存容器数量可由开发者自行配置，参数搜索总空间级别过亿。另外，容器数据访问带宽与容器 vCPU 数量，云盘吞吐量与云盘容量之间均存在依赖关系。综上所述，这些配置值中既包含连续值又包含离散值，且变量之间存在依赖关系，形成了巨大的、高维度的参数搜索空间。

在这种背景下，模型推理服务的服务质量与缓存池资源配置直接相关，因此如何高效地选取合适的缓存池资源配置，在金钱成本和服务质量之间做出平衡是具有挑战性的。如果资源配置过少，则模型加载时间难以达到服务水平目标（SLO，Service Level Object），从而影响端到端推理时延；如果资源配置过多，可能导致资源浪费，造成较高的金钱成本。

（3）云原生容器环境下静态模型缓存难以适应动态变化的推理负载。

在多任务动态的 AI 模型推理场景下，推理负载通常会随着时间以及用户行为的变化而变化。当一批推理任务到来时，为了充分复用模型缓存池中空闲的缓存资源，需要为每个推理服务匹配合适的缓存资源以供复用，以加速推理服务的初始化过程；在推理服务运行过程中，推理服务容器实例会适应性地扩缩容以应对推理负载的动态变化。

然而，在这个过程中存在两个挑战点。首先，模型加载时间受到推理服务资源配置和缓存池资源配置的共同影响，需要为推理服务分配合理的缓存资源以实现推理服务启动时间最小化；其次，缓存池配置在初始化时设定，这种静态的缓存池配置难以适应变化的推理负载，如何根据动态负载对模型缓存进行合理的弹

性伸缩，在确保高服务质量的同时实现缓存资源的高效利用，也是一个亟需解决的难题。

1.4 本文主要研究工作

本文对云原生容器环境下的 AI 模型推理缓存池优化方法进行研究，设计并实现了高性能低成本的模型缓存池编排系统，旨在解决模型缓存池统一管理困难、缓存池资源配置成本与服务质量难平衡、静态模型缓存难以适应动态变化的推理负载等问题。本文的主要研究工作和贡献点包括：

（1）针对云原生容器环境下模型缓存池统一管理困难的问题，本文提出了模型缓存池的统一管理方法，以标准、统一、透明的方式管理 AI 模型缓存。在此基础上，针对 AI 模型推理服务启动过程的预热时间占比大的问题，提出了推理请求动态感知的缓存预热策略并使用近似数据结构降低系统开销，将缓存预热总时间平均降低 25.98%。

（2）针对云原生容器环境下模型缓存池资源配置成本和服务质量难平衡的问题，本文提出了面向最低成本的模型缓存池资源配置算法与短路读优化策略。首先对模型加载时间进行建模，提出了缓存池资源感知的模型加载时间预估方法。在此基础上，对模型缓存池资源配置问题进行形式化定义，将其描述为约束优化问题，并提出了分段序列二次规划算法对该成本优化问题进行求解，相比于主流方法，成本最多降低 23.62%，参数搜索效率提升 25 倍。最后，从云厂商的角度，基于缓存池资源配置进一步采取短路读优化策略，进一步将模型加载时间平均降低 38.52%。

（3）针对云原生容器环境下静态模型缓存难以适应动态变化的推理负载的问题，本文提出了面向多任务动态场景的模型缓存池弹性伸缩机制。首先将推理任务与缓存资源的匹配转化为二分图最大权匹配问题进行求解，实现了模型推理服务启动时间的最小化；进一步地，提出了推理服务容器水平伸缩方法和模型加载时间感知的模型缓存弹性扩缩容算法，在保证模型加载时间的前提下，通过动态调整容器实例的数量，实现资源的高效利用。

（4）基于上述研究，设计并实现了云原生容器环境下的 AI 模型推理缓存池编排系统。该系统构建了统一的模型缓存池，可根据 SLO 约束推荐合适的缓存池配置，并根据动态负载进行资源扩缩容，实现了低时延、低成本的模型推理服务启动过程。

1.5 本文的组织结构

本文的组织结构如下：

第一章，绪论。该章首先介绍研究背景、相关工作与不足；接着，分析云原生容器环境下的 AI 模型推理缓存池优化的研究难点问题；然后介绍本文的主要研究工作与贡献点；最后，介绍本文的组织结构。

第二章，背景知识。该章介绍与本文研究内容相关的背景知识，包括云原生与弹性容器服务、云原生 AI 模型服务以及分布式缓存三个部分。

第三章，云原生推理平台模型缓存池统一管理方法。该章首先介绍问题背景，接着提出了云原生容器平台的模型缓存抽象与管理方法，在此基础上，该章提出了推理请求动态感知的缓存池预热策略，最后对该章的工作进行实验评估和总结。

第四章，面向最低成本的模型缓存池资源配置与短路读优化。该章首先通过动机实验分析问题背景，其次提出了缓存池资源感知的模型加载时间预估方法，接着基于分段序列二次规划方法提出了模型缓存池资源配置算法与短路读优化策略，然后进行实验评估和对比实验，最后对该章内容进行总结。

第五章， 面向多任务动态场景的模型缓存池弹性伸缩机制。本章首先介绍并分析问题背景，接着提出了基于二分图最大权匹配的缓存资源分配策略，然后提出了模型加载时间感知的缓存弹性扩缩容算法，最后对该章的工作进行实验评估和总结。

第六章，系统架构与实现。该章首先介绍云原生容器环境下 AI 模型推理缓存池编排系统的总体架构，接着对每一个组件的实现与运行流程进行介绍，然后通过一个示例介绍系统的使用流程，最后对该章内容进行总结。

第七章，总结与展望。本章首先对本文的研究工作进行总结，接着对下一步研究工作进行展望。

第二章　背景知识

在介绍本文研究提出的方法之前，本章将对本文研究内容的相关背景知识和涉及的基本概念进行介绍。首先，2.1 节介绍云原生基础知识与弹性容器技术。接着，2.2 节介绍云原生 AI 推理服务的特点与典型框架。最后，2.3 节介绍分布式缓存基础知识与典型架构。

2.1 云原生与弹性容器服务

云原生是一套基于云计算环境的应用开发与运行范式，其核心理念包括不可变基础设施（通过容器镜像实现开发与运行环境的一致性）和云应用编排理论（如 Kubernetes[10]容器编排框架）。在云原生环境中，应用负载运行所依赖的所有文件被打包到容器镜像中，应用负载则运行在相对隔离的执行环境中。容器编排框架（如 Kubernetes）负责将应用负载调度到某个特定节点上运行，在容器调度过程中，容器编排工具会充分考虑集群资源情况、应用亲和性等因素做出合理的调度决策。

Kubernetes 是目前部署和操作容器化应用的通用标准 [1]。Kuberntes 中最小的调度单元被称为 Pod，Pod 由一个或一组紧密相关的容器构成，它被 Kubernetes 调度器统一调度到指定节点上运行。

虽然 Kubernetes 实现了运维自动化和弹性部署，但是开发者仍需要对 Kubernetes 所依赖的底层基础设施进行管理，仍有较大的运维成本。为了克服该问题，服务器无感知的弹性容器服务应运而生。其核心优势是用户无需管理底层服务器，只需要提供打包好的容器镜像，即可运行容器，且按照容器运行时间收费。各大云厂商均推出了弹性容器服务，例如 AWS Fargate[7]、Google Cloud Run[8]、阿里云 ECI[9]等产品。

传统的容器技术仅为应用负载提供相对隔离且轻量的执行环境，容器化应用的运行需要依赖底层的基础设施，容器的资源分配与调度依赖于 Kubernetes 等容器编排框架，容器的伸缩策略需要开发者手动配置。此外，在云平台上运行容器化应用负载需要开发者租赁 Kubernetes 集群，带来了运维底层基础设施的负担。

相较于传统的容器服务，弹性容器服务具有定制规格、弹性伸缩、无需运维

[1] https://cloud.google.com/learn/what-is-kubernetes

的特点。一方面，开发者可以任意设置 vCPU 数量以及内存大小自定义容器规格，仅需提供容器镜像，无需维护底层服务器，即可支持容器的秒级创建与扩缩容，云平台负责容器资源的调度和部署；另一方面，弹性容器服务可无缝集成 Kubernetes 等容器编排框架，可以作为 Pod 被 Kubernetes 管理调度。得益于弹性容器服务高度灵活、运维便捷、生态友好的特点，可被广泛应用在数据处理、机器学习、Web 服务等应用领域。

2.2 云原生 AI 推理服务

得益于云原生运维便捷、部署灵活、弹性伸缩等优点，在云原生环境下部署 AI 推理服务正在成为一种趋势，使得低成本、高性能的 AI 推理服务成为可能。为了让用户更便捷地在云原生环境下运行 AI 推理负载，KServe[30]、Seldon-core[31]等开源的云原生 AI 推理框架应运而生。这些 AI 推理框架通常建立在 Kuberenetes 上，简化了 AI 推理应用在云环境中的部署，具有运维方便、扩展性高、成本低廉等特点。接下来，以 KServe 为例介绍云原生 AI 推理服务的特点。

KServe 是一个基于 Kubernetes 的开源模型推理平台，专为云原生环境下的高效机器学习模型推理而设计，旨在简化机器学习模型的部署和管理，同时提供高性能和高扩展性。它广泛支持 TensorFlow、PyTorch、XGBoost、scikit-learn 等主流机器学习框架，同时也支持自动扩缩容能力支持 Serverless 推理的工作负载。

图 2.1 展示了 KServe 控制平面的架构。AI 推理请求首先通过 HTTP 或 gRPC 协议发送到 Transformer Service 进行预处理，接着 Transformer Service 通过 HTTP 或 gRPC 协议请求 Predictor Service 进行模型的推理，Storage Initializer 负责完成 AI 模型的初始化由完成，从远端存储数据源（例如对象存储、Hugging Face）下载 AI 模型。

在 Serverless 部署模式下，KServe 会监控流入应用程序的流量，并根据配置的指标（如并发请求数）自动调整 Pod 的副本数量，实现自动扩缩容的功能。当流量增加时，KPA（KNative Pod AutoScaler）会增加 Pod 副本以处理更多请求；当流量减少时，KPA 会缩减 Pod 副本。当没有推理请求流量时，会将 Pod 数量减少到 0 以节省计算资源。

在后续云原生 AI 推理服务的实现中，本文基于 Kubernetes 以及 KServe 的系统架构设计思想实现模型推理应用，并以 HTTP 协议的方式提供推理服务。

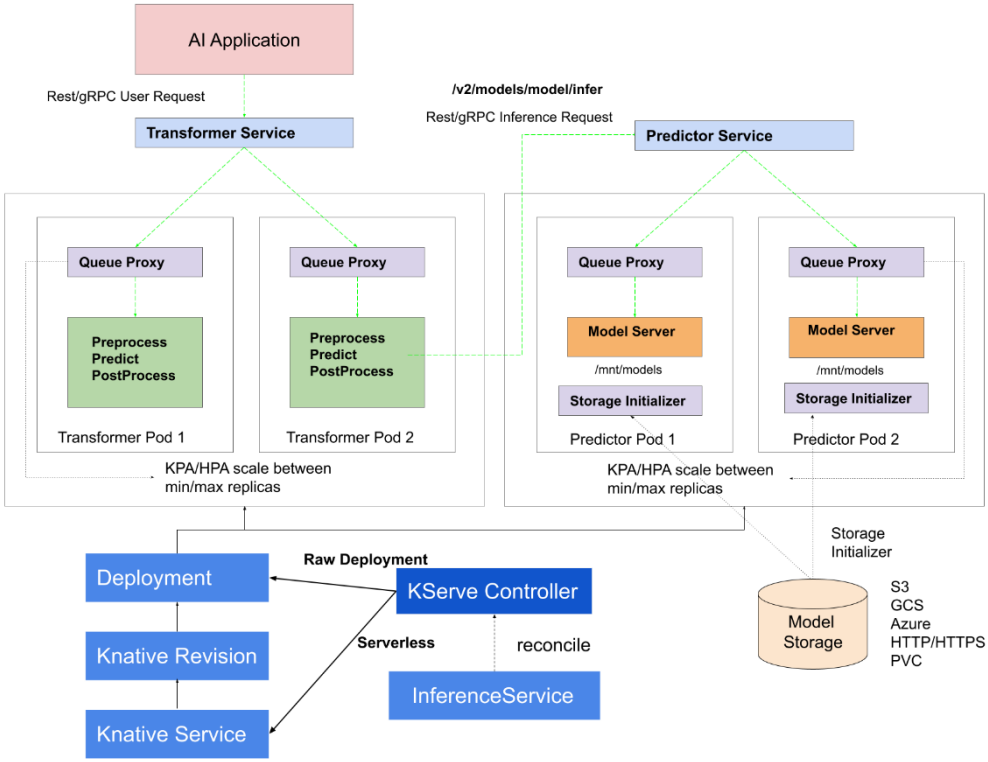


图 2.1 KServe 控制平面架构[32]

2.3 分布式缓存

分布式缓存是将数据存储在多个节点上的缓存系统，通过网络实现数据的共享与访问。与本地缓存不同，分布式缓存支持动态扩展节点以应对高负载需求。分布式缓存主要依靠高速存储介质（如内存、SSD）以及集群内的高速网络带宽实现数据读取的加速。

本小节以 Alluxio[32]为例介绍分布式缓存系统。分布式缓存系统往往采用主从架构（Master-Worker），如图 2.2 所示。一个 Alluxio 分布式缓存系统中包含一个缓存 Master 组件和多个缓存 Worker 组件。其中，缓存 Master 组件负责主要负责元数据管理和集群协调，比如文件系统的元数据，数据块的位置信息，数据块与 Worker 组件的映射关系，缓存文件与存储系统数据文件的映射关系等；缓存 Worker 组件负责数据存储，通过多级缓存将缓存数据存储在不同存储介质中进行分层管理，使用驱逐策略淘汰冷数据。

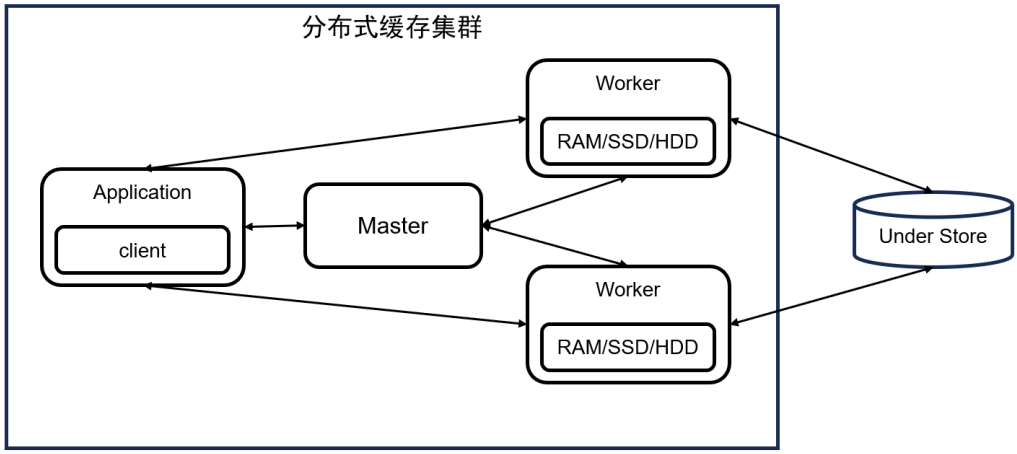


图 2.2 Alluxio 分布式缓存的架构[37]

当客户端读取数据时，存在三种情况：

（1）本地缓存命中。当客户端需要读取的数据被缓存在本地 Worker 上时，客户端首先向 Master 检索该数据在 Worker 上的位置，如果本地 Worker 上存在该数据，客户端将绕过 Worker，通过“短路读”方式直接从本地文件系统读取数据。

（2）远程缓存命中。当客户端需要读取的数据被缓存在远程 Worker 上时，客户端首先向 Master 检索数据在 Worker 上的位置，然后从远程 Worker 读取该数据，最后在本地 Worker 写入一个副本以便后续读取时可以命中本地缓存。在远程缓存命中的情况下数据读取速度可以达到本地网络的传输速度。

（3）缓存未命中。当了客户端请求的数据不在分布式缓存中，客户端必须先从存储系统中读取数据，然后将其缓存到缓存系统中，随后将数据放回给客户端。

第三章　云原生推理平台模型缓存池统一管理方法

云原生容器环境下模型数据源多样，导致上层 AI 推理服务难以以透明统一的方式读取 AI 模型以对外提供推理服务。针对云原生容器环境下模型缓存统一管理困难的问题，本章提出了云原生环境模型缓存池统一管理方法，以标准、统一、透明的方式管理 AI 模型缓存，并根据推理请求访问模式预热模型缓存以缓解推理服务的冷启动问题。本章的内容组织结构如下：3.1 节介绍本章的问题背景，3.2 节介绍云原生环境的 AI 模型缓存的抽象与管理方法，3.3 节介绍推理请求动态感知的模型缓存池预热策略，3.4 节对本章提出的方法和策略进行评估与分析，3.5 节对本章内容进行总结。

3.1 问题背景与分析

得益于云原生范式运维便捷、弹性伸缩、按需付费等显著优势，AI 推理服务正在逐步向云原生环境上迁移，例如 KServe、Seldon-core 等云原生环境下的推理平台。这些推理平台普遍建立于 Kubernetes 之上，通过对推理容器的水平伸缩，发挥云原生环境下极致弹性的优势。

然而，AI 模型推理服务容器首次启动时，需经历从模型存储数据源下载 AI 模型至本地，再将其加载至内存或 GPU 显存的步骤。在云原生容器环境下，由于存储-计算分离的架构，模型推理服务往往需要从异构的多个模型存储源中下载 AI 模型。如图 3.1 所示，这些数据源可能包括 webUFS、NFS、s3 对象存储、HDFS 分布式文件存储等，且各自采用不同的协议。由于不同数据源所提供的 API 不尽相同，开发者在编写程序时往往需要根据不同的数据存储源编写不同的代码，这不仅增加了开发复杂性，还降低了代码的可扩展性。

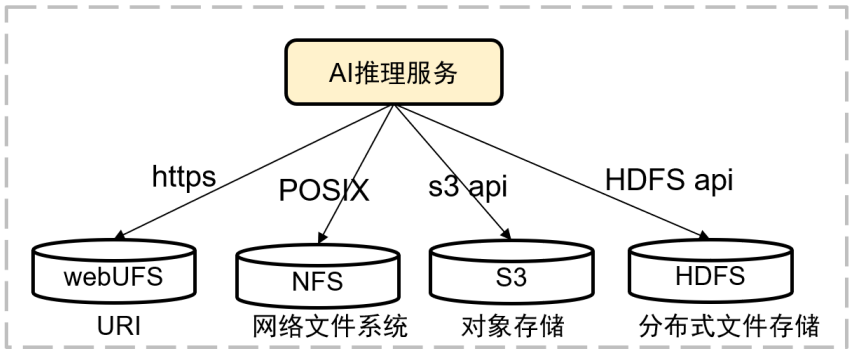


图 3.1 AI 推理服务的不同数据源及其协议

尽管部分云原生推理平台提供了对不同存储源的支持（比如 KServe 提供的 Storage Initializer 容器可以从不同的数据源下载并初始化模型），能一定程度上缓解上述问题，但仍然存在配置复杂的挑战。此外，现有的云原生推理平台普遍缺乏 AI 模型缓存的功能，导致新的推理服务容器启动时，仍然可能需要从远程数据源下载模型，引发冷启动问题。

为了深入阐述 AI 模型推理的冷启动问题，本文测量了三种模型 stable-diffusion-2-1[33]、opt-2.7b[34]、opt-6.7b 在模型推理不同阶段的耗时以及占比，模型的远程存储数据源均为支持 s3 协议的对象存储，模型缓存使用单 Worker 的 Alluxio 缓存系统实现，模型推理服务使用单副本的 KServe 服务，模型推理均在显存大小为 24GB 的 NVIDIA 3090 GPU 上进行。

动机实验结果如图 3.2 所示，AI 模型推理分为三个阶段：预热、加载、推理。其中，预热是指将 AI 模型从远程数据源下载到缓存系统中，加载是指将模型从缓存系统加载到本地内存或者 GPU 显存中以供推理。图 3.2 表明一次模型推理服务过程中，启动阶段占据端到端推理时延的 60%以上，模型预热阶段最多占据了端到端推理时延的 40%以上。由此可见，即使使用了缓存系统预热模型，预热阶段的时间占比仍然较大。

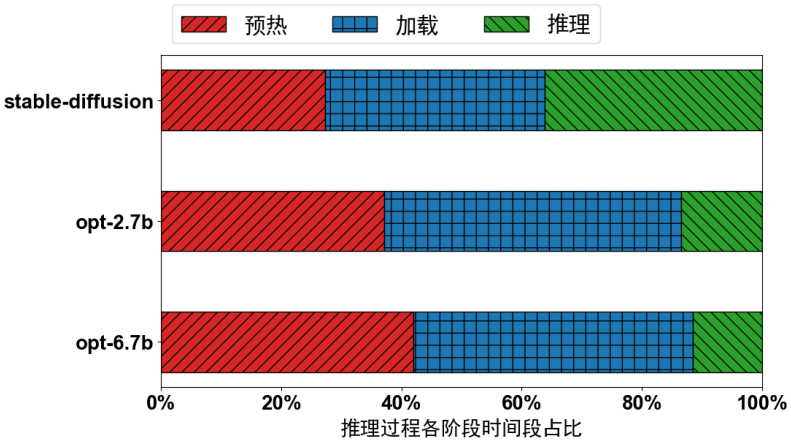


图 3.2 AI 模型推理服务各阶段占端到端时延的比例

综上所述，以上分析说明了云原生容器环境下模型缓存统一管理困难以及冷启动的问题，进一步可以分为以下两个子问题：

子问题 1：AI 模型数据源多样，上层推理引擎存储配置与访问复杂，如何以标准、统一、透明的方式管理 AI 模型缓存？

子问题 2：云原生环境下存储-计算分离，导致 AI 模型推理服务启动时间长，预热时间占比大，如何根据实时推理请求的访问模式选择性地预热模型？

对于子问题1，本章在3.2节提出了云原生环境的模型缓存抽象与管理方法；对于子问题2，本章在3.3节提出了推理请求动态感知的模型缓存池预热策略。

3.2 云原生环境的模型缓存抽象与管理

本节首先提出了AI模型缓存的统一抽象结构DataModel，使得上层推理服务以标准、统一、透明的方式访问AI模型缓存；此后介绍如何对AI模型缓存池中的缓存状态进行管理。

3.2.1 AI模型缓存抽象

为了解决AI模型数据源多样，上层推理引擎存储配置与访问复杂的问题，本节提出了DataModel这一抽象结构，用来描述模型缓存存储数据源、访问路径、模型格式等基本属性。用户可以使用统一的方式创建模型缓存，并且以标准的POSIX接口方式访问缓存数据，降低了开发者编写程序的复杂度。

图3.3给出了AI推理应用通过模型缓存池读取不同数据源中模型的例子。具体而言，应用层包括不同的AI推理服务容器，存储层包含存储在不同数据源的AI模型。本文使用模型缓存池的概念实现了模型缓存的抽象，位于缓存池中的每个DataModel可以用来抽象地描述在不同数据源中的模型。例如，DataModel 1绑定了分布式缓存1，描述了位于WebUFS中的Model A以及位于S3存储中的Model C；DataModel 2绑定了分布式缓存2，描述了位于WebUFS中的Model B。上层的AI推理容器1可以直接通过POSIX接口访问DataModel 1，从而实现从缓存池中加载所需的AI模型Model A与Model C；AI推理容器2也可以直接通过POSIX接口加载所需的AI模型Model B。

现有的AI推理框架均可以通过POSIX接口加载特定路径下的AI模型。因此，推理服务均可通过标准、统一、透明的方式访问AI模型，降低了开发者编写代码的复杂度。

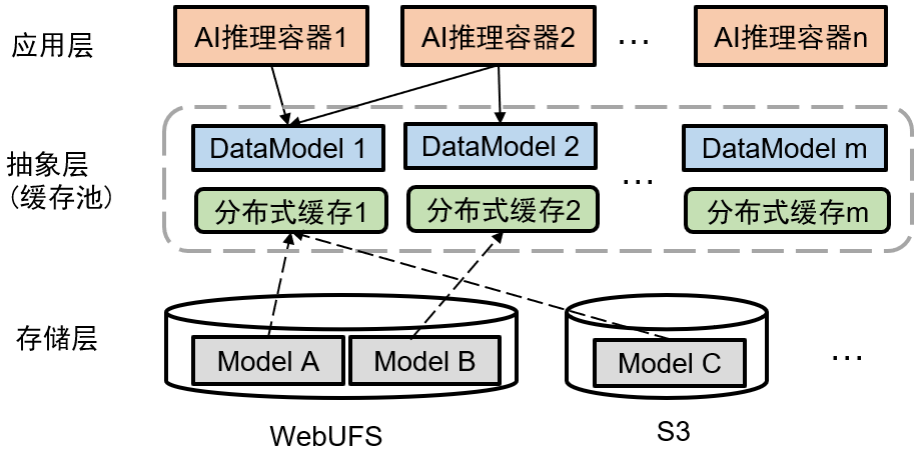


图 3.3 推理应用通过模型缓存池的 DataModel 读取不同数据源中模型

另外，由于 DataModel 描述了模型缓存存储数据源、访问路径、模型格式等基本属性，即使由于数据迁移导致位于存储层的模型的存储路径发生了改变，开发者仅需修改 DataModel 的对应属性即可实现对 AI 模型的加载与访问。

3.2.2 AI 模型缓存池缓存状态管理

DataModel 通过抽象的方式描述了 AI 模型的各种属性信息，而模型的缓存和加速能力则由分布式缓存提供。DataModel 与分布式缓存是一对一绑定关系，即每个 DataModel 都绑定了一个分布式缓存。分布式缓存根据 DataModel 描述的 AI 模型，将远程存储数据源的 AI 模型预热到缓存介质中，位于应用层的 AI 推理服务即可通过访问 DataModel 的方式实现模型加载的加速。

AI 模型缓存池的缓存状态管理流程如图 3.4 所示。用户通过 YAML 文件在 Kubernetes 集群中创建 DataModel 对象时（步骤①），DataModel 进入 Created 状态。接着，根据 DataModel 描述的 AI 模型缓存属性信息，系统将为 DataModel 创建一个分布式缓存系统（步骤②）。在分布式缓存创建过程中，系统将根据 DataModel 的 AI 模型数据源、存储路径等信息，将存储层的 AI 模型路径挂载到分布式缓存中。当分布式缓存系统的各个组件成功启动且挂载完毕后，DataModel 进入 Bound 状态，表示 DataModel 与分布式缓存完成了绑定（步骤③）。绑定完成后，分布式缓存将从对应的远程模型存储数据源中预热数据，将位于远程存储层的 AI 模型预取到分布式缓存集群内以提供加速能力（步骤④）。至此，位于应用层的 AI 推理服务容器即可通过 POSIX 接口访问模型数据，将 AI 模型从分布式缓存系统中加载到 AI 推理容器的内存或者 GPU 显存中，为后续的推理任务做好准备（步骤⑤）。

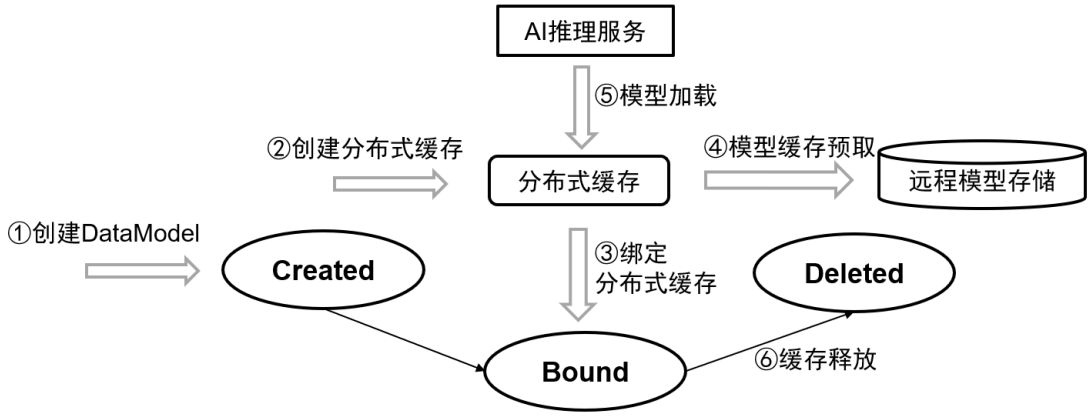


图 3.4 AI 模型缓存池缓存状态管理流程

如果后续的推理任务需要加载相同的 AI 模型，AI 推理服务可以直接从分布式缓存中读取 AI 模型，使用集群内布的节点带宽资源实现加速，避免了再次从远程的模型存储数据眼中再次下载并加载 AI 模型。如果用户认为 DataModel 不需要再被访问时，可以主动将 DataModel 资源删除。系统在删除 DataModel 资源时，会释放对应分布式缓存系统的缓存数据，最后关闭分布式缓存的各个组件，从而节省集群的存储资源。

随着模型种类数量的越来越多，本文采用 DataModel 与分布式缓存一一绑定的设计模式，能有效缓解单个分布式缓存带来的单 Master 节点的性能瓶颈问题。另外，在多推理任务的场景下，为了资源重复利用，用户通常不会立即释放使用完毕的DataModel，系统中的多个分布式缓存可供后续到来的多个推理任务复用，即每个推理任务可以选择适合的 DataModel 进行模型加载，从而加速推理服务的启动过程（见 5.2 节）。

3.3 推理请求动态感知的模型缓存池预热策略

尽管 DataModel 可以有效对云原生容器平台缓存池中的模型缓存进行抽象和统一管理，但是 3.1 节指出，云原生环境下存储-计算分离的固有特性会导致 AI 模型推理服务启动过程的预热时间占比较大。在 AI 推理任务提供服务的过程中，需要根据实时推理请求的访问模式选择性地预热模型，从而缩短总体的预热时间。本节首先提出了一种轻量级的访问频率实时统计方法用来追踪不同模型的热度，此后基于该方法提出了面向缓存收益最大化的 AI 模型缓存预热方法。

3.3.1 轻量级访问频率实时统计方法

为了尽可能减少节点的计算、存储开销，同时保证请求频率统计的准确性，本文使用了带有时间窗口的 Count-Min Sketch[35]数据结构，对过去一段时间窗口内各 AI 模型访问频率进行实时统计，兼具空间高效性、操作实时性、结果准确性。

Count-Min Sketch 是一种概率数据结构，广泛应用于大数据流处理中的频率估计问题。它通过使用多个哈希函数和二维计数器数组，能够在有限的内存空间内高效地统计元素的出现频率。然而，传统的 Count-Min Sketch 数据结构无法统计指定时间窗口内的元素出现频率。鉴于 AI 模型推理请求的频率会随时间动态变化，本文引入了时间窗口机制，通过定期滚动更新频率值，能够实时统计最近时间窗口内的 AI 模型被请求频率，从而更好地适应实时 AI 模型推理的场景需求。

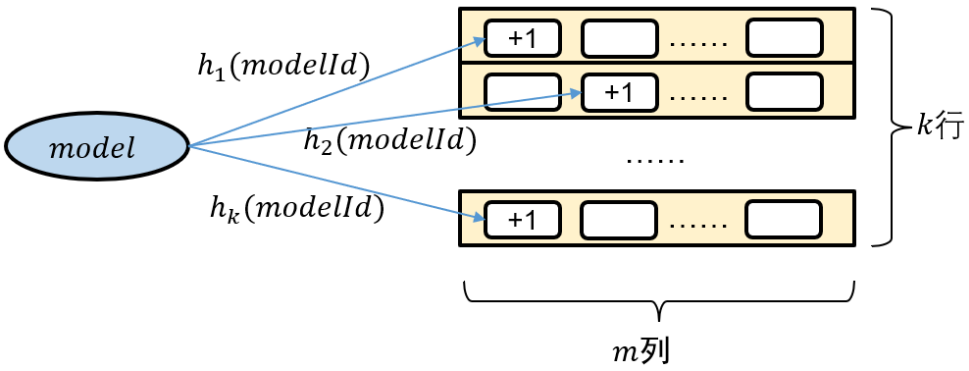


图 3.5 Count-Min Sketch 近似数据结构示意图

如图 3.5 所示，Count-Min Sketch 被设计为一个二维数组，其行数为k，表示所使用的哈希函数的数量；列数为m，表示每个哈希函数对应桶（bucket）的数量。

当某个 AI 模型被访问时（假设该 AI 模型的模型唯一标识符为 $modelId$），首先对$modelId$进行k个哈希函数的计算，并将每个哈希结果对m取模，从而确定该 AI 模型在二维数组中每行对应的列位置，紧接着对各个位置上的值自增 1 即可完成频率的自增。在查询 AI 模型被访问的频率时，同样通过k个哈希函数定位其在每行中的列位置。由于哈希冲突可能导致高估频率的情况，频率计算结果可能偏高，故最终频率值取所有位置频率值的最小值即可。

以图 3.5 为例，当 AI 模型$model$被访问时，首先通过对$modelId$进行第一个

哈希函数的计算$h_1(modelId)$，并将哈希结果对m取模，最终映射到第一行第一列的位置，并对该位置的频率值自增 1；同理，使用第二个哈希函数做出相同的操作，将$modelId$映射到第二行第二列的位置，并对该位置的频率值自增 1。依此类推进行k次哈希操作后，即可完成频率的自增。在查询$modelId$对应的模型访问频率时，由于哈希函数的计算方式不变，故$modelId$在二维数组中被映射到k个位置仍然不变，最终取所有位置值的最小值即为$modelId$被访问频率的估计值。

二维数组的行数k与列数m的值可通过计数误差的范围ϵ和置信度$1-\delta$确定，即$k=\lceil e/\epsilon \rceil$，列数$m=\lceil \ln 1/\delta \rceil$。假设模型$modelId$的真实频率为$a_i$，使用 Count-Min Sketch 计算的估计频率为$\hat{a_i}$，AI 模型总数量为N，通过上述方式设置 sketch 的行数和列数，则可以确保：

$$P(\hat{a_i} \le a_i + \epsilon N) \ge 1-\delta \tag{3-1}$$

即频率估计值$\hat{a_i} \le a_i + \epsilon N$的概率不低于$1-\delta$。

接下来，为了引入时间窗口的概念，本文受到 Sliding Sketch[36]思想的启发，对 Count-Min Sketch 结构做出了一定修改。如图 3.6 所示，二维数组的每个 bucket 被分为了两个部分，分别为$curVal$和$preVal$，分别表示当前时间窗口统计的频率值和上一个时间窗口统计的频率值，通过不间断地周期性扫描方式，对统计的频率进行定期淘汰和更新，确保 Count-Min Sketch 统计到的数据对象访问频率落在最新的时间窗口T内。

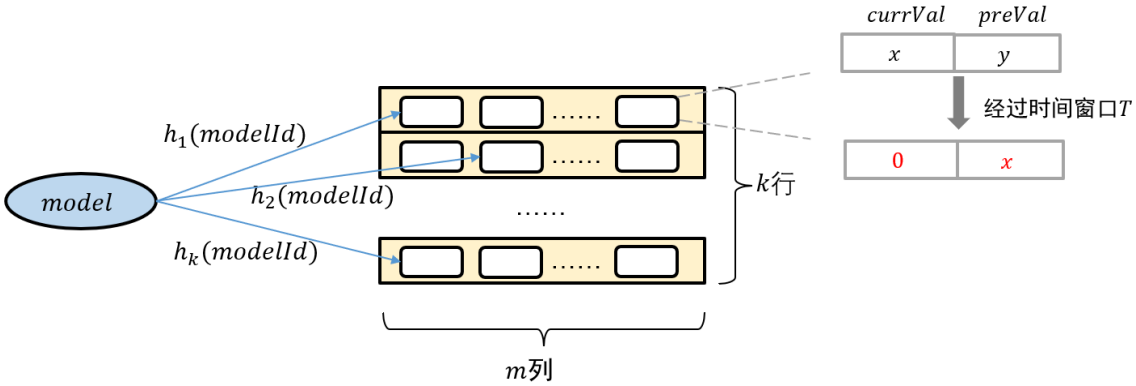


图 3.6 带有时间窗口的 Count-Min Sketch 结构

具体地，将 Count-Min Sketch 的扫描周期设置为T，每扫描到一个 bucket，将上一个时间窗口统计的频率值$preVal$替换为当前时间窗口的频率值$curVal$，并

将当前时间窗口频率值$curVal$置 0，表示一次时间的推移。在对 Count-Min Sketch 进行频率自增操作时，仅需对$curVal$值自增；在对 Count-Min Sketch 进行频率查询操作时，过去一段时间窗口内的频率值为$curVal$与$preVal$之和。

最后，对时间窗口为T的 Count-Min Sketch 的维护做出总结性的描述。数据结构的维护包括三个操作，分别为扫描、查询、更新。

（1）扫描：该操作完成频率的淘汰和更新操作，从而实现时间窗口的推移。对 Count-min Sketch 进行周期性地不间断扫描，遍历 Count-min Sketch 的每一个桶，确保对整个 Count-min Sketch 的扫描周期正好等于时间窗口大小T。每扫描到一个桶，将$curVal$的值赋给$preVal$，并将$curVal$重置为 0。

（2）查询：该操作查询数据对象被访问的频率。对$modeld$通过k个哈希函数计算后分别对m取模，定位$modeId$在每个行内的列位置，并从这k个位置得到k个元组$\{(curVal_i, preVal_i)|1 \le i \le k\}$，该 AI 模型被访问的频率则为$\min\limits_{1\le i\le k}(curVal_i + preVal_i)$

（3）更新：该操作对 AI 模型被访问的频率进行自增。对$modelId$通过k个哈希函数计算后分别对m取模，定位$modeId$在每个行内的列位置，并将每个对应桶的$curVal$值自增 1。

3.3.2 面向缓存收益最大化的模型缓存预热方法

在 AI 模型推理服务的负载会随着时间推移而不断变化，不同的推理请求会使用不用的 AI 模型完成推理服务，表现出数据访问模式的变化。本小节基于 3.3.1 小节提出的访问频率实时统计方法，设计了推理请求动态感知的模型缓存实时预热策略，在 AI 推理应用提供服务时，当推理服务容器启动时延较高时，将根据模型数据的访问模式重新对 AI 模型进行预热。接下来对 AI 模型缓存预热的方法进行详细介绍。

缓存池中分布式缓存的缓存总容量是有限的，由于不同的 AI 模型具有不同的缓存收益，所以在缓存实时预热过程时，倾向于预热具有更高缓存收益的 AI 模型。

本文将 AI 模型的缓存收益定义为一个与模型访问频率和预热时间相关的综合评分指标。具体而言，模型在过去时间窗口内的访问频率越高，表明其热度越

高，相应的缓存收益也越大；同时，模型的预热时间越短，意味着在当前时刻能够完成更多模型的预热，从而提升缓存收益。

某一时刻模型$model$的缓存收益如公式(3-2)所示，其中$freq_{model}$为上一个时间窗口T内$model$的经过 Min-Max 归一化后的被访问频率，可由 Count-Min Sketch 实时计算得到；$warmTime_{model}$为模型$model$的经过 Min-Max 归一化的预热时间，预热时间可以根据模型大小$size_{model}$与远端存储的数据访问带宽B_{remote}计算得到，即$size_{model}/B_{remote}$。w_f与w_u为(0,1)区间内的权重系数。

$$reward_{model} = w_f * freq_{model} + w_t * warmTime_{model}^{-1} \tag{3-2}$$

按照上述方法统计过去一段时间窗口T内$reward_{model}$值，并按照其值降序排序，优先将缓存收益更高的 AI 模型预取到分布式缓存系统中，直至缓存空间被完全占用为止。

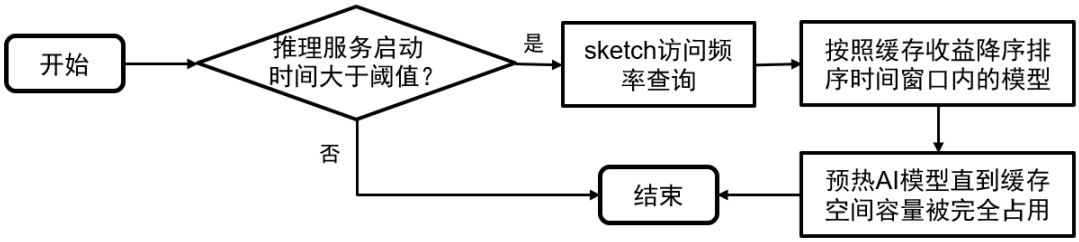


图 3.7 推理请求动态感知的模型缓存池预热策略整体流程

综上所述，推理请求动态感知的模型缓存池预热策略的方法流程如图 3.7 所示。其中，使用 Count-Min Sketch 进行频率实时更新与查询的时间复杂度均为$O(1)$；在缓存收益值排序阶段，尽管需要对时间窗口内的 AI 模型进行排序，但由于参与排序的模型数量有限，其带来额外的时间开销是可控的。

3.4 实验评估与分析

本节通过一系列实验评估本章提出方法的有效性。首先介绍本节的实验评估环境与配置，接着对云原生环境下模型缓存池的缓存有效性和开销进行评估，最后对模型缓存池预热策略的有效性进行评估。

3.4.1 实验环境与配置

本节将分别介绍实验评估的硬件环境、软件环境、数据集与工作负载、评价指标、对比方法等信息。

（1）硬件环境

本节实验在 8 节点的集群中进行。每个节点包含 2 个 Intel(R) Xeon(R) Silver 4314 CPU，每个 CPU 有包含 16 个物理核心，支持个 32 线程。每个节点均有 256GB 内存。其中三个节点各有一块 NVIDIA 3090 GPU，GPU 显存为 24GB。每个节点均挂载了一块 500GB 的 SSD 硬盘以及一块 2TB 的 HDD 硬盘用于存储数据。所有节点均通过 10Gbps 带宽的以太网连接。

（2）软件环境

每个节点的操作系统版本为 CentOS 8.5.2111，系统内核版本为 4.18.0。使用 Kuberenetes v1.22.4 作为容器编排系统，使用 Alluxio v2.9.0[32]分布式缓存系统作为缓存池的实现。每个推理服务容器使用 Python 3.8.10 作为编程语言，CUDA 版本为 11.8，机器学习框架选择 Pytorch 2.0.0，模型推理框架选择 Transformers 4.40.0。模型推理任务均以 HTTP 接口的形式提供服务。

（3）数据集与工作负载

在数据集方面，本文采用以下两种 AI 模型作为数据集：

Opt[34]：Opt 模型是由 Meta AI（前身为 Facebook AI）开发的一系列大规模预训练的语言模型。这些模型基于 Transformer 架构，旨在通过自监督学习从大量文本数据中学习语言表示。Opt 模型有多个不同规模的版本，参数数量从 125M 到 175B 不等。本文采用参数规模为 2.7B 与 6.7B 的 Opt 模型，模型大小分别为 5GB 与 13GB。

Stable-diffusion[33]：Stable diffusion 是一种基于扩散模型（Diffusion Model）的生成模型，主要用于生成高质量的图像。Stable Diffusion 结合了 Latent Diffusion 的思想，在潜在空间中进行扩散过程，从而大幅降低了计算成本，使得模型可以在消费级硬件上运行。该模型在文本到图像生成任务中表现出色，能够根据文本描述生成高度相关且细节丰富的图像。本文采用 stable-diffusion-v2-1 模型作为数据集，模型大小为 9.7GB。

在工作负载方面，本文采用了 Azure 提供的 Serverless 请求负载 Azure Function Innovation Trace 2021[37]作为基础工作负载。为了模拟 AI 模型推理服务的调用场景，本文将数据集中的函数 ID 轮询映射到 AI 模型上，以此表示对模型推理服务的请求。同时，通过函数调用的结束时间与持续时间，计算出函数调用的开始时间，并将其作为推理服务请求的时间戳。

（4）评价指标

本节实验采用模型加载时间、模型预热时间、加载模型时的模型缓存命中率作为评估指标。

（5）对比方法

模型缓存池有效性评估对比方法：

MinIO 对象存储[59]加载：MinIO 是一种支持 s3 协议的，支持云原生 Kubernetes 环境的高性能对象存储系统。它提供了兼容 Amazon Web Services s3 协议的 API，并支持 s3 的所有核心功能。MinIO 采取分布式架构部署，数据被分为多个块分布在不同节点上，并提供了多副本与纠删码的容错机制以保证数据的完整性。MinIO 通过二进制文件即可部署，具有资源占用量低、兼容 s3 生态、高吞吐量等优点，可作为 AI 推理任务、大数据负载的数据湖。

模型缓存池预热策略有效性评估对比方法：

LRU（Least Recently Used，最近最少使用）：不对模型缓存进行预热，直接使用 LRU 策略进行缓存替换。LRU 一种最常见的缓存淘汰算法，其核心思想是优先淘汰最久未被访问的数据，以保留最近被频繁访问的数据，从而提高缓存命中率。

ARC[60]（Adaptive Replacement Cache，自适应替换缓存）：不对模型缓存进行预热，直接使用 ARC 策略进行缓存替换。ARC 是一种结合了 LRU 和 LFU 优点的缓存淘汰算法，其核心思想是通过动态调整缓存策略来适应不同的数据访问模式。ARC 维护了两个缓存队列 T1 与 T2 以及两个幽灵队列 B1 与 B2。其中 T1 用来存储最近访问一次的数据，T2 用来存储最近多次访问的热数据。B1 与 B2 分别用来记录从 L1 与 L2 队列中淘汰的数据，并通过比较 B1 和 B2 的命中情况动态调整 T1 与 T2 的容量占比。

ADTSM[61]（Automating Distributed Tiered Storage Management，自动化分布式分层存储系统管理）：该系统使用机器学习模型学习历史文件的创建时间、访问时间间隔等访问模式，利用该模型预测文件在未来一段时间内的访问频率，从而决定文件在分层存储系统中的存放层级。本实验借助该方法的思想，在训练阶段收集历史访问数据，利用 LightGBM[62]机器学习方法学习 AI 模型的历史访问模式（包括 AI 模型的首次请求时间以及过去 k 次的请求时间间隔）；在预测阶段输出未来一段时间窗口内每个 AI 模型的被请求概率，按照请求概率从高到低的顺序预热 AI 模型。

3.4.2 模型缓存池有效性评估

本小节实验对模型缓存池的缓存效果进行评估。实验选取了 stable-diffusion-v-2-1、opt-2.7b 和 opt-6.7b 三种 AI 模型作为测试对象，并为其分别创建了对应的 DataModel。这些 DataModel 被挂载到不同的模型推理服务容器中，以测量模型推理服务通过模型缓存池加载模型的时间。为了更全面分析缓存介质对加载性能的影响，实验采用了三种不同的存储介质——内存、SSD 和 HDD 来缓存 AI 模型，并对比了它们对模型加载时间的影响。为了评估模型缓存池的有效性，本实验测量了模型推理服务通过 s3 对象存储（使用 MinIO 对象存储系统实现，对象存储最大数据访问带宽为 10Gbps）加载模型的时间，作为对比的基准。

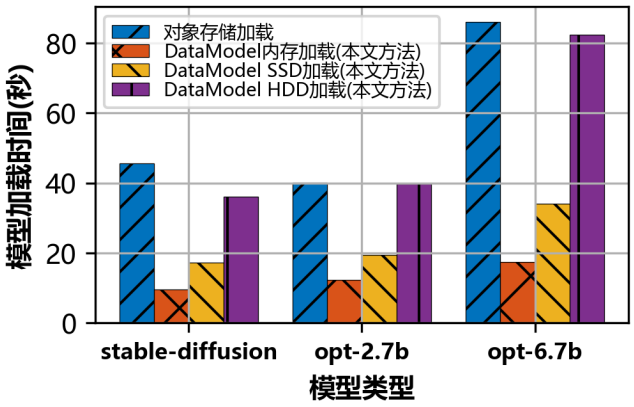


图 3.8 使用不同缓存介质的 DataModel 加载模型和对象存储加载模型的加载时间对比

实验结果如图 3.8 所示，对比了不同模型类型下，使用内存、SSD、HDD 三种缓存介质的 DataModel 加载模型与通过对象存储加载模型的加载时间。实验结果表明，采用 DataModel 内存缓存的加载时间最短，而使用对象存储加载的时间最长。本文提出的 DataModel 缓存最多将模型加载时间平均降低 76.06%。

在 DataModel 提供的缓存方式之中，内存缓存的性能最优，SSD 次之，HDD 相对较慢。然而，本文提出的通过模型缓存池的 DataModel 加载 AI 模型的方式均优于通过对象存储加载模型，即使在 HDD 缓存介质下，DataModel 的加载性能仍优于对象存储加载，尤其在 stable-diffusion 和 opt-6.7b 模型上表现更为突出。这一结果表明，相较于通过远程高性能对象存储加载 AI 模型，通过集群内部高速存储介质缓存 AI 模型能够显著提升加载速度，体现了模型缓存池的有效性。

3.4.3 模型缓存池开销评估

本小节实验对模型缓存池的开销进行评估。实验测量了两种场景下的模型加载时间。第一种场景使用 Kubernetes 的本地路径挂载（hostPath）方式，将 AI 模型直接挂载到推理服务容器内；第二种场景则通过 DataModel 将 AI 模型缓存挂载到推理服务容器内。在第一种场景中模型推理服务直接从本地目录中加载模型，在第二种场景中则会引入 DataModel 所绑定的分布式缓存系统。另外，为了排除节点间网络通信对实验结果的干扰，在第二种场景下，实验将分布式缓存系统的 Worker 组件部署在 AI 推理服务所在的节点上，确保推理服务从本地的 Worker 组件中加载模型，避免了跨节点加载模型的通信开销。

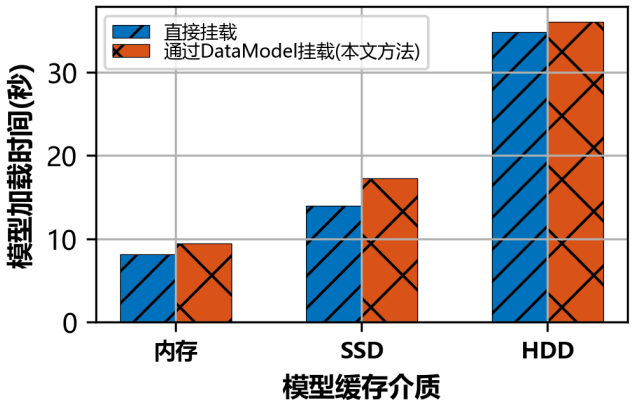


图 3.9 本地直接挂载与 DataModel 挂载方式的模型加载时间对比

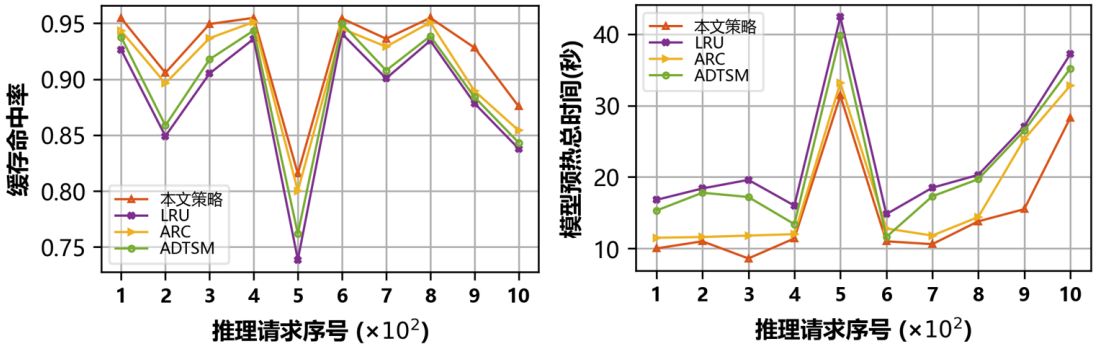
本小节使用 stable-diffusion 模型进行测试，分别测量了使用内存、SSD、HHD 三种存储介质下的模型加载时间。图 3.9 中蓝色柱状图表示直接挂载，橙色柱状图表示通过 DataModel 挂载。排除跨节点读取模型带来的通信开销因素后，本文提出的 DataModel 引入的额外开销平均仅为 1.92 秒，仅占据模型加载时间的极小部分。

模型缓存池的开销主要是由 DataModel 引入的分布式缓存组件和 FUSE 组件造成的。由于模型推理服务是通过 POSIX 接口加载模型，所以模型推理服务会通过 FUSE（用户态文件系统程序）组件从缓存 Worker 组件中读取模型数据，与直接从本地路径加载模型相比，调用路径更长，涉及 localhost 通信以及缓存 Master 组件与缓存 Worker 组件之间的通信与协调。尽管如此，模型缓存池引入的 DataModel 开销仍然在可接受范围内，并未显著影响系统性能。

3.4.4 模型缓存池预热策略的有效性评估

本小节评估在实际 trace 下的缓存池预热策略的有效性，并与现有方法进行对比。本实验分别在 trace 中不同的时刻应用模型缓存池预热策略，测量在未来一段时间内的模型缓存命中率与模型预热总时间。

在实验评估中，参数选择如下：w_f取 0.8，w_t取 0.2，缓存总容量为 50GB。为了确保 AI 模型的多样性，除了使用 3.4.1 节的模型以外，本小节实验还选取 stable-diffusion 以及 opt 模型族下的多种模型。



(a) 不同观测点的缓存命中率　　(b) 不同观测点的模型预热总时间

图 3.10 缓存预热策略的缓存命中率与模型预热时间评估

如图 3.10 所示，红色折线为本文提出的预热策略（淘汰缓存时使用 LRU 策略），绿色折线表示使用 ADTSM 缓存预热策略（淘汰缓存时使用 LRU 策略），紫色折线表示不使用模型预热策略，在缓存淘汰时仅使用分布式缓存的传统LRU缓存替换算法，黄色折线表示仅使用 ARC 缓存替换算法。图 3.10 (a)表明本文提出的缓存预热策略下的平均缓存命中率略高于基准方法，图 3.10 (b)表明本文的预热策略该显著降低了 AI 模型预热总时间。在不降低缓存命中率的情况下，相较于仅使用 LRU 缓存替换算法，本文预热策略将预热时间平均降低 34.40%；相较于使用 ARC 缓存替换算法，本文策略将预热时间平均降低 14.45%；相较于 ADTSM 缓存预热策略，本文预测策略将预热时间平均降低 29.09%。

由于 ARC 方法结合了 LRU 与 LFU 的优点，解决了 LRU 的突发流量淘汰热点数据问题和 LFU 的冷启动问题，故 ARC 的性能优于 LRU 算法。

本文的提出的缓存策略综合考虑了模型缓存收益，优先预热最近一段时间内具有较高访问频率以及预热时间短的模型，确保了有限时间内更多具有高缓存收益的 AI 模型被预热，故相较于仅使用 LRU 与 ARC 缓存淘汰策略可有效降低缓存预热总时间。

ADTSM 采用机器学习方法，根据过去的 AI 模型访问模式预测未来一段时间窗口内的被请求概率，并按照访问概率从大到小的顺序预热 AI 模型。本文提出的预热策略性能仍优于 ADTSM 方法，原因可归纳为以下两个方面：首先，ADTSM 仅学习了 AI 模型首次请求时间、请求时间间隔等访问模式特征而忽略了 AI 模型大小特征，导致较小的 AI 模型可能未被正确预热；其次，对于请求时间间隔特征不足 k 次的情况，LightGBM 无法进行训练，从而忽略了部分请求时间间隔特征不足的 AI 模型，但是这些 AI 模型可能会在未来的时间窗口内被频繁访问，从而延长了预热时间。另外，访问模式的变化会导致 LightGBM 的重新训练，ADTSM 方法难以具备本文策略的轻量级特点。

3.5 本章小结

本章针对云原生容器环境下 AI 模型缓存数据源多样，缓存池统一管理困难的问题，提出了统一的抽象与管理方法与优化策略。首先，本章提出了 DataModel 数据抽象以及缓存池模型缓存状态管理方法，对 AI 模型缓存进行统一管理。在此基础上，针对真实推理场景中，AI 模型推理服务启动过程的预热时间占比大的问题，提出了推理请求动态感知的缓存预热策略并使用近似数据结构降低系统开销，有效降低缓存预热总时间。最后，本章设计了一系列实验对本章的方法进行评估。实验结果表明，本文提出缓存池统一管理方法和策略可以有效降低模型加载时间和缓存预热时间。

第四章　面向最低成本的模型缓存池资源配置与短路读优化

在云原生容器环境的 AI 推理场景下，合理地平衡模型缓存池资源配置成本和 AI 推理任务的服务质量是一个较大的挑战。本章将研究对模型缓存池进行成本高效的资源配置与性能优化方法，以满足 AI 推理服务对模型加载时间的服务质量要求。本章内容组织结构如下：4.1 节介绍并分析本章的问题背景，4.2 节介绍缓存池资源感知的模型加载时延预估方法，4.3 节介绍模型缓存池资源配置算法与短路读优化策略，4.4 节对本章提出的方法进行评估与分析，4.5 节对本章内容进行总结。

4.1 问题背景与分析

本文在第三章提出了云原生环境的模型缓存池统一管理方式，通过引入 DataModel 的抽象概念对 AI 模型缓存对象做出了描述。从 DataModel 的实现角度上来看，缓存池中的每个 DataModel 均绑定了一个分布式缓存系统，分布式缓存系统通常由 Master 组件和 Worker 组件构成，其中 Master 组件负责管理协调各组件，Worker 组件负责缓存数据。

在云原生容器环境下，得益于弹性容器服务按量付费、弹性伸缩等特点，AI 推理服务、缓存 Master 组件、缓存 Worker 组件均可以以弹性容器的形式部署运行。开发者既可以根据需求为每个弹性容器配置合适的计算资源，也可以为每个容器挂载合适规格的云盘，实现数据的持久性存储。在 AI 推理服务运行时，开发者只需根据推理服务运行时间、配置的容器规格、云盘规格对资源进行付费。

图 4.1 展示了云原生容器环境下 AI 推理服务从模型缓存池中读取模型的过程。首先，在推理服务容器加载 AI 模型前，分布式缓存池中与 DataModel 所绑定的分布式缓存从远端模型存储中预取模型，这些 AI 模型均被缓存到分布式缓存系统中；接着，由于云原生环境下存储-计算分离的架构，需要为这些缓存 Worker 容器挂载一个按量付费的云盘用来存储 AI 模型；最后，推理引擎中的推理服务容器借助 FUSE，通过 POSIX 接口从模型缓存池中加载模型，为后续的推理过程做准备。

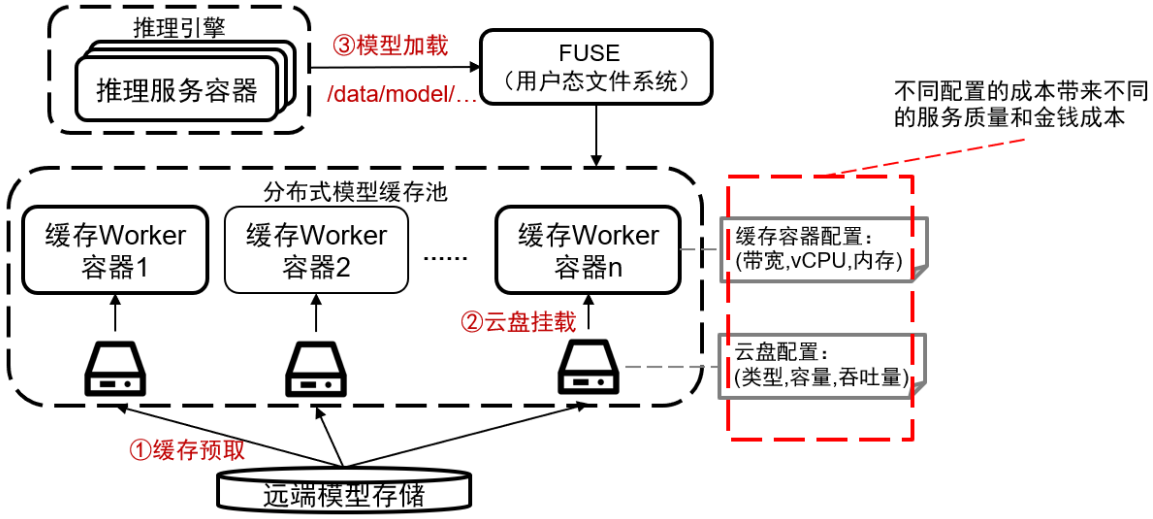


图 4.1 云原生容器环境下 AI 推理服务从模型缓存池中读取模型的过程

图 4.1 的过程展现了云原生容器环境存储-计算分离以及细粒度付费的特点。模型缓存池中的缓存 Worker 容器的 vCPU 核数、内存大小等配置可以由开发者灵活指定，其中容器带宽与 vCPU 核数直接相关[38]；模型缓存池中的云盘的类型、容量、吞吐量也可以根据开发者的需求进行配置，其中云盘吞吐量与云盘容量成正相关[39]。不同的模型缓存池资源配置方案会直接影响推理服务的质量（例如模型加载时间）以及金钱成本。

由于模型加载时间在整个模型推理过程中占比较大（见 3.1 节分析），进而影响推理过程的服务质量，因此本文在 Kubernetes 集群与 Alluxio 缓存系统之上，通过以下两个动机实验揭示模型缓存池资源配置方案对模型加载时间的影响。

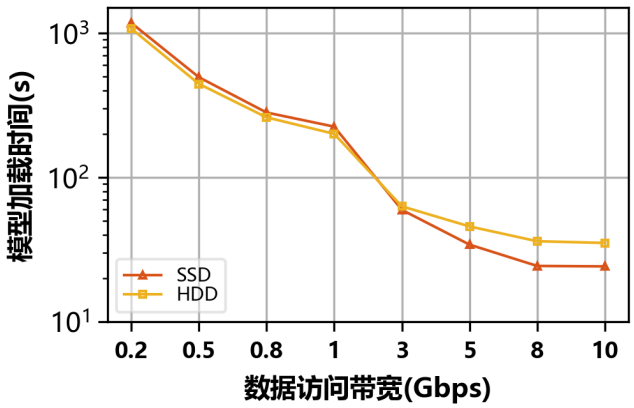


图 4.2 容器数据访问带宽以及云盘类型对模型加载时间的影响

图 4.2 展示了缓存 Worker 容器的数据访问带宽以及云盘类型（以 SSD 和 HDD 为例）对 AI 模型加载时间的影响。实验表明不同的容器数据访问带宽以及

云盘类型对模型加载时间的影响显著，时间跨度为三个数量级，且云盘的吞吐量和容器的数据访问带宽均可能成为瓶颈。当数据访问带宽较小时（小于 1Gbps）时，两种云盘配置下的模型加载时间差异较小，此时容器数据访问带宽成为主要瓶颈；当数据访问带宽较大（大于 5Gbps）时，由于 SSD 吞吐量高于 HDD 吞吐量，故 SSD 能带来更短的模型加载时间，此时云盘吞吐量成为主要瓶颈。因此，在选取缓存 Worker 容器配置和云盘配置时，需综合考虑两者的性能瓶颈。

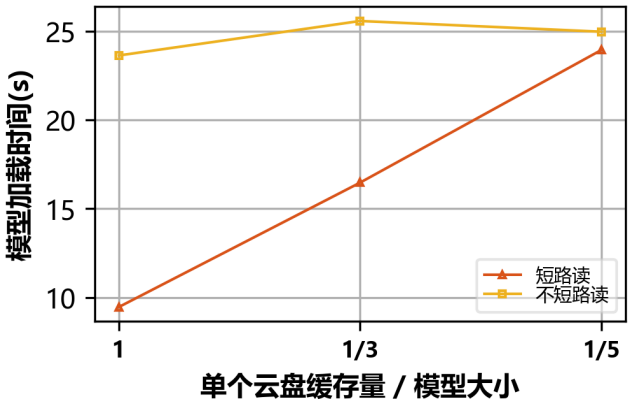


图 4.3 单个云盘缓存量和短路读因素对模型加载时间的影响

图 4.3 展示了单个云盘缓存量和短路读因素对模型加载时间的影响。在缓存 AI 模型时，采用多个缓存容器共同缓存一份 AI 模型的方式，缓存 Worker 容器数量越多，单个缓存 Worker 所挂载云盘的缓存量则越少。短路读指 AI 模型推理容器和任意一个缓存 Worker 容器位于同一节点，推理容器通过直接读取本地文件系统的方式加载模型；非短路读指 AI 模型推理容器和任意一个缓存 Worker 容器均不位于同一节点。图 4.3 表明单个云盘的缓存量越高，短路读相较于非短路读能够更显著地降低模型加载时间。因此，云原生容器服务提供商可以根据开发者的缓存池配置，通过容器的合理部署实现短路读优化策略，进一步降低模型加载时间。

综上所述，从云原生容器平台使用者（开发者）的角度来看，需要为模型缓存池选择成本低廉的资源配置以满足服务质量（如 AI 模型加载时间）的要求；而从云原生容器服务提供者的角度来看，则需要通过系统层面上的优化策略或者容器放置策略进一步降低模型加载时延，从而提升整体服务性能。

然而，云原生容器平台上缓存池的配置种类众多，包含缓存容器数量、缓存容器带宽、缓存容器 vCPU 数量、缓存容器内存、云盘类型、云盘容量、云盘吞吐量等配置，配置值中既包含连续值又包含离散值，组合总数过多，开发者高效

地选择低成本的缓存池配置同时满足 AI 模型加载时间的 SLO 成为一个巨大的挑战。

上述问题可以转化为如何设置缓存池资源与短路读策略，在给定模型加载时间目标下实现成本最低的问题。进一步可以划分为以下两个子问题：

子问题 1：如何根据缓存池资源配置和短路读策略预估 AI 模型加载时间？

子问题 2：在给定模型加载时间 SLO 下，如何高效搜索使得金钱成本最低的缓存池资源配置和短路读策略？

对于子问题 1，本章在 4.2 节提出了缓存池资源感知的模型加载时延预估方法；对于子问题 2，本章在 4.3 节提出了模型缓存池资源配置算法与短路读优化策略。

4.2 缓存池资源感知的模型加载时延预估方法

本节首先在云原生容器环境的模型缓存池场景下，对 AI 模型加载时间进行建模；此后使用回归分析的方法建立模型加载时间与数据访问吞吐量之间的数学关系。

4.2.1 模型加载时间建模

由 4.1 节分析可知，云原生容器环境下从模型缓存池加载模型的时间主要受到云盘类型、云盘吞吐量、缓存 Worker 容器数量、缓存 Worker 容器数据访问带宽影响以及短路读策略的影响。

首先，与模型加载时间直接相关的因素为数据访问吞吐量。数据访问吞吐量则由实际数据访问带宽与云盘吞吐量共同决定。假设 AI 推理服务容器的数量为N_{infer}，每个模型缓存 Worker 容器能提供的数据访问带宽为B_{cache}，开发者配置的缓存 Worker 容器数量为n_{cache}，则实际数据访问带宽B可以表示为公式(4-1)，即实际数据访问带宽等于每个 AI 推理服务容器能分配到的缓存聚合带宽。

$$B = n_{cache} * B_{cache}/N_{infer} \tag{4-1}$$

云盘吞吐量tp则由云盘类型$diskType$和云盘容量$capacity$决定。云盘吞吐量与云盘容量成正相关关系，相关系数为$\alpha_{diskType}$，$\beta_{diskType}$为云盘的最低吞吐量，$maxThroughput_{diskType}$为该类型云盘的最高吞吐量。云盘吞吐量计算方式可由公式(4-2)表示。

$$tp = \min(\alpha_{diskType} * capacity + \beta_{diskType}, maxThroughput_{diskType}) \quad (4\text{-}2)$$

其次，模型加载时间也受到短路读因素的影响。当推理服务容器与缓存 Worker 容器不位于同一节点时，AI 模型的全部数据需通过网络加载，无法通过短路读进行优化。此时，非短路读情况下的模型加载时间t_1可以由公式(4-3)计算。其中$t(m,b)$为模型加载时间关于模型大小与实际数据访问吞吐量的函数,M为 AI 模型的总大小，B为实际数据访问带宽，tp云盘吞吐量。由于实际数据访问带宽与云盘吞吐量均可能成为瓶颈，故实际数据访问吞吐量为两者的最小值。

$$t_1 = t(m,b) = t(M, min\ (tp, B)) \quad (4\text{-}3)$$

当推理服务容器与任意一个缓存 Worker 容器位于同一节点时，推理服务可以通过短路读方式，读取本节点上缓存的部分 AI 模型以加速模型加载过程。假设本地节点上缓存的 AI 模型数据量为M_{local}，那么短路读情况下的模型加载时间t_2可以由公式(4-4)计算，分为远程读与本地读两个部分。

$$t_2 = t(M - M_{local}, min\ (tp, B)) + t(M_{local}, tp) \quad (4\text{-}4)$$

综上，最终的实际模型加载时间t_{load}可以由公式(4-5)表示，其中I_{SC}为布尔变量，表示是否开启短路读策略。由于本文仅关注模型缓存池的配置，故推理服务容器数量N_{infer}与推理服务容器的数据访问带宽B_{cache}均为定值，因此，模型加载时间可以表示为$I_{SC}, diskType, capacaity, n_{cache}, B_{cache}$的函数。

$$\begin{aligned} t_{load} &= t_{load}(I_{sc},\ diskType,\ capacity, n_{cache}, B_{cache}) \\ &= t_2 * I_{sc} + t_1 * (1 - I_{sc}) \end{aligned} \quad (4\text{-}5)$$

4.2.2 模型加载时间回归分析

在 4.2.1 小节中对模型加载时间进行了建模，然而公式(4-3)中的函数$t(m,b)$并未给出具体的解析表达式，因此无法根据给定模型大小与实际数据访问吞吐量计算模型加载时间。故本节采用回归分析方法对该函数进行拟合，以确定其具体形式。

由于每种 AI 模型结构和模型权重各不相同，故本小节对于每一种 AI 模型，固定m参数后再对变量b进行单变量的回归分析，即$t(m,b) = t(m_0,b) = f(b)$。另外，如果需要计算 AI 模型某一部分的加载时间（例如公式(4-4)所述的情况），

可以按照模型大小的比例对时间进行缩放，从而得到相应的估计值。

为了建立模型加载时间t与数据访问吞吐量b的定量关系，本文首先通过使用 Linux 的 cgroup 工具以及网络限速工具 wondershaper 控制数据访问吞吐量，测量在不同数据访问吞吐量下的模型加载时间。以模型 stable-diffusion-v2-1 为例，图 4.4 展示了模型的加载时间与数据访问吞吐量之间的关系，结果表明模型加载时间与数据访问吞吐量并不成简单的线性关系。

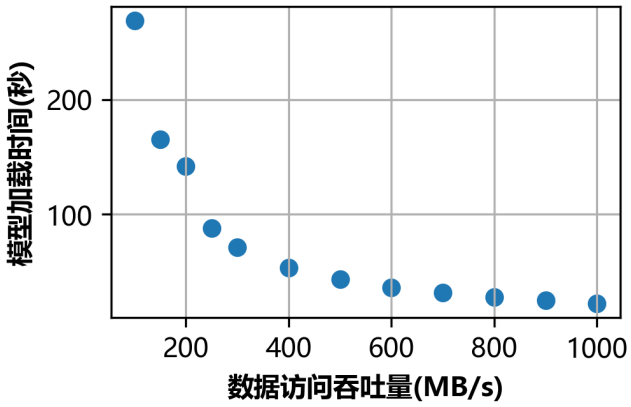


图 4.4 stable-diffusion 模型的加载时间与数据访问吞吐量关系的散点图

为了分析这一非线性关系，本文对原始数据进行了对数变换处理。具体地，分别对模型加载时间和数据访问吞吐量取自然对数（以e为底）。如图 4.5 所示，经过预处理后，两者之间呈现出显著的线性相关性。

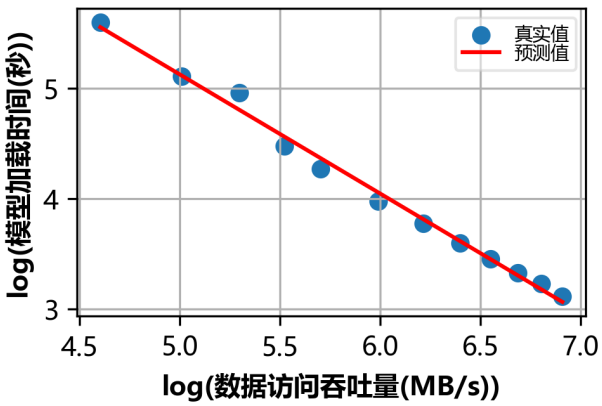


图 4.5 取对数后模型加载时间与数据访问吞吐量之间的关系

此时，模型加载时间t与数据访问吞吐量b之间的关系可以使用公式(4-6)表示。其中，θ_1与θ_2均为待拟合的参数。使用最小二乘法对两者关系进行线性回归分析后，拟合结果如图 4.5 的红色直线所示。获得θ_1和θ_2参数后，即可根据公式(4-7)

通过已知的数据访问吞吐量b预估模型加载时间。

$$log(t) = \theta_1 * log(b) + \theta_2 \tag{4-6}$$

$$t = e^{\theta_2} * b^{\theta_1} \tag{4-7}$$

更多 AI 模型的回归分析结果以及预估方法的准确性评估详见 4.4.2 小节。

4.3 模型缓存池资源配置算法与短路读优化策略

本文第 4.2 节给出了在给定缓存池资源配置下预估模型加载时间的方法。然而，影响模型加载时间和金钱成本的配置参数众多，不同配置的组合导致参数搜索空间巨大，开发者难以高效地搜索到在模型加载时间 SLO 约束下，成本最优的缓存池配置。因此，本节首先给出了问题的形式化定义；接着通过对问题的分析与化简，提出了基于分段序列二次规划的模型缓存池资源配置算法，用于求解该优化问题；最后，从云平台提供商的角度出发，提出了短路读优化策略以进一步缩短模型加载时间。

4.3.1 问题定义与形式化

针对如何搜索在给定模型加载时间 SLO 约束下的成本最低的缓存池配置问题，本节进行问题的形式化定义。首先对模型加载引入的成本进行形式化定义，模型加载的金钱成本包括云盘成本，缓存 Worker 容器的 vCPU 成本以及缓存 Worker 容器的内存成本。模型加载的金钱成本可由公式(4-8)计算，成本由变量$I_{SC}, diskType, capacity, n_{cache}, n_{cpu}, n_{mem}$决定，$cost_{diskType}$表示$diskType$类型云盘的单位成本（元/GB/s），$cost_{cpu}$表示缓存 Worker 容器的单位 vCPU 成本（元/核/s），$cost_{mem}$表示缓存 Worker 容器的单位内存成本（元/GB/s）。其中$diskType$与$cost_{diskType}$的对应关系见表 4-2，$cost_{cpu}$与$cost_{mem}$均为常量。

$$cost = t_{load} * n_{cache} * (cost_{diskType} * capacity + cost_{cpu} * n_{cpu} + cost_{mem} * n_{mem}) \tag{4-8}$$

对模型加载的容器成本和云盘成本进行建模后，优化问题的形式化定义如公式(4-9)所示，其中决策变量包括$I_{SC}, diskType, capacity, n_{cache}, n_{cpu}, n_{mem}, B_{cache}$。该问题记为问题 **P1**。

$$\min cost \tag{4-9}$$

$$s.t.\quad t_{load}(I_{sc}, diskType,\ capacity, n_{cache}, B_{cache}) \le T_{SLO}$$

$$diskType \in \Phi$$

$$M/n_{cache} \le capacity \le maxCapacity_{diskType}$$

$$B_{cache}, n_{cache}, capacity, n_{cpu}, n_{mem} \in Z$$

公式(4-8)和(4-9)中各个符号的含义详见表 4-1，其中t_{load}由公式(4-5)计算。第一个约束条件表示模型加载时间不能超过 SLO；第二个约束条件表示云盘必须从云平台供应商提供的可选类型集合中选取；第三个约束条件保证配置的云盘容量既能满足 AI 模型的缓存需求，又不超过其最大容量限制；第四个约束条件为决策变量的整数约束。

表 4-1 优化问题中各符号的含义

符号	含义
I_{SC}	是否使用短路读策略
$diskType$	云盘类型
$capacity$	云盘容量（GB）
$maxCapacity_{diskType}$	$diskType$类型云盘的最大吞吐量
n_{cache}	缓存 Worker 容器数量
B_{cache}	缓存 Worker 容器数据访问带宽（Mbps）
n_{cpu}	缓存 Worker 容器 vCPU 核数
n_{mem}	缓存 Worker 容器内存大小
tp	云盘吞吐量（MB/s）
$cost_{cpu}$	缓存 Worker 容器的单位 vCPU 成本（元/核）
$cost_{mem}$	缓存 Worker 容器的单位内存成本（元/GB/s）
$cost_{diskType}$	云盘的单位成本（元/GB/s）
T_{SLO}	模型加载时间目标
Φ	云厂商提供的云盘类型集合
M	AI 模型大小（GB）

然而，该优化问题 **P1** 的求解面临以下挑战：首先，该问题共涉及$I_{SC}, diskType, capacity, n_{cache}, B_{cache}, n_{cpu}, n_{mem}$共七个决策变量，既包含连续型又包含离散型的变量，不同决策变量的组合导致解空间巨大，难以使用枚举法在可行时间内获得全局最优解；其次，目标函数与约束条件均包含非光滑的最大值与最小值函数，具有非连续可微的特性，无法使用解析法求数值解，而使用启发式搜索算法容易陷入局部最优解。

4.3.2 基于分段序列二次规划的模型缓存池资源配置算法

公式(4-9)所示的问题 **P1** 是非线性约束混合整数规划问题，本文采用基于分段序列二次规划算法求解该问题。针对该问题的求解，本文提出了一种分段序列二次规划算法（Piecewise Sequential Quadratic Programming, PWSQP）。

首先，针对决策变量过多的问题，本文通过分析基于系统语义，并发掘决策变量之间的关联性，以减少决策变量的数量并简化目标函数；其次，针对目标函数与约束条件的非光滑特性，本文通过分类讨论的方式消除最大值与最小值函数，从而将原问题分解为多个区间段下连续可微的子优化问题。最后，在每个光滑区间内，采用序列二次规划算法进行局部最优解搜索，通过整合各区间的最优解，将连续域上的解映射到离散域上。

（1）目标函数化简

基于弹性容器服务开发透明的特性，本文从开发者的视角出发进行问题求解。由于容器调度策略与短路读优化机制均由云厂商在基础设施层实现，对开发者完全透明，故开发者无需选择短路读优化的决策。因此，本文将缓存池资源配置与短路读优化进行解耦处理，即开发者负责缓存池资源配置，云厂商负责短路读优化。

具体地，首先针对$I_{SC}=0$（不考虑短路读策略）的情况，对优化问题进行求解，获得最优的缓存池资源配置方案（本小节研究内容）。在此基础上，云厂商可根据开发者提供的缓存池配置，通过缓存 Worker 容器与推理服务容器的亲和调度实现短路读优化策略，进一步加速模型加载过程（具体优化策略详见 4.3.3 小节）。

首先，本文针对$I_{SC}=0$的情况对目标函数进行化简，并考虑决策变量之间的关联性，对决策变量空间做降维处理。如公式(4-10)所示，由于缓存 Worker 容器内存大小不影响模型加载时间，故先消除n_{mem}相关的项；在此基础上，由于容器的数据访问带宽B_{cache}与容器的 vCPU 核数n_{cpu}成线性关系[40]，故可以使用B_{cache}来表示n_{cpu}。新的金钱成本记为$cost'$。

$$\begin{aligned} cost &= t_{load} * n_{cache} * (cost_{diskType} * capacity + cost_{cpu} * n_{cpu} + cost_{mem} * n_{mem}) \\ &\rightarrow t_{load} * n_{cache} * (cost_{diskType} * capacity + cost_{cpu} * n_{cpu}) \\ &= t_{load} * n_{cache} * \Big(cost_{diskType} * capacity + cost_{cpu} * (\gamma * B_{cache} + \delta)\Big) = cost' \end{aligned} \tag{4-10}$$

进一步地，如公式(4-11)所示，将模型加载时间的计算表达式(4-5)代入t_{load}中后，通过对$\min(tp,B)$函数的分类讨论，可将原目标函数拆分成两个区间段上的分段函数。由于tp的计算公式(4-2)仍包含最小值函数，所以通过类似的分类方法，可进一步地将原函数拆分成四个区间段上的分段函数，且每个区间段的函数均连续可微。为了简化表示，本文定义$tp_1 = \alpha_{diskType} * capacity + \beta_{diskType}$，$tp_2 = maxThroughput_{diskType}$。

$$cost' = t(M, \min(tp,B)) * n_{cache} * \left(cost_{diskType} * capacity + cost_{cpu} * (\gamma * B_{cache} + \delta)\right) \quad (4\text{-}11)$$

$$= \begin{cases} e^{\theta_2} * tp^{\theta_1} * n_{cache} * \left(cost_{diskType} * capacity + cost_{cpu} * (\gamma * B_{cache} + \delta)\right), tp \le B \\ e^{\theta_2} * B^{\theta_1} * n_{cache} * \left(cost_{diskType} * capacity + cost_{cpu} * (\gamma * B_{cache} + \delta)\right), tp > B \end{cases}$$

$$= \begin{cases} e^{\theta_2} * tp_1^{\theta_1} * n_{cache} * \left(cost_{diskType} * capacity + cost_{cpu} * (\gamma * B_{cache} + \delta)\right), tp_1 \le B \wedge tp_1 < tp_2 \\ e^{\theta_2} * tp_2^{\theta_1} * n_{cache} * \left(cost_{diskType} * capacity + cost_{cpu} * (\gamma * B_{cache} + \delta)\right), tp_2 \le B \wedge tp_1 \ge tp_2 \\ e^{\theta_2} * B^{\theta_1} * n_{cache} * \left(cost_{diskType} * capacity + cost_{cpu} * (\gamma * B_{cache} + \delta)\right), tp_1 > B \wedge tp_1 < tp_2 \\ e^{\theta_2} * B^{\theta_1} * n_{cache} * \left(cost_{diskType} * capacity + cost_{cpu} * (\gamma * B_{cache} + \delta)\right), tp_2 > B \wedge tp_1 \ge tp_2 \end{cases}$$

至此，金钱成本函数被转化为四个区间段上的分段函数，每个区间段上的函数均连续可微。另外，由于云盘类型$diskType$种类为有限的离散值，故本文采用枚举法遍历所有可能的云盘类型，将$diskType$从决策变量集中移除，对于每一个云盘种类求解最优化问题即可。因此，优化问题的决策变量最终化简为云盘容量$capacity$，缓存 Worker 容器数量n_{cache}以及缓存 Worker 的数据访问带宽B_{cache}，新的目标函数记为$objFunc(capacity, n_{cache}, B_{cache})$。

（2）约束条件化简

针对(4-9)中的约束条件，本文新增决策变量上下界约束，将决策变量的约束条件转化如下：

$$\begin{cases} t(M, \min(tp,B)) \le T_{SLO} \\ minCapacity_{diskType} \le capacity \le maxCapacity_{diskType} \\ capacity * n_{cache} \ge M \\ n_{cache_{min}} \le n_{cache} \le n_{cache_{min}} \\ B_{cache_{min}} \le B_{cache} \le B_{cache_{max}} \end{cases} \quad (4\text{-}12)$$

其中，$maxCapacity_{diskType}$、$minCapacity_{diskType}$为$diskType$类型云盘的容量上下界，$n_{cache_{min}}$、$n_{cache_{min}}$、$B_{cache_{min}}$、$B_{cache_{max}}$为新增的上下界约束。针对模型加载时间约束$t(M, \min(tp,B)) \le T_{SLO}$，本文基于目标函数的四个区间段上的自变量定义域，将其转化为等价的连续可微函数约束形式。具体地，通过分

析四个区间段上tp_1, tp_2, B的相对大小关系，可将$t(M, \min(tp, B))$的最值函数展开，将原始的模型加载时间约束条件分解为四个连续可微的子约束，分别为：$e^{\theta_2} * tp_1^{\theta_1} \leq T_{SLO}$，$e^{\theta_2} * tp_2^{\theta_1} \leq T_{SLO}$，$e^{\theta_2} * B^{\theta_1} \leq T_{SLO}$和$e^{\theta_2} * B^{\theta_1} \leq T_{SLO}$。

最终，简化后的优化问题如公式(4-13)所示，记为问题 **P2**。

$$\begin{aligned} \underset{capacity, n_{cache}, B_{cache}}{\operatorname{argmin}} \quad & objFunc(capacity, n_{cache}, B_{cache}) \\ s.t. \quad & t(M, \min(tp, B)) \leq T_{SLO} \\ & minCapacity_{diskType} \leq capacity \leq maxCapacity_{diskType} \\ & capacity * n_{cache} \geq M \\ & n_{cache_{min}} \leq n_{cache} \leq n_{cache_{min}} \\ & B_{cache_{min}} \leq B_{cache} \leq B_{cache_{max}} \end{aligned} \tag{4-13}$$

$objFunc$函数在每个分段区间内均具有连续可微的特性，且约束条件同样满足光滑性要求，故每个分段区间上的子问题均为独立的约束优化问题。本文提出了 PWSQP 算法在连续域内对问题进行求解，即在每个分段上使用序列二次规划算法[40]（Sequential Quadratic Programming, SQP）进行局部优化求解，最终综合各个区间段上的结果获得最优解。

序列二次规划是一种高效的非线性约束优化算法，其核心思想是将原问题转化为一系列二次规划（Quadratic Programming, QP）[41]子问题，通过迭代求解这些二次规划子问题，逐步逼近原问题的局部最优解。该方法结合了牛顿法的快速收敛性和对约束条件的直接处理能力，尤其适用于目标函数或约束高度非线性的复杂优化问题。本文在此基础上提出的 PWSQP 算法流程如算法 4.1 所示。

首先，根据算法输入的模型加载时间 SLO、推理服务容器数、模型大小、云盘种类构建优化问题，获得该优化问题中每个区间段上的目标函数以及对应约束条件（第 2 行）；其次，遍历每个区间段，获得对应区间上的目标函数与约束条件（5~7 行）；然后，使用序列二次规划算法（SQP）对该约束优化问题进行求解，获得最优的缓存池资源配置以及对应的目标函数值（8~14 行）；最后，如果目标函数值小于全局最优值，则对全局最优值以及最优资源配置进行更新（15~18 行）；每个区间段遍历结束后，返回最优资源配置（第 20 行）。

具体地，使用 SQP 方法对约束优化问题求解的步骤如下：首先，随机生成初始缓存池资源配置$config_0$，根据当前的目标函数以及约束条件生成拉格朗日乘子（Lagrangian）[42]；其次，根据拉格朗日乘子推导相应的 KKT（Karush-Kuhn-

Tucker）条件[43]，KKT 条件是非线性规划最优解的必要条件，包括常方程式、原始可行性、对偶可行性以及互补松弛条件，能够处理边界解和内部解的情况；然后，根据 KKT 条件与当前解$config_k$，将寻找下一个解的迭代方向d的过程，转化为对一个二次规划问题（QP）P_k的求解，求解二次规划问题通常可使用变量消去法、内点法（interior point）[44]和积极集法（active set）[45]等方法；最后，检查收敛条件，若满足条件则停止迭代，否则继续下一轮迭代。

算法 4.1：分段序列二次规划算法 searchConfigBySequentialQP

Input：模型加载时间要求T_{SLO}，推理服务容器数n_{infer}，模型大小$size$，云盘种类$diskType$
Output：缓存配置$optimalConfig$
Begin
1. // 根据输入获取各个分段上的目标函数与约束条件
2. $objFuncList, constraints = getObjFuncAndConstraints(T_{SLO}, n_{infer}, size, diskType)$
3. $minObjVal = float("inf")$
4. $optimalConfig = None$
5. **For** $i = 0$ **to** $len(objFuncList)$ **do**
6. $objFunc = objFuncList[i]$
7. $constraint = constraints[i]$
8. $config_k = randomSolution()$
9. 根据$objFunc$与$constraint$计算拉格朗日乘子
10. **Repeat**
11. 根据问题的 KKT 条件和$config_k$，构造一个二次规划问题P_k
12. 求解P_k，将其解对应的梯度方向作为迭代方向d
13. $config_{k+1} = config_k + d$
14. **Until** $|config_{k+1} - config_k| < \epsilon$（$\epsilon$趋近于 0）
15. **If** $objFunc(config_{k+1}) < minObjVal$ **then**
16. $minObjVal = objFunc(config_{k+1})$
17. $optimalConfig = config_{k+1}$
18. **End**
19. **End**
20. **Return** $optimalConfig$
End

由于序列二次规划算法应用广泛，在 Python 提供的 SciPy[46]库有对应的实现（scipy.optimize.minimize(method='SLSQP')）。本文在此基础上实现了 PWSQP 算法，并提出了基于 PWSQP 的缓存池资源配置算法。缓存池及资源配置算法过程如算法 4.2 所示。

首先，依次遍历云盘类型，对于每一种云盘类型，调用 PWSQP 算法获得当前云盘下的最佳缓存池资源配置于目标函数值，并动态更新全局的最佳缓存池资源配置（3~10 行）；其次，根据最佳配置中的缓存 Worker 容器数据访问带宽B_{cache}，计算对应的 vCPU 核数n_{cpu}（11~14 行）；然后，使用二分查找定位满足n_{cpu}需求

的最小的 vCPU 核数（第 15 行）；最后，对连续域内的优化结果进行离散化处理，通过对缓存资源值取整，返回离散域内的资源配置值（第 16 行）。为了防止模型加载时间违反 SLO 的情况，在$round$函数内会先采用四舍五入的取整方法，然后将配置值代入公式(4-5)计算预估模型时间，若违反 SLO 约束，则通过递增缓存 Worker 容器的数量n_{cache}的方式，直至满足服务质量要求。

算法 4.2：基于 PWSQP 的缓存池资源配置算法

Input：模型加载时间要求T_{SLO}，AI 模型对象$model$，推理服务容器数n_{infer}，云盘类型集合Φ，vCPU 核数取值集合C
Output：推荐的缓存池配置$config$
Begin
1. $minObjVal = float("inf")$
2. $config = None$
3. **Foreach** $diskType$ **in** Φ **do**
4. $capacity, n_{cache}, B_{cache} = searchConfigByPiecewiseSQP(T_{SLO}, n_{infer}, model.size, diskType)$
5. $currObjVal = getObjVal(capacity, n_{cache}, B_{cache}, diskType)$
6. **If** $currObjVal < minObjVal$ **then**
7. $minObjVal = currObjVal$
8. $config = (diskType, capacity, n_{cache}, B_{cache})$
9. **End**
10. **End**
11. $n_{cpu} = \gamma * config.B_{cache} + \delta$
12. **If** $n_{cpu} < 0$ **then**
13. $n_{cpu} = 0.25$
14. **End**
15. $num_{cpu} = binarySearch(C, n_{cpu})$
16. **Return** $(config.diskType, round(config.capacity), round(config.n_{cache}), num_{cpu})$
End

进一步地，对算法 4.1 和算法 4.2 的时间复杂度进行分析。对于算法 4.1，假设序列二次规划算法需要经过k次迭代，每次迭代的时间复杂度$O((n+m)^3)$，其中n为决策变量的数量，m为约束数量。考虑到目标函数被划分为 4 个连续可微的分段区间，因此算法 4.1 的总体时间复杂度为$O(4*(n+m)^3)$。

对于算法 4.2，其时间复杂度由两部分构成：首先，遍历了所有云盘类型并调用算法 4.1 进行优化求解，时间复杂度为$O(4*|diskType|*(n+m)^3)$；其次，在 vCPU 核数的离散取值空间C中使用二分查找确定最小核数，时间复杂度为$O(\log|C|)$。因此，算法 4.2 的时间复杂度为$O(4*|diskType|*(n+m)^3+\log|C|)$。由于在实际场景中，云盘类型数量为 4，变量数量为 3，约束数量为 5，故总体时间复杂度在可接受范围内。

4.3.3 短路读优化策略

开发者可使用 4.3.2 节的 PWSQP 算法获得缓存池资源配置，在此基础上，云厂商可进一步通过缓存 Worker 容器与推理服务容器的亲和调度实现短路读优化策略，进一步加速模型加载过程。本小节从云厂商的角度介绍短路读优化策略。

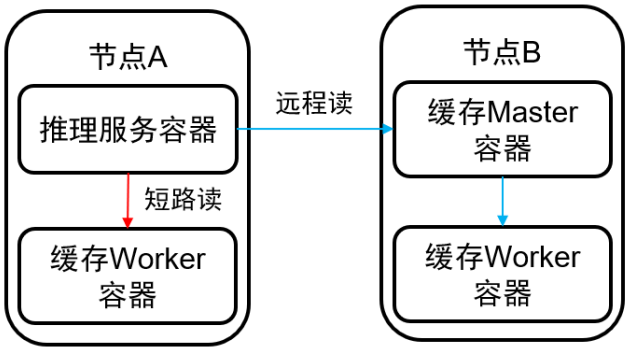


图 4.6 短路读优化策略

如图 4.6 所示展示了短路读优化的示意图。若推理服务容器与缓存 Worker 容器不位于同一节点，推理服务容器需要通过缓存 Master 容器，读取远程节点上的缓存 Worker 容器中的 AI 模型缓存，这个过程涉及跨节点的网络通信；若推理服务容器与缓存 Worker 容器位于同一节点，那么推理服务容器可以绕过缓存 Worker 容器，直接通过本地文件系统读取存储在本节点的 AI 模型缓存，以加速模型的加载过程。

云厂商可以根据开发者提供的缓存池资源配置分别预估短路读情况下的模型加载时间t_2与非短路读情况下的模型加载时间t_1，如果满足$t_1 > t_2$，则可以将推理服务容器调度与任意一个缓存 Worker 容器调度到同一个节点上，以加速模型加载的过程。

代入公式(4-3)与公式 (4-4)后，不等式关系可以转化为：

$$t(M, min\ (tp, B)) > t(M - M_{local}, min\ (tp, B)) + t(M_{local}, tp) \quad (4\text{-}14)$$

进一步地，按照模型大小的比例对时间进行缩放，可以将公式(4-14)转化为公式(4-15)的形式：

$$t(M, min\ (tp, B)) > \frac{M - M_{local}}{M} t(M, min\ (tp, B)) + \frac{M_{local}}{M} t(M, tp) \quad (4\text{-}15)$$

接下来，将模型加载时间表达式公式(4-7)代入公式(4-15)，可以获得短路读优化策略的触发条件，如公式(4-16)所示：

$$\min\left(\alpha_{diskType} * capacity + \beta_{diskType}, maxCapacity_{diskType}\right) * N_{infer} > n_{cache} * B_{cache} \tag{4-16}$$

当开发者提供的配置满足不等式(4-16)的条件时，云厂商可以通过将推理服务容器与缓存 Worker 容器调度到同一节点上，开启短路读策略，加速模型加载的过程。

4.4 实验评估与分析

本节通过一系列实验评估本章提出方法的有效性。首先介绍本节的实验评估环境与配置，其次对模型加载时延预估方法的准确性进行评估，接下来对模型缓存池配置算法的有效性和搜索效率进行对比评估，最后评估短路读策略的有效性。

4.4.1 实验环境与配置

本节将分别介绍实验评估的硬件环境、软件环境、使用的数据集及评价指标等信息。

（1）硬件环境

本节实验的硬件环境与第三章实验相同，参考 3.4.1 节。

（2）软件环境

本节实验的硬件环境与第三章实验相同，参考 3.4.1 节。

（3）数据集与工作负载

在数据集方面，本节采用的 AI 模型与第三章相同，参考 3.4.1 节。

在工作负载方面，本节采用具有不同 SLO（服务质量目标）的 AI 模型推理任务进行实验。模型加载时间 SLO 分别为 30s，40s，50s，60s，70s。

在执行模型推理任务时，模型推理服务容器实例数量设置为 2，容器数据访问带宽为 10Gbps。

表 4-2 阿里云 ESSD 云盘参数

云盘类型	$\alpha_{diskType}$	$\beta_{diskType}$	最小容量	最大容量	最大吞吐量	单价（元/GB/小时）
ESSD0	0.25	100	1GB	65536GB	100MB/s	0.00105
ESSD1	0.25	120	20GB	65536GB	350MB/s	0.0021
ESSD2	0.25	120	461GB	65536GB	750MB/s	0.0042
ESSD3	0.25	120	1261GB	65536GB	4000MB/s	0.0084

模型缓存池资源配置参考阿里云配置的规格与计费方式，云盘类型包括ESSD0~ESSD3[39]，具体参数与计费单价见表 4-2。缓存 Worker 容器参考通用型 ECI 规格配置[38]，其中缓存 Worker 容器 vCPU 的单价为 0.1764 元/核/小时，缓存 Worker 容器内存的单价为 0.0221 元/GB/小时，缓存 Worker 的 vCPU 核数范围为 1~64，内存大小范围为 2~512GB，数据访问带宽范围为 1~20Gbps。

（4）评价指标

本节实验采用R^2系数[47]、平均相对误差MRE、模型加载金钱成本、模型加载时间约束违反率、参数搜索时间作为评价指标。

R^2系数和平均相对误差MRE用于评估模型加载时延预估方法的准确性。其中R^2系数的计算方式见公式(4-17)，平均相对误差的计算方式见公式(4-18)。其中，n为样本数量，y_i为第i个实际值，$\bar{y_i}$为实际值的均值，$\hat{y_i}$为第i个预测值。

$$R^2 = 1 - \frac{\sum_i (\hat{y_i} - y_i)^2}{\sum_i (\bar{y_i} - y_i)^2} \tag{4-17}$$

$$MRE = \frac{1}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y_i}}{y_i} \right| \tag{4-18}$$

模型加载时间违反率用于评估模型缓存池资源配置算法的有效性。假设模型加载任务运行总次数为m，违反模型加载时间的次数为n，则违反率为$\frac{n}{m}$。

（5）对比方法

网格搜索算法（**GS**）：以一定的步长，在参数空间内进行暴力搜索，返回满足约束条件的最优解。缓存 Worker 数量的范围为 1~10，缓存 Worker 容器数据访问带宽在云厂商提供的可选带宽列表中选取，云盘容量在给定云盘类型对应的容量区间内选择。

遗传算法[56]（**GA**）：一种通用的启发式算法，将缓存池资源配置作为染色体，将目标函数作为适应度函数用于评估染色体的适应性。对于不满足约束的参数，本实验在适应度函数中返回一个无穷大值作为惩罚。

贝叶斯优化算法[22]（**BAYES**）：贝叶斯优化算法适用于目标函数计算成本高、导数难以获取或非凸的情况。它通常通过高斯过程建立目标函数的概率模型，结合采集函数（Acquisition Function）来选择采样点。本文采用的初始采样点个数为 10，每次采样的函数调用次数为 100，对于不满足于约束条件的参数组合，返回无穷大值作为惩罚。

4.4.2 模型加载时延预估方法准确性评估

本小节对模型加载时延预估方法的准确性进行评估。为了建立模型加载时间与数据访问吞吐量的定量关系，本文分别将 1 个模型推理服务容器与 1 个缓存 Worker 容器部署在两个不同的节点上，使用 Linux 网络限速工具 wondershaper 控制缓存 Worker 容器所在节点的数据访问带宽，测量在不同数据访问吞吐量下的模型加载时间，以获取数据集。本实验将数据集按照 4：1 的比例划分训练集和测试集，使用训练集上的数据拟合公式(4-7)中的θ_1与θ_2参数，使用测试集上的数据样本对模型加载时延预估方法的准确性进行评估。

在 stable-diffusion-v2-1 模型上的实验结果如图 4.7 所示。红色曲线为预测值，蓝色散点为真实值。实验表明，本文提出的模型加载时延预估方法较为准确，真实值基本位于测试值附近，平均相对误差为 6.82%。

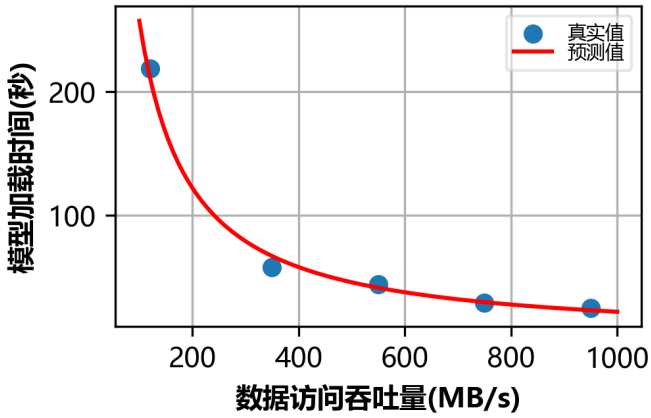


图 4.7 不同数据访问吞吐量下 stable-diffusion-v2-1 模型加载时间的预测值与真实值

另外，本实验还通过R^2系数评估了模型加载时延预估方法在不同模型上的拟合效果，实验结果如表 4-2 所示，表现了较好的拟合效果。

表 4-3 模型加载时延预估方法在不同模型上的拟合效果

模型种类	决定系数R^2
stable-diffusion-v2-1	0.9947
opt-2.7b	0.9785
opt-6.7b	0.9317

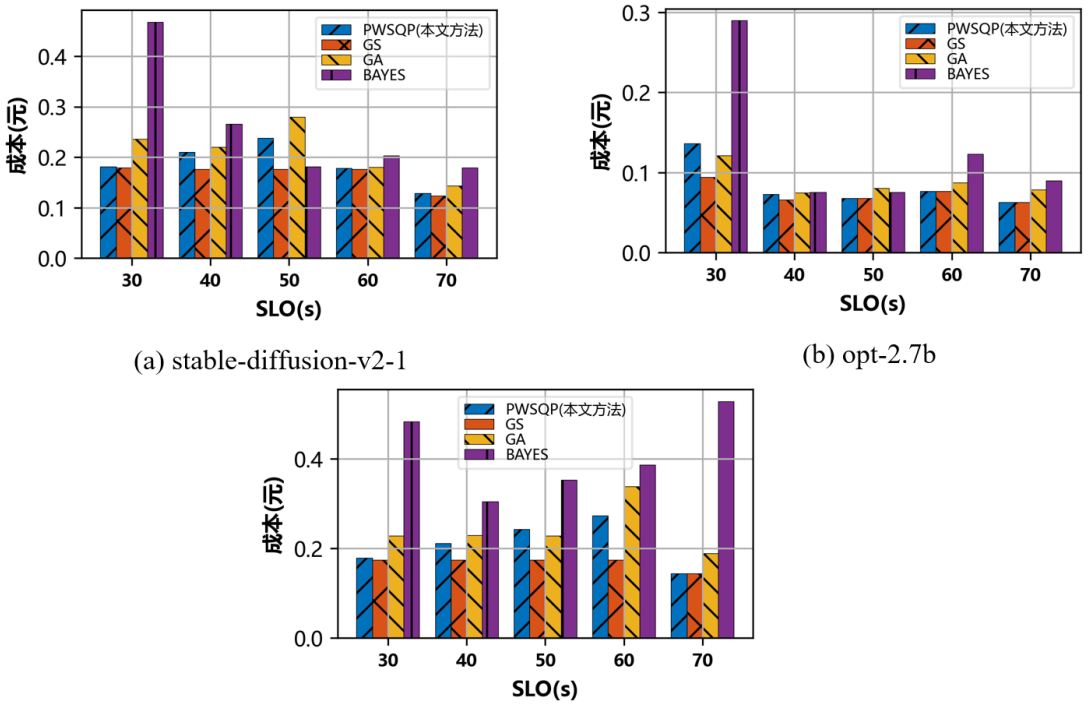
4.4.3 模型缓存池资源配置有效性评估

本小节对模型缓存池资源配置算法的有效性进行评估，分别从金钱成本以及

模型加载时间违反率两个角度进行评估。

本实验在 stable-diffusion-v2-1，opt-2.7b，opt-6.7b 三种 AI 模型上进行实验，模型推理服务容器实例数量设置为 2，vCPU 核数为 24（对应数据访问带宽为 4.5Gbps）。针对模型缓存池资源配置，本实验参考阿里云配置，云盘类型包括 ESSD0~ESSD3[39]，缓存 Worker 容器参考通用型 ECI 规格配置[38]，vCPU 的单价为 0.1764 元/核/小时，缓存 Worker 容器内存的单价为 0.0221 元/GB/小时。本实验对模型推理任务依次设置 30s，40s，50s，60s，70s 的模型加载时间 SLO，比较不同资源配置算法下的金钱成本与 SLO 违反率。

不同数据集下不同资源配置搜索算法对应的金钱成本如图 4.8 所示。横坐标表示不同的模型加载时间 SLO，纵坐标表示模型加载过程中产生的金钱成本，蓝色柱状图为本文提出的 PWSQP 方法。



(a) stable-diffusion-v2-1

(b) opt-2.7b

(c) opt-6.7b

图 4.8 不同数据集下不同资源配置搜索算法对应的金钱成本

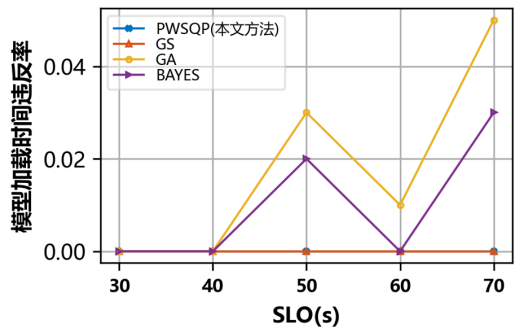
实验结果显示，使用 GS 方法可以获得最低的成本，这是因为 GS 方法遍历所有的缓存池资源配置组合，可以获得理论上最低的金钱成本。进一步地，随着模型加载时间 SLO 的上升，金钱成本呈下降趋势，这是因为 SLO 越宽松，则对缓存池资源配置的要求更低，对应的金钱成本越低。

在大多数情况下，本文提出的 PWSQP 方法可以获得仅次于 GS 方法的金钱

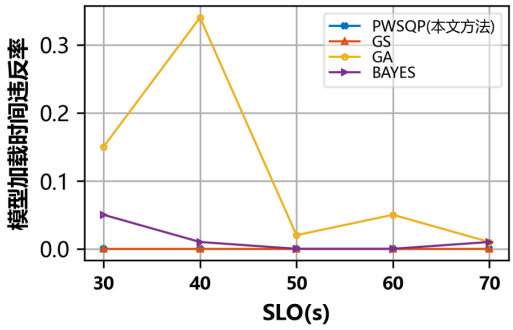
成本，这证明了方法的有效性。另外，在少数情况下，PWSQP 方法获得的金钱成本略高于 GA 方法（比如 SLO=30 秒的 opt-2.7b 模型推理服务），这是因为在算法 4.2 中取整的过程中，可能存在模型加载时间不满足 SLO 的情况，算法通过递增缓存 Worker 容器数量的方式使模型加载时间满足 SLO 要求，导致金钱成本的升高。

本文提出的 PWSQP 方法整体上优于 GA 方法与 BAYES 方法。首先，由于 GA 算法通过模拟种群演化的方式优化目标函数，存在一定的随机性，而且由于数搜索空间大，可能会陷入局部最优解，因此 PWSQP 方法相比于 GA 方法能获得更低的金钱成本；其次，BAYES 方法本质上通过高斯过程模拟目标函数，并通过采样的方式选择采样点，适用于目标函数计算成本高、导数难以获取或非凸的情况，无法结合本文问题的具体特点（目标函数各区间段上连续可微）进行求解，故本文提出的 PWSQP 方法优于 BAYES 方法。

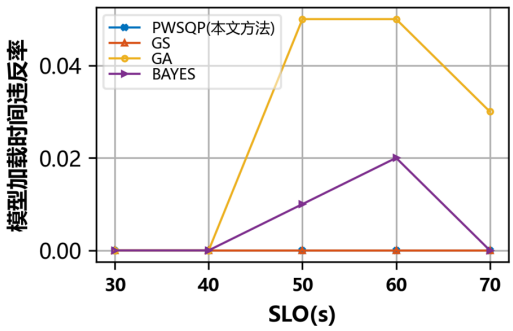
综上所述，本文提出的 PWSQP 方法的模型加载金钱成本相较于全局最优成本（GS 方法）达到次优解，相较于 GA 方法将金钱成本平均降低 10.58%，最多降低 23.62%；相较于 BAYES 方法将金钱成本平均降低 30.16%。



(a) stable-diffusion-v2-1



(b) opt-2.7b



(c) opt-6.7b

图 4.9 不同数据集下不同资源配置搜索算法对应的模型加载时间违反率

不同数据集下不同资源配置搜索算法对应的模型加载时间违反率如图 4.9 所

示。横坐标表示不同的模型加载时间 SLO，纵坐标表示模型加载过程中产生的金钱成本，蓝色曲线为本文提出的 PWSQP 方法。

实验结果表明，本文提出的 PWSQP 方法在不同模型加载时间 SLO 下的违反率均为 0%。这是因为算法 4.2 的取整机制可以严格保证最终的资源配置方案满足模型加载时间要求。由于 GS 方法遍历的全部的参数组合且保证了模型加载时间约束严格成立，因此 GS 方法下的模型加载时间 SLO 违反率也为 0%。由于 BAYES 方法的随机采样以及 GA 方法的随机变异均具有一定的随机性，且在连续解到离散解的映射过程中缺乏严格的约束保证机制，导致存在少量 SLO 违反情况。

4.4.4 模型缓存池资源配置算法搜索效率对比

本小节实验对模型缓存池资源配置算法的搜索效率进行对比。本文通过测量不同资源配置以及不同模型加载时间 SLO 下的平均参数搜索时间，来对比不同资源配置算法的搜索效率。

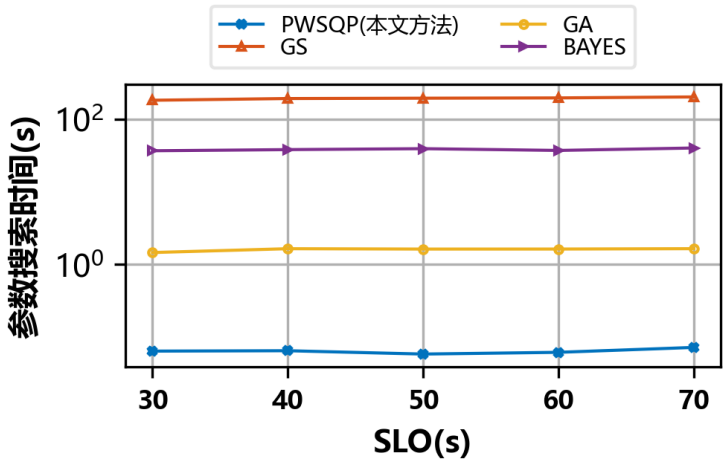


图 4.10 不同资源配置搜索算法下的平均参数搜索时间

实验结果如图 4.10 所示。本文提出的 PWSQP 方法具有最高的参数搜索效率，可以在10^{-2}~10^{-1}级别数量级的时间下完成搜索，这是因为本文通过对问题的分析将目标函数转化为多个区间段上连续可微的目标函数，可以基于梯度迭代的方法求解，具有最高的搜索效率。BAYES 方法与 GA 方法均为启发式的方法，计算的复杂度高，迭代次数多，无法保证较高参数搜索效率。GS 方法采用暴力搜索参数空间的方式获得最优解，虽然可以获得理论上的最低成本，但是参数搜

索耗时最长。

综上所述，本文提出的 PWSQP 方法在参数搜索效率上，相较于 GS 方法提高了 2958 倍的性能，相较于 BAYES 平均提高了 592 倍的性能，相较于 GA 平均提高了 25 倍的性能。

4.4.5 短路读优化有效性评估

本小节实验对短路读优化策略的有效性进行评估。该实验使用 Kuberenetes 集群中的 Pod 模拟弹性容器，通过推理服务 Pod 与缓存 Worker Pod 的亲和性调度实，从而实现短路读策略。由于 stable-diffussion-v2-1、opt-2.7b 和 opt-6.7b 模型大小各异，故该实验将 opt-2.7b 模型推理服务的加载时间 SLO 设置为 30 秒，将 stable-diffusion-v2-1 模型与 opt-6.7b 模型推理服务的 SLO 设置为 60 秒。在这三种测试场景下，分别使用 PWSQP 方法获取各自模型推理服务下的最优配置，且所有测试场景均满足不等式(4-16)所定义的短路读开启条件。接下来，本实验通过推理服务 Pod 与缓存 Worker Pod 的调度，对比启用与禁用短路读策略下的模型加载时间，以验证该优化策略的有效性。

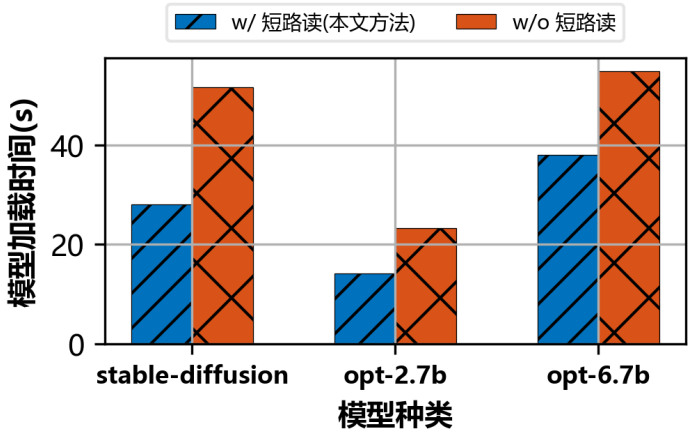


图 4.11 不同模型下短路读策略对模型加载时间的影响

图 4.11 的实验结果表明，得益于推理服务 Pod 与缓存 Worker Pod 的亲和性调度，推理服务 Pod 可直接通过访问本地文件系统模型缓存的方式，显著降低了模型加载时间。本文提出的短路读策略相比于不使用短路读策略，可以将模型加载时间平均降低 38.52%。

4.5 本章小结

本章针对云原生容器环境中，AI 模型加载加速场景下的缓存池资源配置优化问题展开研究，解决了成本与服务质量之间的权衡难的问题。首先，本章对问题背景进行分析，明确了在保证模型加载时间约束的前提下实现低成本缓存池配置的问题；其次，本章对模型加载时间进行建模，提出了缓存池资源感知的模型加载时间预估方法；然后，对模型缓存池资源配置问题进行形式化定义，将其描述为约束优化问题，并提出了分段序列二次规划算法对问题进行求解，并基于缓存池资源配置进一步采取短路读优化策略，以加速 AI 模型加载的过程；最后，本章设计了一系列实验对本章的方法进行评估。实验结果表明，本章提出的 PWSQP 方法可以获得接近最优金钱成本，并最高提高了 592 倍的参数搜索效率；短路读优化策略将模型加载时间平均降低 38.52%。

第五章　面向多任务动态场景的模型缓存池弹性伸缩机制

在多任务动态的 AI 模型推理场景下，缓存资源通常不会释放，可以为新的推理服务提供缓存，而模型缓存池中现有的静态的资源配置难以适应动态变化的推理负载。本章针对云原生容器环境下静态缓存难以动态适应变化负载的问题，提出了多任务动态场景下的模型缓存池弹性伸缩机制。本章内容组织结构如下：5.1 节介绍本章的问题背景，5.2 节提出了基于二分图最大匹配的模型缓存资源分配策略，进行多任务下推理服务于模型缓存资源的匹配，5.3 提出了模型加载时间感知的缓存弹性扩缩容算法，5.4 节对本章提出的方法进行评估与分析，5.5 节对本章内容进行总结。

5.1 问题背景与分析

在多任务 AI 模型推理场景中，典型的应用系统通常由多个 AI 推理服务构成流水线，以提供完整的端到端的服务能力[48]。例如，一个智能对话系统至少包括三个核心的 AI 推理服务：语音识别模块（Speech-to-Text）负责讲输入的语音转化为文字，大语言模型模块（LLM）负责根据输入的文本生成相应内容，文本转语音模块（Text-to-Speech）负责将生成的文本内容转化为语音输出[51]。

由于不同 AI 推理服务承担着各自的功能，所以它们通常会使用不同的 AI 模型。在云原生容器场景下，每个 AI 推理服务均需要从远端存储数据源加载相应的 AI 模型。如果分布式模型缓存池中存在和该推理服务相关的 AI 模型，则推理服务可以直接从分布式模型缓存池中复用已有的缓存资源，以加速模型推理服务的就绪。

另一方面，AI 推理服务负载通常会随着时间以及用户行为的变化而变化。即使 AI 推理服务与模型缓存池中的分布式缓存资源匹配后，此时静态的缓存配置也难以适应动态变化的推理负载。图 5.1 展示了 NYS Thurway 数据集[50]的负载变化情况。不同曲线代表着不同日期内的请求数随时间的变化，呈现出以下变化规律：首先，白天的请求数明显大于夜晚请求数，表明推理负载受同一天内时间的影响；其次，在同一天内，负载的波动较为明显，且会出现负载激增和激减的情况（比如 10/01 曲线 6：00 以后的负载）。由此可知，AI 推理服务的负载收

到日期、时间、用户行为等多重因素的影响，且突发的负载是无法准确预测的。因此，云上的 AI 推理系统通常会根据推理负载 RPS（请求数量/秒）对 AI 推理服务的容器进行弹性伸缩，以满足流量的动态变化。

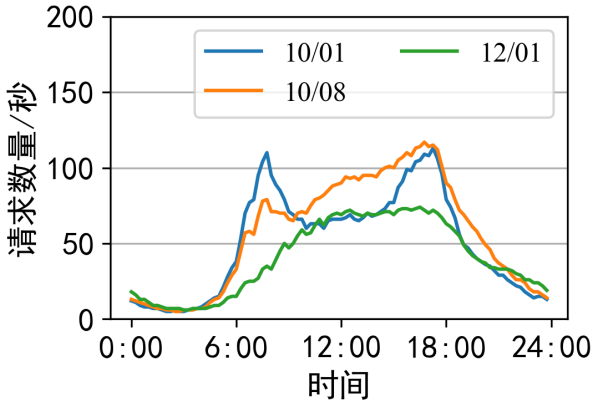


图 5.1 NYS Thrway 数据集负载变化

进一步地，AI 推理服务所匹配的缓存资源也需要动态适应 RPS 的变化，以满足推理服务的就绪时间要求。图 5.2 展示了多任务动态场景下的容器扩缩容过程。首先，当一批推理任务到来时，为了充分复用模型缓存池中空闲的缓存资源（每个缓存资源均为一个分布式缓存集群），需要为每个推理服务匹配合适的缓存资源以供复用，以加速推理服务的初始化过程；接着，系统根据实时监控的 RPS 指标动态调整推理服务容器的数量，当负载增加时，扩展容器实例以满足用户对推理的实时性要求，当负载降低时，缩减容器实例以提高资源利用率；最后，由于推理服务的容器数量会影响 AI 模型加载时间，因此系统需要对相应分布式缓存资源中的缓存 Worker 容器数量进行适应性的水平扩缩容，确保模型加载时间维持在 SLO 要求的范围内。

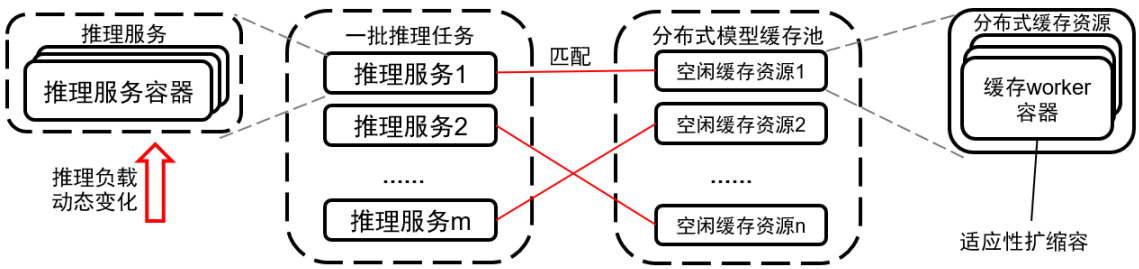


图 5.2 多任务动态场景下的容器扩缩容

综上所述，本章主要解决多任务动态推理场景下的模型缓存资源复用与模型缓存扩缩容的问题。进一步地，划分为以下两个子问题：

子问题 1：如何为应用系统中的一批推理服务分配缓存池中已有的模型缓存资源，确保最短的服务就绪时间？

子问题 2：如何根据推理任务动态变化的负载，对缓存池中相应的缓存容器弹性扩缩容，既保证模型加载时间按不超过 SLO 约束，又防止缓存资源的浪费？

对于子问题 1，本章在 5.2 节提出了基于二分图最大权匹配的模型缓存资源分配策略；对于子问题 2，本章在 5.3 节提出了 RPS 感知的推理服务容器水平伸缩算法。

5.2 基于二分图最大权匹配的模型缓存资源分配策略

本节首先介绍多任务场景下，AI 推理服务与模型缓存资源匹配问题的形式化描述，然后将资源分配问题转化为二分图最大权匹配问题进行求解，对资源分配策略算法做出描述。

5.2.1 问题定义与形式化

如 5.1 节所述，在多任务场景下，一个应用系统通常会包含一批 AI 推理服务，当应用系统首次提供服务时，每个 AI 推理服务均需要从远端存储数据源加载相应的 AI 模型。为了加速模型推理服务的就绪过程，AI 推理服务可以通过复用模型缓存池中已有缓存资源的方式，加速模型加载的过程。

针对缓存资源数量大于或等于推理服务数的情况，所有的推理服务均可充分复用模型缓存池中的缓存资源，无需申请新的缓存资源；针对缓存资源数量小于推理服务数的情况，无法为所有推理服务匹配已有的缓存资源，此时需要为未匹配缓存资源的推理服务分配新的分布式缓存集群，以加速模型加载的过程，此时可使用第四章所述方法，创建新的 DataModel，使用 PWSQP 方法对模型缓存池资源进行配置。

本小节针对缓存资源数量大于或等于推理服务数的情况，将 AI 推理服务与已有模型缓存资源的匹配问题做出形式化描述。

设一个应用系统中 AI 推理服务集合$V_1 = \{v_1, v_2, \dots, v_m\}$，其中$v_i\ (1 \le i \le m)$表示推理服务集合中的第$i$个推理服务；模型缓存池中已有的空闲缓存资源集合$V_2 = \{u_1, u_2, \dots u_n\}$，其中$u_j\ (1 \le j \le n)$表示模型缓存池中的第$j$个空闲的缓存资源，且$m \le n$。

对于任意$v \in V_1$，任意$u \in V_2$，可以根据 AI 推理服务v的容器数量、推理容

器数据访问带宽以及空闲缓存资源u的云盘配置、缓存 Worker 容器配置，使用第四章的公式(4-5)计算模型加载时间，记为$weight(v,u)$。

根据以上信息构造二分图$G=(V_1,V_2,E)$，其中边集$E=\{(v,u)|v\in V_1\wedge u\in V_2\}$，权值$weight(v,u)$为预估的模型加载时间。二分图[51]指的是一类特殊的图，它将图的顶点分成两个互斥的集合V_1,V_2，且所有的边均连接了V_1中的一个顶点以及V_2中的一个顶点。本文定义的图G满足二分图的定义。

为了最小化这一批 AI 推理服务的启动时间，需要寻找二分图G中的最小权完美匹配M，M为一组两两无公共点的边集且$|M|=|V_1|$。形式化地，该问题可以被描述为公式(5-1)的形式，记为问题 **P3**。

$$\begin{aligned}&\underset{M}{\operatorname{argmin}}\sum_{(v,u)\in M}weight(v,u)\\&s.t.\quad |M|=|V_1|\\&\qquad M\subset E\end{aligned}\tag{5-1}$$

5.2.2 分配策略算法描述

本小节对 5.2.1 节提出的问题 **P3** 进行求解，将二分图最小权匹配问题转化为二分图最大权匹配进行求解，并对模型缓存资源分配算法做出描述。

首先，找到一个足够大的数C，使得对于所有的边$(v,u)\in E$，都有$C>weight(v,u)$。由于所有边的权值均为正数，于是令：

$$C=1+\max_{v\in V_1,u\in V_2}weight(v,u)\tag{5-2}$$

进而，构造新的二分图$G'=(V_1,V_2,E')$，新的二分图边(v,u)的权值为$weight'(v,u)=C-weight(v,u)$。此时在新图$G'$中求最大权值匹配等价于在原图$G$中求最小权匹配问题，于是原问题可以转化为求二分图$G'$的最大权匹配。形式化地，该问题可以被描述为公式(5-3)的形式，记为问题 **P4**。

$$\begin{aligned}&\underset{M}{\operatorname{argmax}}\sum_{(v,u)\in M}weight'(v,u)\\&s.t.\quad |M|=|V_1|\\&\qquad M\subset E'\end{aligned}\tag{5-3}$$

解决二分图最大全匹配问题通常可使用 Kuhn-Munkres 算法[52]（KM 算法）。其主要思想如下：首先对原图两个集合的做预处理，对点数较少的集合补点，使

得两个点集的大小相同，再将原图不存在的边权重设置为 0；接下来使用 KM 算法通过寻找原图的相等子图、寻找增广路径、调整顶标的方式寻找相等子图中的完美匹配[53]。

由于二分图匹配算法应用广泛，在 Python 提供的 networkx[56]库有对应的实现（networkx.max_weight_matching）。Networkx 是一个用于创建、操作、研究复杂网络结构的工具，本文在此基础上实现了模型缓存资源分配策略。模型缓存分配策略的算法流程如算法 5.1 所示。

算法 5.1：模型缓存资源分配算法

Input：AI 推理服务集合V_1，缓存池空闲缓存资源集合V_2
Output：AI 推理服务与缓存资源的匹配策略M
Begin
1. $edge_matrix = get_loading_time(V_1, V_2)$
2. $G = create_bipartite_graph(V_1, V_2, edge_matrix)$
3. $M = nx.max_weight_matching(G)$
4. **If** $|V_1| > |V_2|$ **then**
5. **Foreach** v_1 **in** V_1 **do**
6. **If** $!M.containsNode(v_1)$ **then**
7. $createCacheByPiecewiseSQP(v_1)$
8. **End**
9. **End**
10. End
11. **Return** M

End

首先，基于 AI 推理服务集合和缓存池空闲资源集合，计算每个推理服务与可用缓存资源之间的预估模型加载时间，构建邻接矩阵（第 1 行）；接着，根据图的边集和点集构造完全二分图，并调用 KM 算法求解该二分图最大权匹配问题（2~3 行）；最后，判断 AI 推理服务和缓存资源的数量关系，如果存在未匹配缓存资源的推理服务，则为其分配新的分布式缓存集群，并调用 PWSQP 方法确定缓存池的缓存 Worker 容器、云盘参数等配置，以满足模型加载时间 SLO 约束（4~9 行）。

在算法 5.1 最耗时的操作为二分图最大权匹配的过程，故算法 5.1 的整体时间复杂度主要取决于该过程，其时间复杂度为$O((|V_1| + |V_2|)^3)$，其中$|V_1|$与$|V_2|$分别为两个二分图的顶点集规模。

5.3 模型加载时间感知的缓存弹性扩缩容算法

本节首先介绍在动态负载场景下，提出 RPS 感知的 AI 推理服务容器水平弹

性伸缩机制，通过实时监控和 RPS 变化，调整推理服务容器实例数量以满足流量需求；接着根据推理服务容器的数量的变化，提出了模型加载时间感知的 AI 模型缓存扩缩容算法，保证模型加载时间满足 SLO 约束要求。

5.3.1 RPS 感知的推理服务容器水平伸缩方法

本小节基于 Kubernetes 集群的 Pod 水平自动扩缩容组件（HPA，Horizontal Pod Autoscaler）组件提供的标准化伸缩流程，提出了 RPS 感知的推理服务容器水平伸缩方法。

HPA 是一种自动化的扩缩容机制，如图 5.3 所示，它通过控制循环（Reconile Loop）不断地采集资源的使用情况（例如 CPU、内存或其他自定义指标），自动调整 Pod（容器或容器组，Kuberenets 中最小的调度单元）数量，以适应工作负载的动态变化。具体地，HPA 通过监控系统采集每个容器的指标，然后将用户定义的目标值和采集到的当前值进行对比，根据两者差异计算所需的 Pod 副本数，最终通知 ReplicaSet 或 Deployment（Kubernetes 中管理 Pod 的对象）调整 Pod 数量。

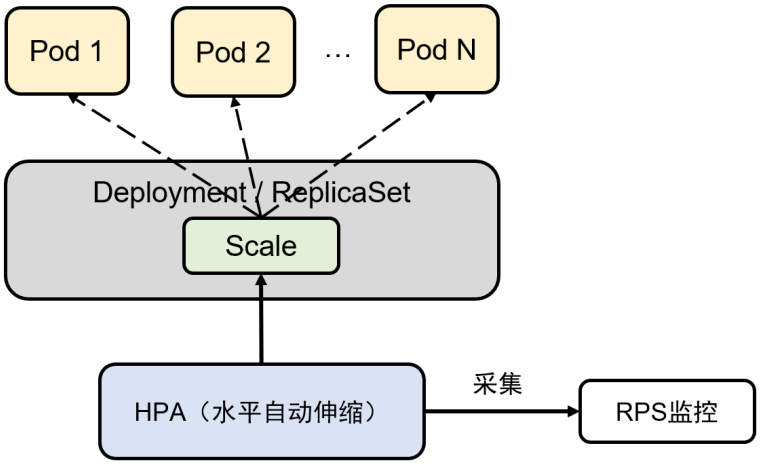


图 5.3 HPA 自动扩缩容机制

在本章所述的场景下，将自定义的推理负载 RPS 作为容器水平伸缩的指标。此时，推理服务容器实例扩缩容的规则可以由公式(5-4)表示。

$$desiredReplicas = \left\lceil currentReplicas * \frac{currentRPS}{targetRPS} \right\rceil \tag{5-4}$$

其中$currentReplicas$为当前的推理服务容器数，$currentRPS$为当前推理负载 RPS，$targetRPS$为用户设定的目标 RPS，$desiredReplicas$为扩缩容之后的推

理服务容器数。本文采用了 Kubernetes 内置的容器水平伸缩方式，根据当前负载与目标值的比例关系，合理计算所需的容器实例副本数，实现了推理服务容器平滑扩缩容。

5.3.2 模型缓存弹性扩缩容算法

本小节在基于 HPA 实现推理服务容器水平弹性伸缩的基础上，进一步对模型缓存 Worker 容器进行弹性扩缩容，在保证模型加载时间按不超过 SLO 约束的同时，防止缓存资源的浪费。具体地，当推理服务容器数量急剧增加时，新启动的推理容器实例为了提供模型推理服务，需要从缓存池中加载 AI 模型，此时原有的缓存 Worker 容器配置可能无法提供足够的聚合带宽，从而导致模型加载时间可能超出 SLO 限制，故需要对模型缓存容器进行扩容；反之，当推理服务容器数量降低时，由于并发负载降低，每个推理服务容器实例往往可以获得足够的缓存聚合带宽，导致现有缓存资源可能出现利用率低的情况，此时需要对缓存 Worker 容器进行缩容以避免资源浪费。

考虑到缓存池的资源配置（包括缓存 Worker 容器的内存、vCPU 数量以及云盘容量和吞吐量）在创建时即已固定，如果要对这些固有配置进行修改，需要销毁旧的容器与云盘并创建新资源，导致显著的时延开销，因此无法做到扩缩容的及时性。因此，本文通过增加或减少缓存 Worker 容器实例实现模型缓存的弹性扩缩容，这种方式有以下几个优点：首先，对容器实例数量的伸缩开销低，无需对所有容器以及云盘的固有配置做出修改；其次，通过调整容器实例数量可以达到改变缓存聚合带宽的目的，从而调整模型加载时间以匹配 SLO 要求。

算法 5.2：模型缓存弹性扩缩容算法

Input：模型加载时间约束 T_{SLO}，当前缓存 worker 容器数n_{cache}
Output：扩缩容后的缓存 worker 容器数量
Begin
1. $t_{load} = getLoadTime(I_{sc}, diskType, capacity, n_{cache}, B_{cache})$
2. **while** $t_{load} \leq T_{SLO}$ **do**
3. $\quad n_{worker} = n_{worker} - 1$
4. $\quad t_{load} = getLoadTime(I_{sc}, diskType, capacity, n_{cache}, B_{cache})$
5. **Return** $\max(1, n_{worker} + 1)$
6. **while** $t_{load} > T_{SLO}$ **do**
7. $\quad n_{worker} = n_{worker} + 1$
8. $\quad t_{load} = getLoadTime(I_{sc}, diskType, capacity, n_{cache}, B_{cache})$
9. **Return** n_{worker}

End

本文提出了一种启发式的自适应的模型缓存扩缩容算法，通过调整缓存

Worker 容器的数量以匹配模型加载时间 SLO 要求。如算法 5.2 所示，其核心思想是通过当前缓存 Worker 容器与云盘配置使用公式(4-5)预估模型加载时间，如果预估模型加载时间小于或等于 SLO，则表示可能存在缓存资源浪费的情况，此时递减缓存容器实例数量，直到预估模型加载时间超过 SLO，最后返回使得模型加载时间恰好不违反 SLO 的缓存容器数量（1~5 行）；如果预估模型加载时间违反 SLO，则递增缓存容器实例数量，直到预估模型加载时间满足 SLO 为止（6~9 行）。

5.4 实验评估与分析

本节设计了一系列实验以评估本章提出的方法的有效性。首先介绍本节的实验评估环境与配置，其次对模型缓存分配策略的有效性进行评估，接下来从容器实例数量的角度验证模型缓存弹性伸缩机制的有效性。

5.4.1 实验环境与配置

本节将分别介绍实验评估的硬件环境、软件环境、使用的数据集及评价指标等信息。

（1）硬件环境

本节实验的硬件环境与第三章实验相同，参考 3.4.1 节。

（2）软件环境

本节实验的硬件环境与第三章实验相同，参考 3.4.1 节。

（3）数据集与工作负载

在数据集方面，本节采用的 AI 模型与第三章相同，参考 3.4.1 节。

在工作负载方面，本节采用了 3.4.1 节所述的 Serverless 请求负载 Azure Function Innovation Trace 2021 作为基础工作负载。为了构造请求负载，本文通过计算每个时间点的请求调用数来生成 RPS 序列，并根据该序列生成以 60 秒为周期的真实调用请求，作为 AI 模型推理服务的负载，并将用户设定的目标 RPS 设置为 10，用于在 5.4.3 节中评估弹性伸缩机制的有效性。

（4）评价指标

本节实验采用推理服务启动时间、模型加载时间、容器实例数量作为评价指标。其中，推理服务启动时间为模型预热时间与模型加载时间之和，用于评估模型缓存分配策略的有效性。

（5）对比方法

ARIMA[63]（Autoregressive Integrated Moving Average Model，自回归差分移动平均模型）：ARIMA 是一种常用的时间序列预测方法，结合了自回归、差分、以及移动平均三个部分。其核心思想是首先利用差分的方法将非平稳的时间序列转化为平稳的时间序列，接着使用自回归部分以及移动平均部分的线性组合表示未来的时序数据，其中自回归部分考虑历史数据对当前值的影响，移动平均部分考虑预测误差对当前值的影响。本文使用 ARIMA 方法根据当前以及历史 RPS 序列对未来的 RPS 序列进行实时预测，并以预测的 RPS 为标准，使用模型加载时间感知的缓存弹性扩缩容算法，计算所需的推理服务容器数与缓存容器 Worker 数量。

5.4.2 模型缓存分配策略有效性评估

本小节通过比较使用缓存分配策略与不使用缓存分配策略两种情况下的推理服务启动时间，对模型缓存分配策略有效性进行评估。其中，不使用缓存分配策略作为基准方法，在一批推理任务到达时，不对模型缓存池中的已有的缓存资源进行复用，而是直接使用 PWSQP 方法创建新的缓存资源供推理服务使用。

本实验对比了 stable-diffusion、opt、mix（既包含 stable-diffusion 又包含 opt 模型服务）三种模型服务的场景下，使用模型缓存分配策略与不使用缓存分配策略下的推理服务启动时间。

实验参数设置如下：每批推理任务包含 5 个推理服务，推理服务的容器资源配置采用随机的方式生成（缓存 Worker 数量设置在 1~10 之间取值，缓存 Worker 的 vCPU 核数在 1~64 之间取值）；缓存 Worker 容器以及云盘的初始配置由 PWSQP 方法确定。

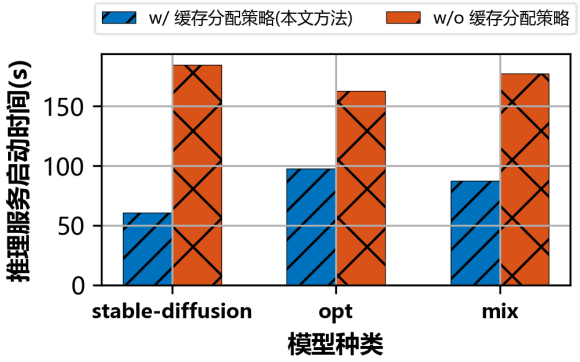


图 5.4 使用/不使用缓存分配策略下的推理服务启动时间对比

图 5.4 展示了本文提出的缓存分配策略与基准方法在推理服务启动时间上的性能对比。实验表明，本文提出的缓存分配策略相较于直接创建新缓存的基准方法，平均降低了 52.57%的推理服务启动时间。这一提升主要取决于以下两个因素：首先，通过二分图最大权匹配算法优化了缓存资源分配，实现了推理服务启动时间的最小化；其次，充分利用现有缓存池资源，避免了重复的缓存资源创建开销和模型预取时间，从而加速推理的服务就绪的过程。

5.4.3 模型缓存池弹性伸缩机制有效性评估

本节对模型缓存池弹性伸缩机制的有效性进行评估。首先，通过跟踪推理服务容器实例随着 RPS 的变化验证推理服务容器水平伸缩方法的有效性；接着，通过跟踪模型缓存容器实例数量随推理服务容器实例数量的变化验证模型缓存弹性扩缩容算法的有效性。

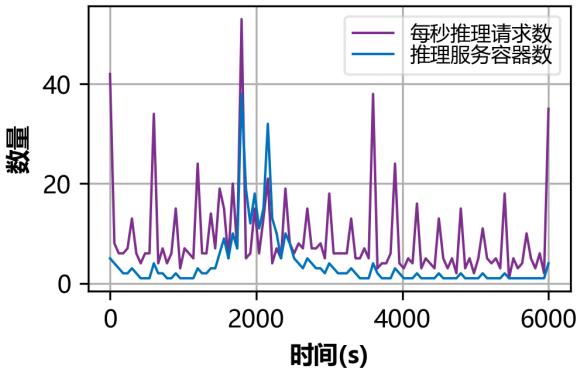
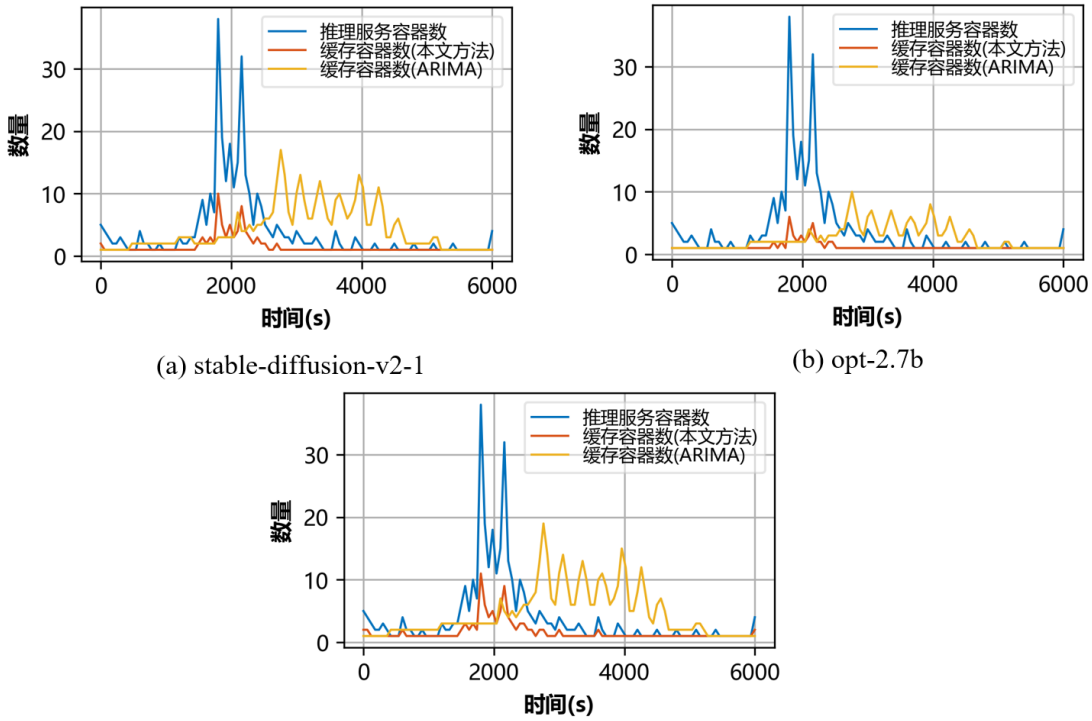


图 5.5 推理服务容器数与 RPS 随时间的变化

图 5.5 展示了推理服务容器实例的数量随着每秒推理请求数（RPS）的动态变化情况，扩缩容的决策周期为 60 秒，紫色曲线表示每秒推理请求数，蓝色曲线表示推理服务容器实例的数量。实验结果表明随着 RPS 的动态波动，推理服务容器实例数量随之变化，例如在 2000 秒前后出现 RPS 剧烈波动时，HPA 准确且及时地对推理服务容器进行水平伸缩以应对突发负载。



(a) stable-diffusion-v2-1

(b) opt-2.7b

(c) opt-6.7b

图 5.6 缓存容器数随推理服务容器数的变化

图 5.6 展示了三种不同模型推理服务下，缓存 Worker 容器实例数量随着推理服务容器实例数的变化趋势，该实验将模型加载时间的 SLO 设置为 75 秒，橙色折线表示使用本文模型缓存弹性扩缩容算法下的缓存 Worker 容器实例数量，蓝色折线为推理服务容器实例数量。实验结果表明在 2000 秒左右时出现了负载高峰期，为应对推理服务容器规模的快速扩展，模型缓存弹性扩缩容算法及时增加了缓存 Worker 容器实例，达到了增加缓存聚合带宽的效果，从而满足模型加载时间 SLO 要求。在 2000 秒以后，由于推理服务负载下降且相对平稳，模型缓存弹性扩缩容算法适应性地对模型缓存进行缩容，在保障模型加载时间 SLO 的前提下节省了缓存资源，实现了资源的高效利用。

另外，本文也对比了基于时序预测方法的缓存伸缩方法（ARIMA），在该对比方法中，使用 ARIMA 方法根据当前以及历史 RPS 序列对未来的 RPS 序列进行实时预测，并以预测的 RPS 为标准，使用本章提出的缓存弹性扩缩容算法，计算所需的推理服务容器数与缓存容器 Worker 数量。图 5.6 的黄色折线表示使用 ARIMA 时序预测算法下的缓存 Worker 实例数量，可见缓存容器的数量难以实时应对 2000 秒左右突发的负载变化。这是因为在实时推理场景中，只能使用推理负载运行过程中实时产生的 RPS 以及历史的 RPS 序列对 ARIMA 模型进行训练，用于 ARIMA 训练的时序数据有限，且 AI 推理服务的突发负载往往难以

预测，故缓存容器数的变化具有一定的滞后性。

综上所述，在多任务动态推理场景下，本章提出模型缓存弹性伸缩机制能有效适应推理服务容器数量的变化，在确保模型加载时间要求的同时，避免了缓存资源的浪费。

5.5 本章小结

本章针对多任务动态推理场景下，静态模型缓存配置难以适应动态变化负载的问题展开了研究，通过基于缓存资源复用的模型缓存分配策略以及模型缓存弹性伸缩机制，提升了推理服务启动效率。首先，本章对问题背景进行分析，提出了多任务动态场景下的缓存资源复用与扩缩容问题；其次，本章将多个推理任务与缓存资源的匹配转化为二分图最大权匹配问题进行求解，实现了模型推理服务启动时间的最小化；进一步地，针对静态缓存配置难以适应动态推理负载的问题，提出了模型加载时间感知的模型缓存弹性扩缩容算法，在保证模型加载时间的前提下实现资源的高效利用；最后，本章设计了一系列实验在真实负载下进行评估。实验结果表明，本章提出模型缓存分配策略平均降低 52.57%的推理服务启动时间，模型缓存弹性扩缩容算法有效适应了动态负载的变化，在保障模型加载时间 SLO 的前提下实现了缓存资源的高效利用。

第六章　系统设计与实现

本章介绍云原生容器环境下 AI 模型推理缓存池编排系统的具体实现，其内容组织如下：6.1 节介绍系统架构，6.2 节详细介绍各组件的实现细节及其运行流程，6.3 节介绍系统运行的示例，6.4 节对本章内容进行小结

6.1 系统架构

针对本文提出的方法和策略，本文基于 Kuberenetes 容器编排工具，设计并实现了云原生容器环境下 AI 模型推理缓存池编排系统。本文提出的系统架构如图 6.1 所示，其中灰色部分为本文实现的组件。

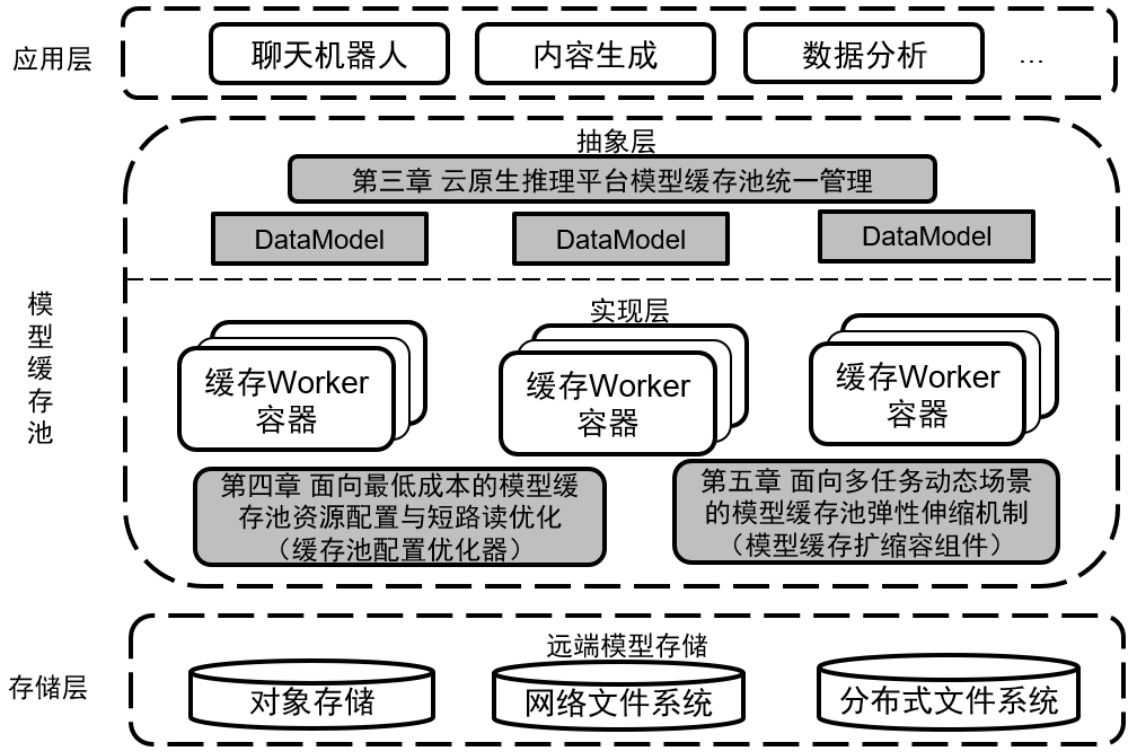


图 6.1 云原生容器环境下 AI 模型推理缓存池编排系统架构图

架构图的最上层为应用层，包括不同种类的 AI 模型服务，开发者可以使用不同的 AI 推理框架为用户提供 AI 推理服务。架构图的最底层为存储层，包括不同种类的模型远端存储数据源，比如 s3 对象存储、webUFS、分布式文件系统 HDFS 等，用于存储 AI 推理服务需要使用的 AI 模型。

架构图的中间层为本文提出的 AI 模型推理缓存池编排系统，实现了第三章

缓存池统一管理的功能。具体而言，缓存池分为两个部分。上层的 DataModel 为缓存池抽象层，DataModel 是 Kuberentes 中由 CRD（Custom Resource Definition，自定义资源描述）创建的 CR（Custom Resource，自定义资源），用于对 AI 模型缓存对象进行抽象和描述，上层的推理应用可以通过挂载 DataModel 的方式无缝接入不同存储源的模型数据，并使用分布式缓存的数据缓存能力；下层的实现层为 DataModel 的具体实现，每个 DataModel 与一个分布式缓存系统进行绑定，分布式缓存系统由一个缓存 Master 组件与多个缓存 Worker 组件以容器化（Pod）的方式部署。分布式缓存池中的 DataModel 控制器负责对缓存创建、绑定、释放等状态进行统一管理。

架构图最下方的缓存池配置优化器与模型缓存池扩缩容组件分别实现了第四章与第五章所述的方法与策略。缓存池配置优化器根据用户提供的 SLO 要求与上层推理服务容器的配置，使用 PWSQP 方法推荐缓存 Worker 容器与云盘资源配置，并通过缓存 Worker 容器与推理服务容器的亲和性调度实现分布式缓存的短路读，加速模型加载的过程；在多任务场景下，缓存池配置优化器通过预估模型加载时间实现推理服务与空闲缓存资源的分配。模型缓存扩缩容组件通过实时监控推理 RPS 以及推理服务容器实例的数量，对分布式缓存资源中的缓存 Worker 容器实例进行水平伸缩，以满足用户的 SLO 要求。

6.2 组件实现与运行流程

本节对本文提出系统的组件实现和工作流程进行详细介绍。

6.2.1 分布式缓存池

分布式缓存池是本系统的核心组件，本系统使用 Alluxio 分布式缓存系统实现分布式模型缓存池。为了将分布式缓存池与 Kuberentes 系统进行集成，本文采用了 Operator[55]的设计模式。Operator 包含了若干个 CRD，每个 CRD 均有对应的控制器组件，用户则可以根据 CRD 创建 CR 对象。本文参考 Fluid[57][58]的设计思想，设计了 DataModel 与 Cache Runtime 两种 CRD 用来描述模型缓存的状态与属性，并且设计相应的控制器管理器生命周期。本系统选用 Alluxio 分布式缓存作为 Cache Runtime 的实现。

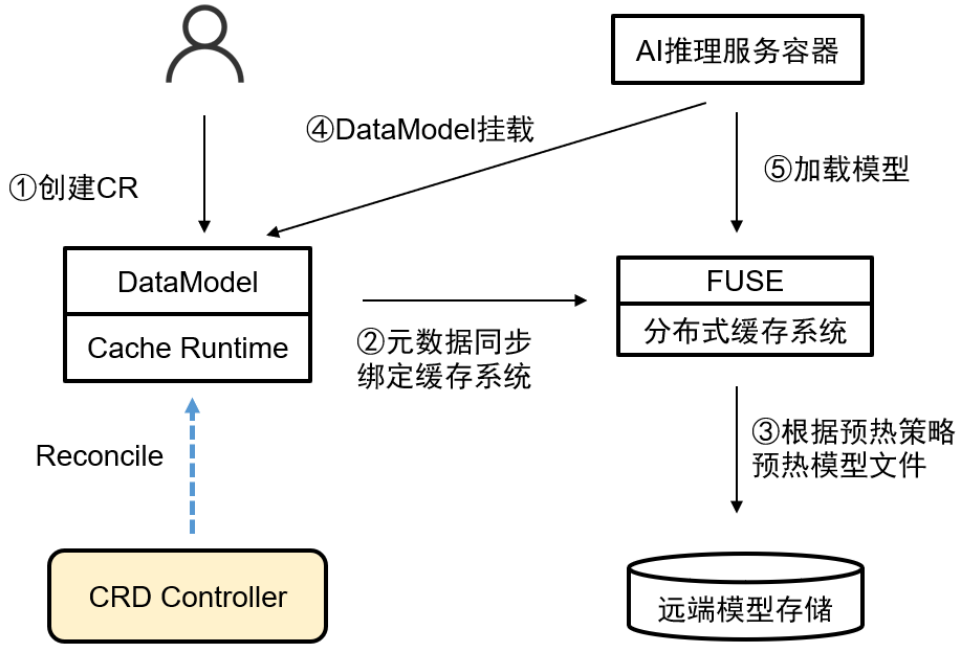


图 6.2 分布式缓存池控制器运行流程

图 6.2 展示了控制器的运行流程。步骤如下：

（1）用户按照 DataModel 与 Cache Runtime CRD 定义的格式，创建 DataModel 以及与其绑定的 Cache Runtime CR 对象。

（2）用户创建 CR 对象后，CRD 控制器开始通过控制循环（Reconcile）管理 DataModel 与 Cache Runtime 的生命周期。系统首先根据缓存池配置优化器推荐的容器与云盘配置创建分布式缓存系统，接着控制器根据 DataModel CR 描述的 AI 模型缓存的存储数据源、访问路径、模型格式等基本属性，为分布式缓存系统同步相应的元数据，进行 DataModel 与分布式缓存系统的绑定。同时，控制器将创建与 DataModel 同名的持久卷声明（PVC，Persistent Volume Claim），以供推理 Pod 后续的挂载。

（3）分布式缓存系统根据实时的推理请求频率统计信息以及 DataModel 描述模型的大小信息，计算每个模型的缓存收益。在此基础上，根据面向缓存收益最大化的预热策略，在缓存 Master 组件上调用 alluxio fs distributedload 命令，从远端模型存储选择性地预热 AI 模型，为后续模型加载的加速做准备。

（4）用户向系统部署 AI 推理服务容器（Pod）后，部署在 Kubernetes 集群中每个节点上的 CSI 存储插件负责 DataModel 的挂载职责。DataModel 具体的挂载过程如下：首先，用户会在 AI 推理服务 Pod 内挂载步骤（2）中的 PVC；然后，CSI 存储插件在推理服务所在节点上创建分布式缓存系统的 FUSE Pod（用户态文件系统程序）；接着，FUSE Pod 在该节点上的文件系统中挂载一个目录；

最后，CSI 插件将该目录挂载到 AI 推理服务 Pod 内的指定位置。

（5）AI 推理服务通过 POSIX 接口，以访问本地文件系统的方式加载模型，并执行后续的模型推理过程。

6.2.2 缓存池配置优化器

缓存池配置优化器负责根据用户提供的加载时间 SLO，使用 PWSQP 方法推荐合适的缓存池配置以及短路读策略。6.2.1 节的控制器组件在步骤（2）中根据该组件推荐的容器与云盘配置创建分布式缓存系统。

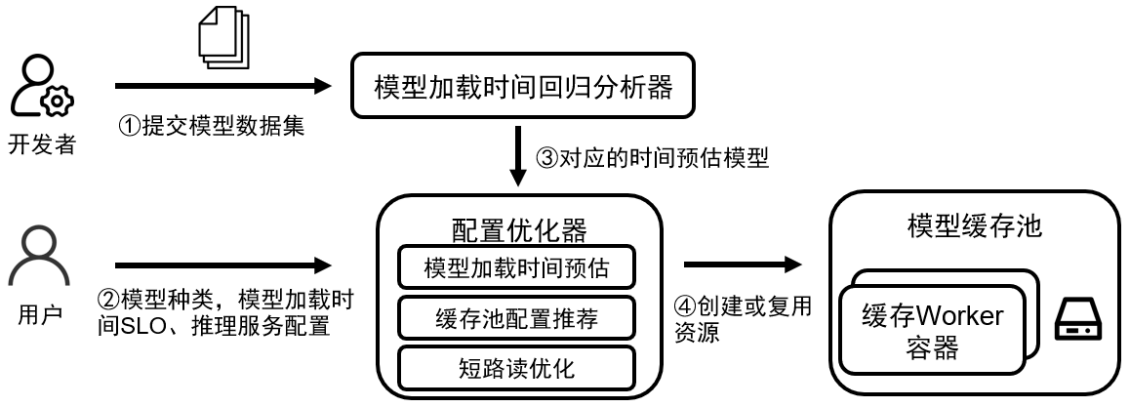


图 6.3 缓存池配置优化器运行流程

缓存池配置优化器的运行流程如图 6.3 所示。步骤如下：

（1）开发者收集模不同的 AI 模型数据集，数据集在离线阶段生成，包括 AI 模型在不同数据访问吞吐量下的模型加载时间。模型加载时间回归分析器对提交的模型数据集进行回归分析，从而构建加载时间预估模型，并将每种 AI 模型及其对应的加载时间预估模型存储在数据库中。

（2）用户需要执行推理任务时，将模型种类、模型加载时间 SLO、推理服务容器的配置提交到配置优化器。

（3）配置优化器根据用户提供的模型种类，从模型加载时间回归分析器中获取对应 AI 模型的加载时间预估模型。

（4）配置优化器根据用户的 SLO，预估不同配置下的模型加载时间，并推荐满足 SLO 约束下成本高效的缓存池配置。根据推荐的缓存池配置在缓存池中创建缓存资源后，通过容器的亲和性调度实现模型加载的短路读优化。另外，在多任务推理场景下，对于批量到来的推理任务，配置优化器对不同缓存配置下的模型加载时间进行预估，通过推理服务与空闲缓存资源匹配的方式实现缓存资源

的复用。

6.2.3 模型缓存扩缩容组件

模型缓存扩缩容组件通过实时监控推理 RPS 以及推理服务容器实例的数量，对分布式缓存资源中的缓存 Worker 容器实例进行水平伸缩，以满足用户的 SLO 要求。

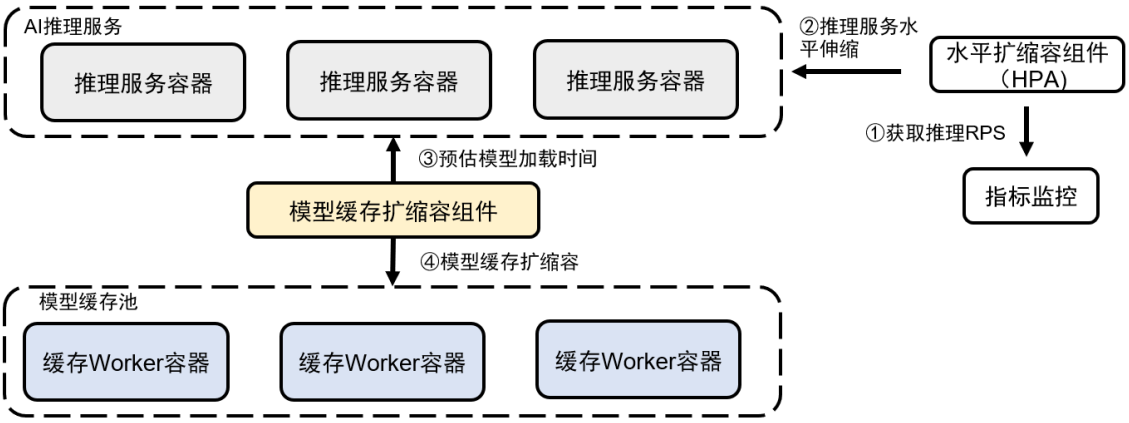


图 6.4 模型缓存扩缩容组件运行流程

模型缓存扩缩容组件的运行流程如图 6.4 所示。步骤如下：

（1）指标监控系统每隔 60 秒收集一次推理 RPS 指标，Pod 水平扩缩容组件（HPA，Horizontal Pod Autoscaler）从获取推理 RPS 实时指标。

（2）HPA 根据预先设置的目标 RPS 值，通过修改推理服务 Deployment 资源对象的 Pod 数的方式，调整推理服务容器副本数，对推理服务进行水平伸缩。

（3）推理服务经过水平伸缩后，模型缓存扩缩容组件根据推理服务的容器配置，以 RPC 的方式调用配置优化器中的模型加载时间预估模块，计算当前缓存池配置下的模型加载时间。

（4）模型缓存扩缩容组件根据模型加载时间，执行模型缓存弹性扩缩容算法，计算扩缩容后的缓存 Worker 容器实例数，通过 Kuberentes Python Client 提供的 API 操作缓存 SatatefulSet 资源对象的缓存 Worker 容器实例副本数，实现模型缓存容器的扩缩容。

6.3 系统运行示例

本节从 opt-2.7b 模型的一次推理过程出发，从用户的角度介绍系统的完整运

行流程。

在用户使用系统前，模型加载时间回归分析器通过离线分析，构建 opt-2.7b 加载时间预估模型，将回归模型的参数以 JSON 格式保存。如图 6.5 所示，其中 intercept 为公式(4-7)中的θ_2参数，coef 为公式(4-7)中的θ_1参数。


```
{
    "intercept": 9.820998957062177,
    "coef": -1.0676760329162287
}
```


图 6.5 opt-2.7b 加载时间回归模型参数

```
disk_id=3 begins! ----------------
seq_id=0
Optimization terminated successfully    (Exit mode 0)
            Current function value: 500.40066091246496
            Iterations: 8
            Function evaluations: 18
            Gradient evaluations: 8
seq_id=1
Positive directional derivative for linesearch    (Exit mode 8)
            Current function value: 1862.7839039254648
            Iterations: 25
            Function evaluations: 109
            Gradient evaluations: 21
seq_id=2
Optimization terminated successfully    (Exit mode 0)
            Current function value: 500.4006675137777
            Iterations: 8
            Function evaluations: 8
            Gradient evaluations: 8
seq_id=3
Optimization terminated successfully    (Exit mode 0)
            Current function value: 1945.7679974936902
            Iterations: 6
            Function evaluations: 6
            Gradient evaluations: 6
disk_id=2, disk_capacity=577, n_cache=1, n_cpu=32  最佳配置
理论成本0.07264133360545322
```

图 6.6 缓存池配置优化器返回最优容器与云盘配置

接下来介绍在线推理的流程。首先，用户向缓存池配置优化器输入模型种类 opt-2.7b、模型加载时间 SLO 以及推理服务容器的配置，优化器返回满足 SLO 约束下成本低廉的云盘配置与缓存 Worker 容器配置。如图 6.6 所示，展示了缓存

池配置优化器运行的部分日志，对第三种云盘类型进行计算时，调用 SQP 算法分别计算了四个区间段上的最优解。最终，综合所有云盘以及所有区间段上的结果，给出最佳配置，即选择 ESSD2 云盘，云盘容量取 577GB，缓存 Worker 容器数设置为 1，vCPU 核数为 32。

接着，用户编写用于描述 DataModel 对象的 YAML 文件，其中包含 opt-2.7b 存储数据源、访问路径、模型格式等属性。缓存配置优化器则根据推荐的缓存 Worker 容器配置与云盘配置，创建用于描述 Cache Runtime 对象的 YAML 文件。用户使用 kubectl 工具将上述两个 YAML 文件提交给 Kubernetes 集群后，系统将推荐的云盘以 PVC 的方式挂载到缓存 Worker 容器所在的 Pod 中。用于描述 DataModel 与 Cache Runtime 对象的 YAML 文件分别如图 6.7 与图 6.8 所示，DataModel 文件中 modelSource、name、path、bin 字段分别描述了 opt-2.7b 的存储数据源、名字、访问路径、模型格式等属性；Cache Runtime 文件的 levels 字段描述了缓存 Worker 容器挂载的云盘类型、路径，worker 字段描述了缓存 Worker 容器的 vCPU 与内存配置。

```yaml
kind: DataModel
metadata:
  name: model
spec:
  models:
    - modelSource: s3://models/opt-2.7b
      name: opt-2.7b
      path: /models/opt-2.7b
      format: bin
      options:
        alluxio.underfs.s3.endpoint: http://192.28.132.21:9000
        alluxio.underfs.s3.disable.dns.buckets: "true"
        alluxio.underfs.s3.inherit.acl: "false"
        s3a.accessKeyId: xxxxx
        s3a.secretKey: xxxxx
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: model-cache
              operator: In
              values:
                - "true"
```

图 6.7 描述 DataModel 的 YAML 文件

```yaml
kind: AlluxioRuntime
metadata:
  name: model
spec:
  replicas: 1
  tieredstore:
    levels:
      - mediumtype: SSD
        path: /home/cqm
        quota: 20Gi
        high: "0.99"
        low: "0.99"
  worker:
    resources:
      requests:
        memory: "4Gi"
        cpu: "32000m"
      limits:
        memory: "4Gi"
        cpu: "32000m"
```

图 6.8 描述 Cache Runtime 的 YAML 文件

此后，系统根据实时的推理请求频率统计信息以及模型的大小信息，根据面向缓存收益最大化的预热策略，在缓存 Master 组件上调用 alluxio fs distributedload 命令，从远端模型存储选择性地预热 AI 模型。在本示例下，系统将对 opt-2.7b 模型进行预热。图 6.9 展示了缓存 Master 组件预热模型成功的日志，预热耗时为 9.166 秒。

```
Successfully loaded path /models/opt-2.7b/.git/objects/e3/ad115fbf9c54bc9d84ced44049740e75c80bd7
Successfully loaded path /models/opt-2.7b/.git/objects/e7/c495d7262254f8c6ef07ea7d385f01a8b70faa
Successfully loaded path /models/opt-2.7b/.git/objects/f0/7423ec7ca6dd12ac6ab1b3337ac2f06bff22ec
Successfully loaded path /models/opt-2.7b/.git/objects/f1/67a85cbf10d845b7ac5673b61a27046a00dc84
Successfully loaded path /models/opt-2.7b/.git/objects/f2/13a3ca283cb8774abedb42bfab88e0a573d71a
Successfully loaded path /models/opt-2.7b/.git/objects/f9/0147b7e36dd15004fe9fb5cb8aa9bfcbacde86
Successfully loaded path /models/opt-2.7b/.git/objects/fc/2edfb22962bf5762b6014920d8af6464d46d62
Successfully loaded path /models/opt-2.7b/.git/packed-refs
Successfully loaded path /models/opt-2.7b/.git/refs/heads/main
Successfully loaded path /models/opt-2.7b/.git/refs/remotes/origin/HEAD
Successfully loaded path /models/opt-2.7b/.gitattributes
Successfully loaded path /models/opt-2.7b/README.md
Successfully loaded path /models/opt-2.7b/config.json
Successfully loaded path /models/opt-2.7b/generation_config.json
Successfully loaded path /models/opt-2.7b/merges.txt
Successfully loaded path /models/opt-2.7b/pytorch_model.bin
Successfully loaded path /models/opt-2.7b/special_tokens_map.json
Successfully loaded path /models/opt-2.7b/tokenizer_config.json
Successfully loaded path /models/opt-2.7b/vocab.json
Total completed file count is 113, failed file count is 0
Finished running the command, jobControlId = 1741231116037
distributedLoad on /models/opt-2.7b ends

real    0m9.166s    模型缓存预热时间
user    0m3.253s
sys     0m0.229s
+ echo -e 'distributedLoad on /models/opt-2.7b ends'
+ (( i++ ))
+ (( i<1 ))
```

图 6.9 模型预热成功日志

最后，用户启动以容器化的方式启动推理服务，并向推理服务提供的接口发起 HTTP 请求执行一次模型推理。模型推理服务利用模型缓存池的能力，直接从分布式缓存中读取模型，达到加速模型的加载效果。图 6.10 展示了模型推理的结果，图 6.11 展示了模型推理容器内的日志，其中模型加载时间为 12.152 秒。

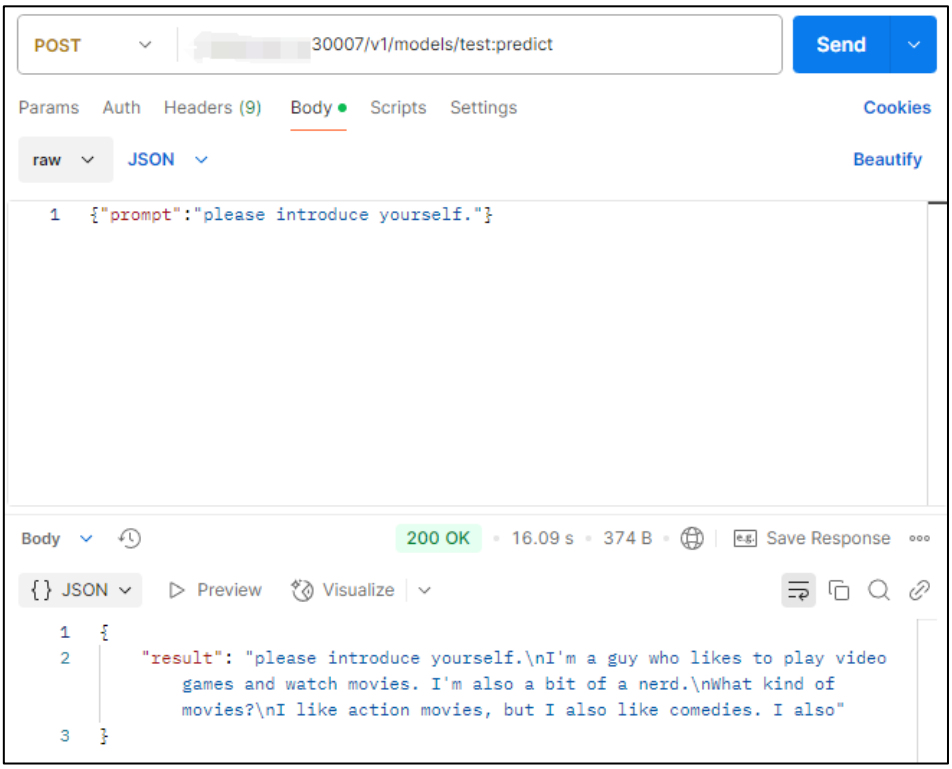


图 6.10 模型推理结果

```
2025-03-06 03:24:40,953 INFO werkzeug MainThread : Press CTRL+C to quit
2025-03-06 03:24:40,961 INFO werkzeug MainThread :  * Restarting with stat
2025-03-06 03:24:42,255 WARNING werkzeug MainThread :  * Debugger is active!
2025-03-06 03:24:42,298 INFO werkzeug MainThread :  * Debugger PIN: 111-903-165
2025-03-06 03:25:43,051 INFO http-server Thread-2 : CUDA is avaiable: True
2025-03-06 03:25:43,051 INFO http-server Thread-2 : Using device: cuda
2025-03-06 03:25:43,060 INFO http-server Thread-2 : Device Count: 1
2025-03-06 03:25:43,060 INFO http-server Thread-2 : Device ID: 0
2025-03-06 03:25:43,060 INFO http-server Thread-2 : Device Name: NVIDIA GeForce RTX 3090
2025-03-06 03:25:43,060 INFO http-server Thread-2 : Memory Usage:
2025-03-06 03:25:43,061 INFO http-server Thread-2 : Allocated: 0.0 GB
2025-03-06 03:25:43,061 INFO http-server Thread-2 : Cached: 0.0 GB
2025-03-06 03:25:43,061 INFO http-server Thread-2 : Loading Model from /data/model
2025-03-06 03:25:43,061 INFO http-server Thread-2 : Loading Model in 8 bit: False
2025-03-06 03:25:43,990 INFO torch.distributed.nn.jit.instantiator Thread-2 : Created a temporary directory at /tmp/tmpd
uzr3gk8
2025-03-06 03:25:43,990 INFO torch.distributed.nn.jit.instantiator Thread-2 : Writing /tmp/tmpduzr3gk8/_remote_module_no
n_scriptable.py
/opt/venv/lib/python3.8/site-packages/torch/_utils.py:776: UserWarning: TypedStorage is deprecated. It will be removed i
n the future and UntypedStorage will be the only storage class. This should only matter to you if you are using storages
 directly.  To access UntypedStorage directly, use tensor.untyped_storage() instead of tensor.storage()
  return self.fget.__get__(instance, owner)()
2025-03-06 03:25:55,212 INFO http-server Thread-2 : Model loading time: 12.151511192321777
2025-03-06 03:25:55,212 INFO http-server Thread-2 : Model loaded successfully
2025-03-06 03:25:55,216 INFO http-server Thread-2 : to GPU time: 0.0033864974975585938
2025-03-06 03:25:59,051 INFO werkzeug Thread-2 : 210.28.132.14 - - [06/Mar/2025 03:25:59] "POST /v1/models/test:predict
HTTP/1.1" 200 -
```

模型加载时间

图 6.11 模型推理容器内日志

6.4 本章小结

本章介绍了云原生容器环境下的 AI 模型推理缓存池编排系统的设计与实现，首先，介绍了该系统的整体框架，并简要介绍系统各组件的功能。然后，对系统各个组件的运行流程进行了详细介绍。最后，通过一个 AI 模型推理的完整示例展示了该系统的运行流程。

第七章　总结与展望

7.1 本文工作总结

如今，以深度学习和大模型为代表的人工智能技术正在飞速发展，人工智能模型（AI 模型）在众多领域得到了广泛的应用，例如数据分析、推荐系统、图像识别、内容生成、聊天机器人等领域。模型推理是释放人工智能技术在行业应用价值的重要途径，当前越来越多行业在加速部署构建 AI 模型推理服务应用。然而，AI 模型推理负载在资源开销和推理性能方面具有低成本、高吞吐、低延迟方面的要求。

得益于云原生应用范式运维便捷、弹性伸缩、按需付费等优点，在云平台上部署 AI 推理服务正在成为一种趋势。弹性容器服务是一种新颖的云原生应用部署方式，用户无需管理底层服务器，只需要提供打包好的容器镜像，自定义容器规格，即可运行容器。然而，云原生容器环境下通常采用存储-计算分离的架构，这导致了以下两个问题：首先，云上异构存储系统使得 AI 模型数据被存储在不同数据源上，导致推理引擎访问数据的方式复杂，模型缓存统一管理困难；其次，存算分离的架构会导致跨网络数据传输，AI 模型加载的高延迟会加剧推理服务冷启动问题，从而影响端到端的推理时延。

现有关于云原生 AI 推理的相关工作主要关注推理时延、吞吐量以及金钱成本优化。然而，这些工作缺少针对 AI 推理服务启动阶段性能与成本的联合优化。为了解决云原生容器环境下 AI 模型推理服务启动慢的问题，本文基于分布式缓存池架构，对模型缓存预热阶段缓存数据的统一访问、模型加载阶段缓存池资源配置成本和服务质量的平衡、以及模型服务运行阶段的模型缓存弹性伸缩三个方面进行了优化，设计并实现了云原生容器环境下 AI 模型推理缓存池编排系统。本文的主要研究工作和贡献点包括：

（1）针对云原生容器环境下模型缓存池统一管理困难的问题，本文首先提出了模型缓存池的统一管理方法，以标准、统一、透明的方式管理 AI 模型缓存。其次，针对 AI 模型推理服务启动过程的预热时间占比大的问题，设计了一种轻量级的近似数据结构实时统计推理请求访问频率，并提出了推理请求动态感知的缓存预热策略，有效降低模型预热时间。

（2）针对云原生容器环境下模型缓存池资源配置成本和服务质量难平衡的

问题，本文提出了面向最低成本的模型缓存池资源配置算法与短路读优化策略。首先对模型加载时间进行建模，提出了缓存池资源感知的模型加载时间预估方法。其次，对模型缓存池资源配置问题进行形式化定义，将其描述为约束优化问题，并提出了高效的分段序列二次规划算法对该优化问题进行求解。最后，从云厂商的角度，进一步通过容器的亲和性调度实现分布式缓存的短路读优化策略，进一步降低模型加载时间。

（3）针对云原生容器环境下静态模型缓存难以适应动态变化的推理负载的问题，本文提出了面向多任务动态场景的模型缓存池弹性伸缩机制。首先将推理任务与缓存资源的匹配转化为二分图最大权匹配问题进行求解，实现了缓存资源的复用。进一步地，提出了模型加载时间感知的模型缓存弹性扩缩容算法，通过动态调整容器实例数量，在满足服务质量的前提下实现资源的高效利用。

（4）基于上述研究，设计并实现了云原生容器环境下的 AI 模型推理缓存池编排系统。该系统构建了统一的模型缓存池，可根据 SLO 约束推荐合适的缓存池配置，并根据动态负载进行资源扩缩容，实现了低时延、低成本的模型服务启动过程。与前沿方法相比，首先，本文提出的缓存预热方法将 AI 模型缓存预热总时间平均降低 25.98%；其次，本文提出的缓存池资源配置算法将模型加载成本最多降低 23.62%，将参数搜索效率提升 25 倍，短路读优化策略将模型加载时间进一步降低 38.52%；最后，本文提出的缓存弹性扩伸缩机制能有效适应推理负载的动态变化。

7.2 下一步工作

本文的下一步工作可以考虑从以下几个方面展开。

（1）本文提出的缓存池编排系统在进行资源配置时仅考虑了模型缓存池云盘吞吐量、容器的数据访问带宽、容器放置策略等因素。然而，在实际的 AI 模型推理平台中，模型加载过程和受更多因素的影响，例如内存带宽、GPU 性能、网络波动等，而且在实际场景中，云厂商通常会提供不同类型的异构实例（例如 GPU 实例、NPU 实例）以供选择。本文在模型加载时间建模的过程中，为了简化问题，将 GPU 配置视为定值，且所有计算实例均为同构实例。在接下来的研究中，本文将考虑异构实例下的模型加载建模，并深入剖析 AI 模型加载到 GPU 的机制，建立更具有普适性的数学模型，提出更具有鲁棒性的模型缓存池资源配置算法。

（2）本文采用的轻量化 AI 模型虽能满足基础实验需求，但在实际场景中，百亿级参数大模型的加载过程涉及 KV Cache 初始化、CUDA 图构建等机制，在模型加载时间的建模中需要考虑大模型加载相关的特征。在接下来的研究中，本文将对大模型的加载过程进行深入研究，例如大模型参数分片加载中的通信开销优化、异构实例的适配性等因素，探索云原生场景下更多加速的空间。

（3）本文仅对云原生环境下的 AI 推理服务启动过程进行了优化，缺少对推理执行阶段的分析与优化。在后续研究中，本文考虑如何充分利用多个弹性容器协同完成一次 AI 模型的推理过程，实现端到端推理的低时延与低成本。

参考文献

[1] Wu Y, Dinh T T A, Hu G, et al. Serverless data science-are we there yet? a case study of model serving[C]//Proceedings of the 2022 International Conference on Management of Data (SIGMOD). 2022: 1866-1875.

[2] Bao K, Zhang J, Lin X, et al. Large language models for recommendation: Past, present, and future[C]//Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR). 2024: 2993-2996.

[3] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 770-778.

[4] Rombach R, Blattmann A, Lorenz D, et al. High-resolution image synthesis with latent diffusion models[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2022: 10684-10695.

[5] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback[J]. Advances in Neural Information Processing Systems (NeurIPS), 2022, 35: 27730-27744.

[6] Jonas E, Schleier-Smith J, Sreekanti V, et al. Cloud programming simplified: A berkeley view on serverless computing[J]. arXiv preprint arXiv:1902.03383, 2019.

[7] AWS Fargate[EB/OL]. https://aws.amazon.com/fargate/. Accessed: 2025-03-02.

[8] Google Cloud Run[EB/OL]. https://cloud.google.com/blog/products/serverless/cloud-run-bringing-serverless-to-containers. Accessed: 2025-03-02.

[9] 阿里云弹性容器实例[EB/OL]. https://www.aliyun.com/product/eci. Accessed: 2025-03-02.

[10] Kubernetes[EB/OL]. https://kubernetes.io/. Accessed: 2025-03-02.

[11] Aslani A, Ghobaei-Arani M. Machine learning inference serving models in serverless computing: a survey[J]. Computing, 2025, 107(1): 47.

[12] San Juan J, Wong B. Reducing the Cost of GPU Cold Starts in Serverless Deep Learning Inference Serving[C]//Proceedings of the 2023 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops). IEEE, 2023: 225-230.

[13] Zhao K, Zhou Z, Jiao L, et al. Taming serverless cold start of cloud model inference with edge computing[J]. IEEE Transactions on Mobile Computing, 2023, 23(8): 8111-8128.

[14] Geng H, Zeng D, Li Y, et al. Plays: Minimizing dnn inference latency in serverless edge cloud for artificial intelligence-of-things[J]. IEEE Internet of Things Journal, 2024, 11(23): 37731-37740.

[15] 王书麟. 面向 Serverless 计算环境的 AI 推理高效数据编排方法与系统[D]. 南京: 南京大学, 2024.

[16] Fu Y, Xue L, Huang Y, et al. ServerlessLLM: Low-Latency serverless inference for large language models[C]//Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2024: 135-153.

[17] Zeng S, Xie M, Gao S, et al. Medusa: Accelerating Serverless LLM Inference with Materialization[C]//Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2025: 653-668.

[18] Hong Z, Lin J, Guo S, et al. Optimus: warming serverless ml inference via inter-function model transformation[C]//Proceedings of the 19th European Conference on Computer Systems (EuroSys). 2024: 1039-1053.

[19] Gallego A, Odyurt U, Cheng Y, et al. Machine Learning Inference on Serverless Platforms Using Model Decomposition[C]//Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing. 2023: 1-6.

[20] Lu C, Xu H, Li Y, et al. SMIless: Serving DAG-based Inference with Dynamic Invocations under Serverless Computing[C]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC). 2024: 38-54

[21] Yang Y, Zhao L, Li Y, et al. INFless: a native serverless system for low-latency, high-throughput inference[C]//Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2022: 768-781.

[22] Li B, Roy R B, Patel T, et al. Ribbon: cost-effective and qos-aware deep learning model inference using a diverse pool of cloud computing instances[C]//Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC). 2021: 24-36.

[23] He S, Feng B, Ding Z. Proactive Elastic Scheduling for Serverless Ensemble Inference

Services[C]//Proceedings of the IEEE International Conference on Web Services (ICWS). 2024: 1025-1035.

[24] Chen J, Xu F, Gu Y, et al. HarmonyBatch: Batching multi-SLO DNN inference with heterogeneous serverless functions[C]//Proceedings of the 32nd IEEE/ACM International Symposium on Quality of Service (IWQoS). 2024: 1-10.

[25] Choi Y, Kim J, Rhu M. ElasticRec: A Microservice-based Model Serving Architecture Enabling Elastic Resource Scaling for Recommendation Models[C]//Proceedings of the 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA). 2024: 410-423.

[26] Pei Q, Yuan Y, Hu H, et al. AsyFunc: A high-performance and resource-efficient serverless inference system via asymmetric functions[C]//Proceedings of the 2023 ACM Symposium on Cloud Computing (SoCC). 2023: 324-340.

[27] Zhao X, Yang S, Wang J, et al. FaPES: Enabling Efficient Elastic Scaling for Serverless Machine Learning Platforms[C]//Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC). 2024: 443-459.

[28] Gu J, Zhu Y, Wang P, et al. FaST-GShare: Enabling efficient spatio-temporal GPU sharing in serverless computing for deep learning inference[C]//Proceedings of the 52nd International Conference on Parallel Processing (ICPP). 2023: 635-644.

[29] Li Y, Lin Y, Peng S, et al. FLASH: Low-Latency Serverless Model Inference with Multi-Core Parallelism in Edge[C]//Proceedings of the 2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS). 2023: 2640-2646.

[30] KServe[EB/OL]. https://kserve.github.io/website/. Accessed: 2025-03-04.

[31] Seldon-core[EB/OL]. https://docs.seldon.io/projects/seldon-core/en/latest/. Accessed: 2025-03-04.

[32] Alluxio[EB/OL]. https://www.alluxio.com.cn. Accessed: 2025-03-04.

[33] Rombach R, Blattmann A, Lorenz D, et al. High-resolution image synthesis with latent diffusion models[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2022: 10684-10695.

[34] Zhang S, Roller S, Goyal N, et al. Opt: Open pre-trained transformer language models[J]. arXiv preprint arXiv:2205.01068, 2022.

[35] Cormode G, Muthukrishnan S. An improved data stream summary: the count-min sketch and

its applications[J]. Journal of Algorithms, 2005, 55(1): 58-75.

[36] Gou X, He L, Zhang Y, et al. Sliding sketches: A framework using time zones for data stream processing in sliding windows[C]//Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD). 2020: 1015-1025.

[37] Zhang Y, Goiri Í, Chaudhry G I, et al. Faster and cheaper serverless computing on harvested resources[C]//Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP). 2021: 724-739.

[38] 阿里云. 指定算力类别创建 ECI 实例[EB/OL]. https://help.aliyun.com/zh/eci/user-guide/specify-the-compute-category-to-create-an-instance. Accessed: 2025-02-25.

[39] 阿里云. ESSD 云盘[EB/OL]. https://help.aliyun.com/zh/ecs/user-guide/essds. Accessed: 2025-02-25.

[40] Gill P E, Wong E. Sequential quadratic programming methods[M]//Mixed integer nonlinear programming. Springer, New York, NY, 2012: 147-224.

[41] Wikipedia: Quadratic programming[EB/OL]. https://en.wikipedia.org/wiki/Quadratic_programming. Accessed: 2025-02-25.

[42] Boyd S P, Vandenberghe L. Convex optimization[M]. Cambridge university press, 2004.

[43] Wikipedia: Karush–Kuhn–Tucker conditions[EB/OL]. https://en.wikipedia.org/wiki/Karush%E2%80%93Kuhn%E2%80%93Tucker_conditions Accessed: 2025-02-25.

[44] Potra F A, Wright S J. Interior-point methods[J]. Journal of Computational and Applied Mathematics (CAM'00), 2000, 124(1-2): 281-302.

[45] Ferreau H J, Kirches C, Potschka A, et al. qpOASES: A parametric active-set algorithm for quadratic programming[J]. Mathematical Programming Computation (MPC), 2014, 6(4): 327-363.

[46] Scipy[EB/OL]. https://scipy.org/. Accessed: 2025-02-25.

[47] Neter J, Kutner M H, Nachtsheim C J, et al. Applied linear statistical models[J]. 1996.

[48] Hapke H, Nelson C. Building machine learning pipelines[M]. O'Reilly Media, 2020.

[49] Radzihovsky M, Memarian F, Can E, et al. Serving ml model pipelines on nvidia triton inference server with ensemble models[EB/OL]. https://developer.nvidia.com. Accessed: 2025-02-26.

[50] Nys thruway origin and destination points for all vehicles[EB/OL]. https://data.ny.gov/Transportation/NYS-Thruway-Origin-andDestination-Points-for-All-/em4e-ui5w. Accessd: 2025-02-26.

[51] Bipartie graph[EB/OL]. https://en.wikipedia.org/wiki/Bipartite_graph. Accessed: 2025-02-27.

[52] Kuhn H W. The Hungarian method for the assignment problem[J]. Naval research logistics quarterly, 1955, 2(1-2): 83-97.

[53] Maximum weight matching[EB/OL]. https://en.wikipedia.org/wiki/Maximum_weight_matching. Accessed: 2025-02-27.

[54] NetworkX[EB/OL]. https://networkx.org/. Accessed: 2025-02-25

[55] Kubernetes. Operator Pattern[EB/OL]. https://kubernetes.io/docs/concepts/extend-kubernetes. Accessed: 2025-03-05

[56] Mahgoub A, Wood P, Medoff A, et al. SOPHIA: Online reconfiguration of clustered NoSQL databases for Time-Varying workloads[C]// Proceedings of the 30th USENIX Annual Technical Conference (ATC). 2019: 223-240.

[57] Gu R, Zhang K, Xu Z, et al. Fluid: Dataset abstraction and elastic acceleration for cloud-native deep learning training jobs[C]//Proceedings of the 2022 IEEE 38th International Conference on Data Engineering (ICDE). 2022: 2182-2195.

[58] Gu R, Xu Z, Che Y, et al. High-Level Data Abstraction and Elastic Data Caching for Data-Intensive AI Applications on Cloud-Native Platforms[J]. IEEE Transactions on Parallel and Distributed Systems, 2023, 34(11): 2946-2964.

[59] Gadban F, Kunkel J. Analyzing the performance of the S3 object storage API for HPC workloads[J]. Applied Sciences, 2021, 11(18): 8540.

[60] Megiddo N, Modha D S. Outperforming LRU with an adaptive replacement cache algorithm[J]. Computer, 2004, 37(4): 58-65.

[61] Herodotou H, Kakoulli E. Automating distributed tiered storage management in cluster computing[J]. Proceedings of the VLDB Endowment, 2020, 13(1): 43-56.

[62] Ke G, Meng Q, Finley T, et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree[C]//Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS). 2017: 3146-3154.

[63] Shumway R H, Stoffer D S, Shumway R H, et al. ARIMA models[J]. Time series analysis and its applications: with R examples, 2017: 75-163.

致 谢

行文至此，恍然发觉三年时光已过，求学十九载，学生生涯已达终点。回望往昔，仿佛昨日刚刚提着行李箱初到南京，怀着期待、兴奋、紧张的心情踏入南京大学，研究生三年光景犹如一幕幕电影画面从脑海中闪过；驻足当下，一种闪回感扑面而来，我似乎又成了三年前青涩懵懂的自己，对人生的下一站又一次充满期待，只不过这一次身份有所转变，即将脱离学生的身份；放眼未来，我即将离开南京，再次提上行李箱踏上新一段旅途。在离开金陵古城之际，请允许我向这三年来所有给予我帮助与支持的人表达谢意。

我首先要感谢我的指导老师黄宜华教授。感谢黄老师给我加入课题组的机会，将我领进大数据与云计算领域的大门，为我提供良好的科研环境，并在研究方向上给予了高屋建瓴的指导，引领我完成研究生三年的研究工作。同时也感谢黄老师教导我们为人处世的道理，这些宝贵的教诲令我们受益终身。

特别感谢我的导师顾荣老师。首先感谢顾老师在科研上给我的帮助，帮我培养发现问题、解决问题、凝练表达能力。在论文指导方面，顾老师态度严谨认真，一次又一次详细深入的学术探讨，推动着我完成研究工作与毕业论文。在学术竞赛方面，感谢顾老师在“挑战杯”赛事上的精心指导，培养了我们梳理逻辑与凝练表达的能力，帮助我们收获了优异的成绩。

感谢实验室的小伙伴们，让我拥有了一段难忘的研究生时光。我要感谢王书麟师兄在大论文技术细节上的帮助与支持，感谢基国鼎师弟在小论文写作以及校企合作项目上的鼎力相助。我也要感谢同级的六位小伙伴，能与各位度过研究生三年时光是我的幸运，我们在科研工作上互相帮助，克服种种困难，在日常生活中分享快乐，共同成长。感谢顾老师实验室中所有的师兄师弟们，每一次相遇都是可贵的缘分。

感谢我在美团实习时的同事们，感谢你们对我工作成果的肯定以及帮助。特别感谢我的 mentor 赵志伟，在研发工作上给予我的帮助与指导，提高了我沟通需求、系统设计、工程开发的能力，并在未来职业发展上给我的中肯建议。

感谢我家乡朋友们一直的陪伴。我们分布在五湖四海，所以每一次在家乡的相遇都显得弥足珍贵，感激与你们共度的宝贵时光，是你们给我注入了精神能量。

感谢我求学路上遇到的所有的小学、初中、高中、本科老师。求学十九载，

如今已经画上了圆满的句号，正是你们的无私帮助与谆谆教导，造就了现在的我，让我以更昂扬的姿态望向前方新的道路。

最后，感谢我的父母，你们给了我生命，给了我幸福的家，你们是我最坚强的后盾，我无论遇到任何困难时，你们都会无私地帮助我、支持我，和我一起面对压力，感谢你们的陪伴。感谢我的女朋友刘萌，你陪我走过了硕士生涯，感谢你和我分享快乐、分担痛苦，你的出现让我更加确定了未来的路，让我坚定地选择我想要的生活。一路陪伴，三生有幸，未来有你。

求学生涯已经结束，但人生还有很长的路要走。离别之际，感谢在这条路上遇到的所有人。回首向来萧瑟处，归去，也无风雨也无晴。

附 录

研究期间申请的专利

[1] 顾荣，**陈启明**，基国鼎《一种面向云原生容器环境的AI模型推理缓存配置方法》（第一学生发明人，申请号：202510544057.6）

研究生期间参与的科研项目

[1] 南京大学-中国移动《跨域缓存弹性分配机制研究》前沿探索基金课题，项目起止年月：2024年8月~2025年8月，项目负责人为指导老师，本人为项目骨干

[2] 南京大学-中国中车《基于Fluid及容器技术的车载融合主机应用管控平台研究》，项目起止年月：2022年10月~2023年7月，项目负责人为指导老师，本人为项目骨干

研究生期间获得的奖项

[1] 第十八届“挑战杯”全国大学生课外学术科技作品竞赛全国一等奖，江苏省特等奖，鲍文杰、**陈启明**、王问骁等（排名第二）